

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Design and optimization of a custom QLoRA training engine for small language models on memory-constrained GPUs

Endika Blanco Cabana

Bilbao, junio de 2026

Grado en Ingeniería Informática

Informatikako Ingeniaritzako gradua

Proyecto fin de grado

Gradu amaierako proiektua

Design and optimization of a custom QLoRA training engine for small language models on memory-constrained GPUs

Endika Blanco Cabana

Bilbao, junio de 2026

Abstract

This project focuses on the design and development of a custom training engine for Small Language Models (SLMs), built for severely constrained consumer hardware such as an NVIDIA RTX 3060 Ti with an 8 GB VRAM budget. To work within this limit, the project implements a native PyTorch training loop from scratch. By avoiding high-level wrappers such as the Hugging Face Trainer, the architecture keeps direct control over execution phases, memory allocation and the computational graph. The core system combines native 4-bit quantization with a custom, dependency-free LoRA (Low-Rank Adaptation) injection mechanism, avoiding PEFT overhead and ensuring that gradients flow only through the low-rank adapters while the base model remains frozen.

The engine includes a precise memory management framework. In advanced AI training pipelines, especially reinforcement learning for LLMs, text generation and gradient calculation place different demands on video random-access memory (VRAM). To handle this, the engine uses strictly scoped generation phases with immediate key-value (KV) cache cleanup, gradient checkpointing and dynamic micro-batching. The memory manager can recover from out-of-memory (OOM) states and clear residual tensors between frozen log-probability precomputation and the backward pass.

The implementation is hardware-aware. It uses Ampere-specific optimizations such as TensorFloat-32 (TF32) matrix multiplications and native BFloat16 (BF16) precision, and explores custom Triton and CUDA kernels where they improve throughput. The framework includes tests and live profiling tools to monitor VRAM spikes. The expected result is a documented, testable engine that shows stable LLM training on commodity GPUs and provides infrastructure for memory-constrained AI research.

Descriptors

QLoRA, Small Language Models, PyTorch, VRAM Optimization, Memory-Constrained GPUs

Index

1. Introduction.....	1
1.1 Context and motivation.....	1
1.1.1 Evolution of LLMs and the need for efficient fine-tuning.....	1
1.1.2 Reasoning-oriented post-training.....	1
1.1.3 Hardware limitations of consumer GPUs.....	1
1.1.4 Gap between theoretical research and practical implementation.....	2
1.1.5 Personal motivation: bridging the gap between research and real constraints.....	3
1.2 Background and previous situation.....	4
1.3 Technical and economic justification.....	4
1.4 Project objectives.....	6
1.5 Scope and limitations.....	6
1.6 Structure of the memory.....	7
2. Methodology and planning.....	9
2.1 Integrated development methodology.....	9
2.2 Comprehensive validation methodology.....	12
2.3 Global planning.....	14
2.4 Task distribution by work area.....	18
2.5 Budget overview and cost distribution.....	21
3. Computational theory foundations.....	23
3.1 GPU architecture and CUDA programming model.....	23
3.2 Memory management in deep learning systems.....	25
3.3 Quantization and data representation.....	27
3.4 Computational bottlenecks of transformer architectures.....	28
4. State of the art.....	32
4.1 Single-GPU memory reduction techniques.....	32
4.2 Consumer-GPU fine-tuning stacks.....	34
4.3 Optimized kernels and runtime paths.....	36
4.4 Gap in current tooling for local post-training.....	37
5. System architecture and development.....	39
5.1 System architecture design.....	39
5.1.1 Architectural objectives and global structure.....	39
5.1.2 Phase boundaries and data flow.....	41
5.1.3 Reuse across training, benchmarking, and profiling.....	43
5.2 Architectural design decisions.....	43
5.2.1 Native control over framework abstraction.....	43
5.2.2 Manual LoRA and adapter visibility.....	43
5.2.3 Memory control as an architectural constraint.....	44
5.2.4 Why the design stays narrow.....	44

5.3 Core engine components.....	45
5.3.1 Model loading and quantized startup.....	45
5.3.2 Adapter path and manual LoRA layers.....	46
5.3.3 Memory supervision during execution.....	46
5.3.4 Trainer as coordination center.....	46
5.3.5 Checkpointing, recovery, and export.....	47
5.3.6 Dependency order and centralization.....	47
5.4 Configuration, logging and operational support.....	48
5.4.1 Configuration as execution contract.....	48
5.4.2 Logging and runtime observability.....	48
5.4.3 Recovery and profiler integration.....	49
5.4.4 Operational baseline and reproducibility.....	49
5.5 Testability as a design requirement.....	50
5.5.1 Why testability shapes the design.....	50
5.5.2 Coverage across components and execution modes.....	50
5.5.3 Regression protection for architecture-critical paths.....	51
6. Memory optimization and VRAM management.....	52
6.1 Memory consumption analysis.....	52
6.1.1 RTX 3060 Ti (8 GB) VRAM budget.....	52
6.1.2 Component-level footprint: model, activations, gradients, optimizer, KV cache.....	52
6.1.3 Critical operating margin near the OOM boundary.....	55
6.2 Quantized backbone and precision strategy.....	56
6.2.1 4-bit NF4 loading in the runtime.....	56
6.2.2 Double quantization: savings and added cost.....	58
6.2.3 Storage precision vs compute precision in fine-tuning.....	58
6.3 Gradient checkpointing and VRAM-driven activation policy.....	59
6.3.1 Computation-memory trade-off.....	59
6.3.2 Selective checkpointing in practice.....	60
6.3.3 vram_auto thresholds, update interval, and activation logic.....	60
6.4 Memory manager and runtime release policies.....	62
6.4.1 Cache clearing, fragmentation control, and peak tracking.....	62
6.4.2 CPU-backed intermediate state and staged GPU transfer.....	63
6.4.3 Aggressive recovery and memory-pressure responses.....	64
6.5 Phase-aware OOM recovery.....	64
6.5.1 Detecting failure by phase.....	64
6.5.2 Independent generation and training micro-batch backoff.....	65
6.5.3 Replay-safe retries, regrowth probes, and resume consistency.....	66

7. Training pipeline implementation.....	75
7.1 End-to-end pipeline structure.....	75
7.2 Generation phase.....	77
7.3 Pre-computation phase.....	78
7.4 Training and update phase.....	79
7.5 Optional pipeline enhancements.....	80
8. Performance engineering, Triton, and benchmark-driven optimization.....	82
8.1 Performance-analysis methodology.....	82
8.2 Bottleneck identification in the native engine.....	84
8.3 Triton as a selective acceleration strategy.....	87
8.3.1 Why Triton was introduced at all.....	87
8.3.2 Why Triton was not treated as a universal answer.....	87
8.3.3 Why selective acceleration became the final policy.....	88
8.4 Custom triton kernels in the project.....	88
8.4.1 Kernel families and engineering intent.....	88
8.4.2 Fused LoRA execution.....	89
8.4.3 Fused GRPO loss.....	90
8.4.4 Entropy-mask kernels.....	90
8.4.5 Paged-KV generation support.....	91
8.4.6 Fallbacks, input contracts, and practical maintainability.....	92
8.5 Correctness validation before performance acceptance.....	93
8.6 Benchmark-driven optimizer design.....	94
8.7 Optimizer workflow in practice.....	96
8.8 Final performance findings.....	97
9. Operational results, incidents, and engineering lessons.....	100
9.1 Final operating configuration.....	100
9.2 Stability, recovery, and the practical operating envelope.....	102
9.3 Main incidents and what they revealed.....	104
9.3.1 Feasibility before optimization.....	104
9.3.2 Making sampled responses usable for training.....	105
9.3.3 Repeated OOM pressure as a design condition.....	106
9.3.4 High step times and generation dominance.....	107
9.3.5 Broader dataset support and workload fit.....	108
9.3.6 Profiling and Triton uncertainty.....	108
9.4 Lessons for local native RL training.....	109
10. Ethics.....	111
10.1 Ethical issues in local post-training.....	111
10.2 Alternatives, decisions, and justification.....	112
10.3 Application to the engineering work.....	113

10.4 Use of AI assistance and academic integrity.....	114
10.5 Ethical assessment.....	115
11. Conclusions, limitations, and future work.....	116
11.1 Engineering contributions and verified outcomes.....	116
11.2 Bounded claims, limitations, and threats to validity.....	119
11.3 Future work prioritized by expected value.....	122
12. Bibliography.....	124

Index of figures

Figure 1.1: Motivation chain of the project.	10
Figure 2.1: Integrated development methodology.	17
Figure 2.2: Integrated 600-hour FDP planning baseline.	24
Figure 2.3: Integrated FDP task network.	28
Figure 3.1: Roofline model and measured decoder performance on constrained hardware.	30
Figure 3.2: GPU memory hierarchy from device memory to registers.	32
Figure 3.3: CUDA grid, block, warp, and thread hierarchy.	32
Figure 3.4: Full finetuning, LoRA, and QLoRA memory paths.	35
Figure 3.5: Decode workload trapped under the memory-bandwidth roofline.	37
Figure 4.1: Tiled IO-aware attention compared with the naive attention path.	44
Figure 5.1: GRPO training engine architecture.	47
Figure 6.1: Response-cap trade-offs across the preserved evidence.	63
Figure 6.2: Precision split used by the runtime.	64
Figure 6.3: vram_auto checkpointing hysteresis in the accepted configuration family.	68
Figure 7.1: Explicit three-phase training pipeline in the local runtime.	77
Figure 7.2: SENT curriculum staging as used by the runtime.	81
Figure 8.1: Benchmark-driven promotion workflow used for performance decisions.	84
Figure 8.2: Bottleneck structure in the native engine.	86
Figure 8.3: Triton kernel families and final acceptance status.	89
Figure 8.4: Benchmark-governed optimizer loop.	96
Figure 8.5: Compare-bench snapshot for two nearby Triton-on commits.	98
Figure 9.1: OOM-recovery timeline from the preserved 1024 response-length run.	106

Index of tables

Table 1.1: Hardware regime, engineering stance, and main exclusions.....	5
Table 2.1: Methodological evidence chain.....	12
Table 2.2: Formal 600-hour planning baseline.....	15
Table 2.3: Planned work allocation by activity block.....	17
Table 2.4: Task distribution by work area.....	19
Table 2.5: Human-resource cost distribution.....	22
Table 2.6: Itemized project budget.....	22
Table 3.1: Main memory-pressure sources and trade-offs in constrained transformer training.....	27
Table 3.2: Prefill and decode bottlenecks in decoder-only execution.....	29
Table 4.1: Memory-reduction intervention families for constrained training.....	33
Table 4.2: Consumer-GPU fine-tuning stacks and their local-control trade-offs.....	35
Table 5.1: Core architectural responsibilities of the training engine.....	41
Table 5.2: Phase boundaries and memory implications in one training step.....	42
Table 5.3: Scope boundaries and protected engineering properties.....	45
Table 5.4: Test coverage aligned with architectural boundaries.....	51
Table 6.1: Response-cap regimes, speed, truncation pressure, and recovery behavior.....	54
Table 6.2: Accepted memory-control surfaces.....	62
Table 6.3: Phase-aware OOM recovery contract.....	67
Table 7.1: Three-phase training-step contract.....	69
Table 8.1: Selective Triton acceptance outcomes.....	86
Table 9.1: Accepted operating profile and operational purpose.....	94
Table 9.2: Practical operating regimes near the memory boundary.....	96
Table 10.1: Ethical alternatives, adopted decisions, and justifications.....	106
Table 11.1: Verified engineering contributions and claim boundaries.....	111
Table 11.2: Main limitations and their effect on the conclusions.....	113

1. INTRODUCTION

1.1 CONTEXT AND MOTIVATION

1.1.1 Evolution of LLMs and the need for efficient fine-tuning

Large language models (LLMs) have moved from general-purpose text generation toward tasks that require more structured reasoning, such as mathematics, programming, and step-by-step problem solving. This shift made post-training more important, because the usefulness of a model increasingly depends on how it is adapted after pretraining and how reliably it can follow a reasoning process.

The project started from that technical shift, but also from a more practical concern. Using an already trained reasoning model through a library or an interface gives little visibility into the post-training process itself. Running that process locally is different: the training loop has to be executed, measured, and debugged under constraints that cannot be hidden. Reasoning-oriented work such as DeepSeek-R1 made reinforcement learning for reasoning concrete enough to study seriously [1]. The question that mattered here was what part of that idea could be made inspectable under ordinary hardware constraints.

1.1.2 Reasoning-oriented post-training

Reinforcement learning is relevant because it allows a model to be optimized from feedback about its behavior, not only from examples of desired text. In reasoning tasks, that feedback can often be tied to verifiable outcomes: a mathematical answer is either correct or incorrect, a format is either respected or broken, and a generated reasoning trace can be evaluated against explicit criteria.

Group Relative Policy Optimization (GRPO) was selected because it fits this setting while avoiding part of the complexity of older actor-critic pipelines. DeepSeekMath introduced GRPO as a group-relative alternative to Proximal Policy Optimization (PPO)-style value-model training for mathematical reasoning [2] [3]. Instead of centering the project on a large auxiliary value model, the training loop can be organized around groups of sampled responses and their relative rewards. That choice kept the algorithmic idea close to current reasoning-model practice while making the implementation more realistic for a single local graphics processing unit (GPU). GRPO gave the project a feasible post-training target rather than only a theoretical one.

1.1.3 Hardware limitations of consumer GPUs

The project is defined by a hard hardware constraint: a single NVIDIA RTX 3060 Ti with 8 GB of video random-access memory (VRAM). That limit was not treated as an incidental inconvenience. It

determined the model size, the training loop, the memory policy, the validation strategy, and the kind of engineering contribution that could be made.

Consumer GPUs are attractive because they are accessible, but their memory ceiling is unforgiving. Even parameter-efficient fine-tuning remains sensitive to sequence length, batch size, optimizer state, and precision choices on this class of hardware [4]. Reinforcement learning adds more pressure because generation, cached probabilities, activations, rewards, and repeated updates all compete for the same memory budget. The project therefore could not rely on simply scaling the hardware. The engineering problem was to make the available card sufficient through explicit control.

1.1.4 Gap between theoretical research and practical implementation

The distance between research descriptions and local execution was the main practical gap behind the work. Many training recipes are easy to describe at a high level: load a model, generate responses, compute rewards, update the policy, and evaluate the result. The difficulty appears when those steps must run repeatedly on a small card without losing visibility over memory, timing, and failure modes.

That gap made the project an engineering exercise rather than a simple application of an existing training script. The implementation had to expose the phases that usually remain hidden inside high-level frameworks: generation, frozen-policy evaluation, adapter updates, checkpointing, cache release, and recovery after memory pressure. Large-scale training could not be reproduced exactly on this hardware. The project therefore asks which parts of reasoning-oriented post-training can still be built, controlled, and validated when rollout length, model size, optimizer state, and debugging time are all constrained at the same time.

Figure 1.1 summarizes the conceptual constraint chain behind that question. The arrows do not describe the runtime order of the system; they show why the project moves from reasoning-model post-training to rollout and memory pressure, then to a local hardware constraint, and finally to an implementation strategy based on explicit memory control.

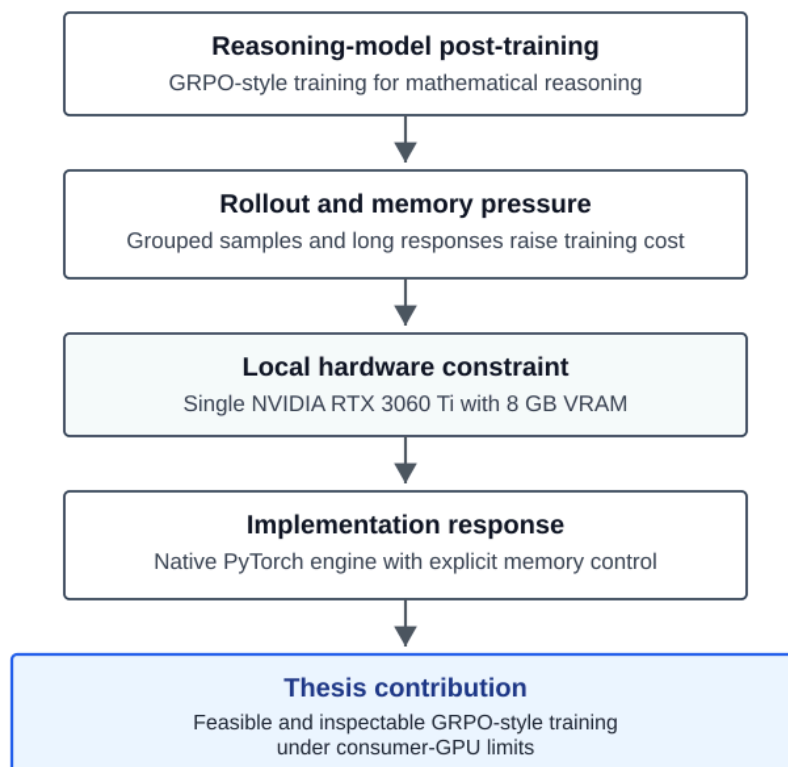


Figure 1.1: Motivation chain of the project.

1.1.5 Personal motivation: bridging the gap between research and real constraints

The personal motivation was to move beyond using pretrained models as black boxes. I wanted to understand how a reasoning-oriented post-training loop behaves when the system is built from the ground up and forced to run inside real limits. That meant accepting a smaller scale, but it also made the work more concrete: every memory spike, slow phase, unstable response, or failed run had to be explained through code and measurements rather than hidden behind an external service.

The implementation is meant to be studied end to end. Its contribution comes from that control: knowing when the model fits, why it stops fitting, how the runtime reacts near the out-of-memory (OOM) boundary, and which engineering decisions make local experimentation possible.

1.2 BACKGROUND AND PREVIOUS SITUATION

The previous situation combined useful tooling with a persistent lack of control near the hardware limit. High-level libraries already made many parts of local model adaptation accessible: model loading, adapter injection, low-bit execution, training orchestration, and evaluation scripts. That ecosystem lowered the entry barrier, but it did not remove the systems problem created by strict VRAM limits.

Several mature components were available before this project began:

- Transformer and trainer libraries provided standard model loading and training abstractions.
- Adapter-based methods made it possible to tune a small number of trainable parameters instead of the full model.
- Low-bit loading reduced the memory footprint of the frozen backbone.
- RL-oriented tooling made reward-based training easier to launch.
- Low-level tools opened the door to targeted performance work on selected execution paths.

The remaining difficulty was not the absence of tools. It was the lack of direct visibility and control when all parts of the loop had to coexist on an 8 GB card. A GRPO-style run needs generation, reward computation, reference or frozen-policy quantities, adapter updates, checkpointing, logging, and evaluation. Each phase may be manageable on its own, while the full loop can still fail because tensors live too long, caches are not released at the right time, or retry behavior is too coarse.

This starting point defined the project. The aim was not to replace the whole ecosystem or build a general-purpose training platform. The aim was to implement a focused training engine where memory-critical phases could be controlled directly, profiled with enough detail, and adapted when the run approached the OOM boundary.

1.3 TECHNICAL AND ECONOMIC JUSTIFICATION

Given those conditions, a native PyTorch implementation is a technical requirement. If the goal is to inspect and control each major phase of GRPO-style training on a single local GPU, then the training loop must remain exposed rather than being hidden behind several orchestration layers. A native implementation makes it possible to separate generation from frozen-policy computation, release intermediate tensors deliberately, control when Low-Rank Adaptation (LoRA) adapters are active, and introduce explicit recovery behavior when memory pressure exceeds the safe margin. It also makes profiling more meaningful, because the observed bottlenecks correspond directly to code that can be changed.

There is also a clear economic argument behind the same decision. Frontier reasoning-model training is normally associated with long runs, cluster-scale infrastructure, and costs that place systematic

experimentation outside the reach of most individual students or small teams. Efficient compact-model work suggests that this barrier can be reduced without making the research trivial [5]. Optimizing for a local RTX 3060 Ti therefore lowers direct financial cost and the dependence on rented infrastructure, which matters in a project built around repeated profiling, modification, and controlled reruns in the same environment.

The accessibility argument follows from the same constraint. If reasoning-model post-training can only be explored by teams with large compute budgets, then the range of testable ideas is narrowed in advance. A local implementation cannot match datacenter-scale training, but it can make the mechanics of post-training visible, reproducible, and easier to inspect. That is the level at which this project contributes: by turning a constrained machine into a controlled experimental environment.

Table 1.1 condenses the hardware regime, engineering stance, and main exclusions that the rest of the memory develops in detail.

Aspect	Project decision
Target hardware	Single NVIDIA RTX 3060 Ti with 8 GB VRAM
Base model family	DeepSeek-R1-Distill-Qwen-1.5B
Training style	Native PyTorch GRPO-style post-training
Feasibility strategy	4-bit NormalFloat 4 (NF4) backbone, manual LoRA, bounded rollout groups, explicit memory cleanup
Main systems priorities	VRAM control, throughput, phase visibility, OOM avoidance and recovery
Core operating assumption	Single-GPU local execution near the memory boundary
Out of scope	Multi-GPU, distributed training, cloud deployment, general-purpose framework coverage

Table 1.1: Hardware regime, engineering stance, and main exclusions.

1.4 PROJECT OBJECTIVES

The work is organized around four engineering objectives:

- **O1. Implement a native PyTorch GRPO-style training engine for consumer hardware.** The main objective is to design and implement a training loop capable of running GRPO-style post-training on a single consumer GPU under a strict 8 GB VRAM constraint. The execution phases must remain explicit, inspectable, and controllable so that memory behavior, performance bottlenecks, and failure modes can be understood at system level.
- **O2. Improve end-to-end computational efficiency.** Once feasibility is established, the system must reduce avoidable overhead in throughput and training step time. Efficiency is not reduced to raw speed: a faster isolated kernel is not useful if it destabilizes generation or makes long runs slower overall. The target is end-to-end efficiency, supported by bounded benchmarking, profiling, and selective low-level optimization.
- **O3. Manage limited GPU memory robustly.** The system must minimize persistent memory use through quantization and low-rank adaptation, control tensor lifetime between phases, reduce unnecessary cache retention, and avoid catastrophic OOM failures through bounded operating policies. Since OOM is part of the expected environment near the hardware boundary, the target is not only to fit, but to remain usable under pressure.
- **O4. Provide fine-grained runtime control over training behavior.** The runtime must expose direct control over manual LoRA integration, checkpointing as a configurable memory-performance trade-off, and memory policies adapted to the different phases of RL training. Reimplementing these features allows direct management of memory allocation and gradient timing, and gives more transparency than standard high-level libraries during high-demand tasks.

1.5 SCOPE AND LIMITATIONS

The scope includes the components required to execute and study GRPO-style training on the target hardware: optimized loading of the selected 1.5B base model, 4-bit quantization of the frozen backbone, manual LoRA injection, explicit memory-management routines, the training orchestrator, checkpoint and resume mechanisms, structured logging, benchmarking support, and profiling tools for observing VRAM behavior and runtime hotspots.

Distributed training, multi-GPU parallelization, and cloud deployment remain outside the scope of the project. The same applies to turning the system into a general-purpose platform for arbitrary hardware tiers or production inference services. These exclusions keep the work centered on its main systems question: how much can be achieved through native control on one constrained local machine.

The hardware constraint is both a boundary and a methodological premise. The system is designed around a single RTX 3060 Ti with 8 GB of VRAM, and major architectural choices are evaluated against that limit. This affects model selection, rollout size, response length, precision mode, checkpointing strategy, and the acceptance criteria used during benchmarking. The project therefore does not claim hardware independence; its engineering decisions are intentionally specialized to a narrow consumer-GPU regime.

The final limitation is development time. Building a native reinforcement-learning training engine, validating its correctness, and profiling it under constrained hardware creates a substantial workload before long experimental campaigns are even considered. The project prioritizes the engineering core: system feasibility, runtime stability, profiler-backed analysis, and benchmark-backed configuration decisions. Broader kernel specialization, deeper low-level kernel rewrites, and support for larger hardware targets remain future-work paths rather than current deliverables.

1.6 STRUCTURE OF THE MEMORY

The memory is organized as follows:

- Chapter 1 introduces the project context, motivation, technical justification, objectives, scope, and limitations.
- Chapter 2 presents the methodology, planning, validation approach, workload distribution, and budget baseline.
- Chapter 3 develops the computational foundations needed to understand the system, including GPU execution, memory management, quantization, and transformer bottlenecks.
- Chapter 4 reviews the state of the art in constrained LLM training optimization, covering memory-reduction techniques, consumer-GPU fine-tuning stacks, optimized runtime paths, and the remaining tooling gap.
- Chapter 5 describes the system architecture, design decisions, core engine components, configuration structure, logging, checkpointing, and testability requirements.
- Chapter 6 focuses on memory optimization and VRAM management, including the quantized backbone, LoRA strategy, tensor lifetime control, OOM handling, and profiling.
- Chapter 7 explains the training pipeline, including rollout generation, reward computation, loss evaluation, optimization, checkpointing, and recovery behavior.
- Chapter 8 presents performance engineering, benchmark-driven optimization, Triton-related work, and the evaluation of selected low-level improvements.
- Chapter 9 reports operational results, incidents, limitations observed during execution, and the engineering lessons derived from the experimental runs.
- Chapter 10 discusses ethical, social, environmental, professional, and sustainability considerations.
- Chapter 11 summarizes the conclusions, the degree of objective fulfillment, the main limitations, and future work.

- Chapter 12 contains the bibliography used throughout the memory, including the research papers, technical documentation, and institutional sources that support the theoretical, engineering, and methodological claims.

2. METHODOLOGY AND PLANNING

2.1 INTEGRATED DEVELOPMENT METHODOLOGY

The project was developed through an integrated methodology that combined software-engineering iteration with a data-science workflow oriented to reinforcement learning experimentation. This structure followed directly from the nature of the work. The system had to be implemented as a native PyTorch training engine with explicit control over quantization, LoRA injection, memory cleanup, checkpointing, and OOM recovery, but the same codebase also had to support an experimental process centered on verifiable mathematical reasoning tasks, GRPO-style optimization, deterministic reward verification, curriculum learning, and benchmark-backed validation. Grade School Math 8K (GSM8K) served as the initial reference task, while later work generalized the data path to broader math datasets.

For that reason, the project did not follow a linear waterfall sequence. It advanced through short iterative cycles in which a hypothesis or requirement was implemented, tested under the target hardware constraint, measured, and then refined. In practice, many tasks returned repeatedly to earlier stages. A change in the training loop could require new checks in the GRPO implementation, new profiler runs to inspect VRAM behavior, and new benchmark comparisons to determine whether the modification represented a real improvement or only a local optimization with hidden regressions. The practical reference on the engineering side was an Agile style of work based on incremental implementation, fast feedback, and continuous correction. On the experimental side, the workflow followed the logic of the Cross-Industry Standard Process for Data Mining (CRISP-DM): problem framing, task and data definition, iterative modeling, bounded evaluation, and interpretation of results. These stages were not isolated. They overlapped throughout development.

Figure 2.1 summarizes this integrated process. The diagram shows how the Agile engineering cycle and the CRISP-DM data-science cycle were joined in practice: shared framing led to an engineering increment, a data-science modeling step, integrated validation, joint interpretation, and a decision point that either triggered a new iteration or consolidated the next methodological baseline.

Integrated Development Methodology

How Agile engineering iteration and CRISP-DM data-science workflow were combined in one development cycle

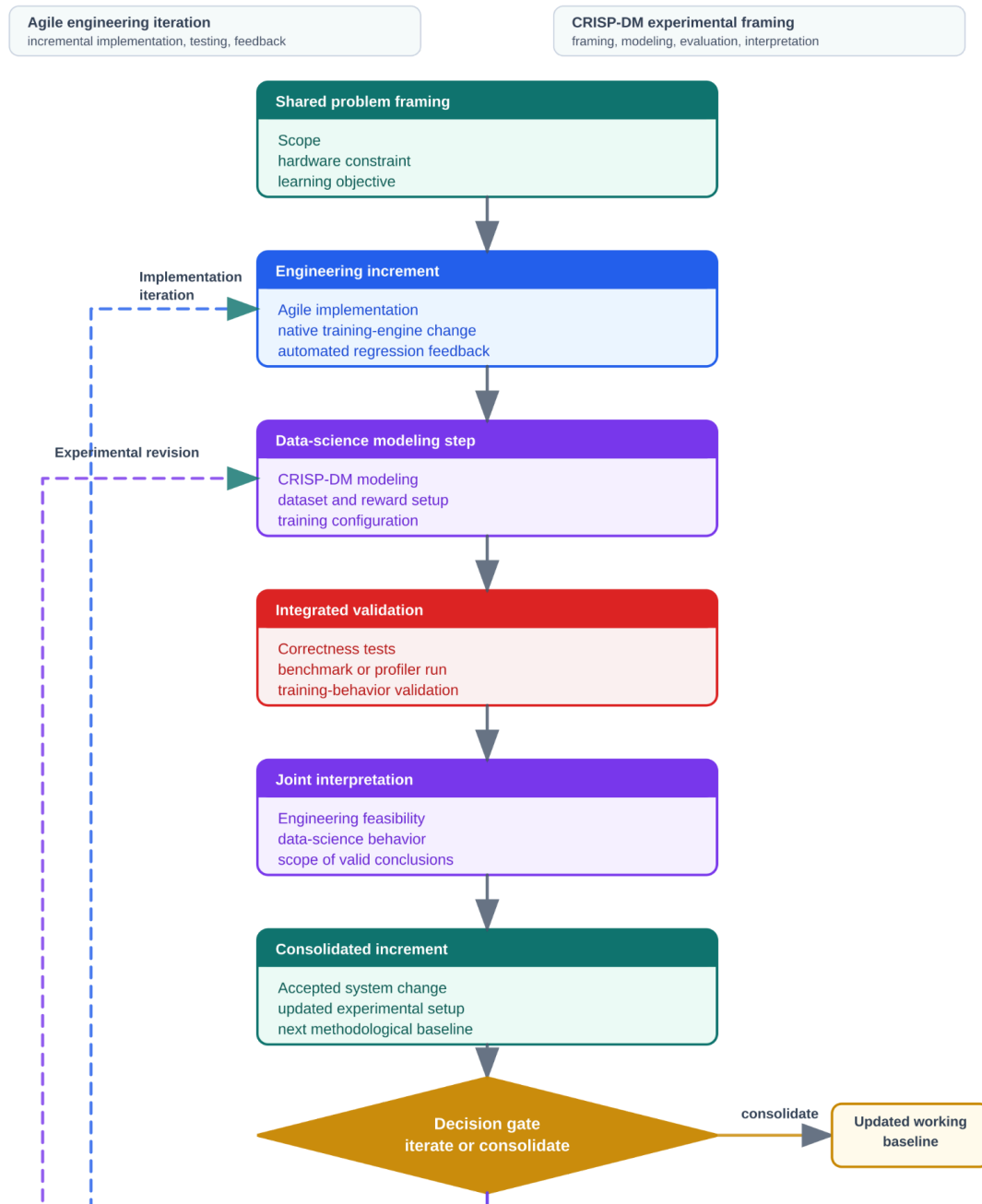


Figure 2.1: Integrated development methodology.

Version control was treated as a methodological tool rather than a simple backup mechanism. Development was organized around Git and GitHub with a branch-based workflow centered on main, develop, and feature branches. This structure mattered because the training engine contained

low-level changes that could introduce subtle regressions in memory behavior, generation logic, or optimizer-state handling, and because experimental work often required temporary branches for trying kernels, runtime policies, or benchmarking workflows before promoting them to the main development line. The project log history preserved a clear sequence of implementation phases, from the initial native GRPO engine through Semantic-Entropy Curriculum Learning (SENT) integration, memory-safety improvements, profiler development, Triton experimentation, packing and checkpointing changes, optimizer automation, benchmark-suite consolidation, and final preset selection. That sequence supported both software maintenance and academic traceability, because each implementation phase could be tied back to a concrete state of the codebase.

Experimental tracking also relied on several complementary mechanisms. During training, the system generated structured logs for runtime metrics and debugging. When enabled, Weights & Biases preserved convergence-related signals and run metadata. In parallel, the benchmark workflow produced repeatable command-line suites with machine-readable outputs such as JSON, CSV, and JSONL records. Configuration choices, benchmark outputs, intermediate reports, and final LoRA weights were all treated as reproducible artifacts tied to a concrete code state. That discipline reduced the risk of relying on isolated console outputs or handwritten observations when comparing runs.

The same methodological rigor was used in testing and validation. The project adopted a correctness-first philosophy because failures in a low-level reinforcement-learning training engine are expensive to diagnose and easy to misinterpret. An apparently minor change in generation, entropy masking, gradient accumulation, or checkpointing can alter the whole learning process or make a benchmark incomparable with earlier runs. For that reason, the automated test suite was organized around several layers:

- Unit tests for mathematically sensitive or structurally delicate components, including the GRPO loss, clipping behavior, reward handling, memory management, and data preprocessing.
- Integration tests for the trainer loop, checkpoint-resume behavior, Kullback-Leibler (KL) and reference passes when applicable, SENT stage transitions, and OOM retry handling.
- Specialized tests for Triton correctness, profiler behavior, benchmark logic, optimizer promotion gates, and hardware-sensitive fallbacks.
- Direct numerical checks for the mathematical behavior of the GRPO objective, so that the loss was verified through reproducible calculations rather than informal inspection alone.

Within that framework, the broader trainer behavior was validated through reproducible integration scenarios before the corresponding runs were interpreted as evidence.

Table 2.1 makes the methodological evidence chain explicit.

Method layer	Concrete project mechanism	What it validates
Iterative implementation	branch-based Git workflow on main, develop, and feature branches	that algorithm and systems changes remained attributable and reversible
Correctness-first testing	unit, integration, verifier, benchmark, profiler, and SENT tests	that sensitive mathematical and pipeline contracts were checked before interpreting runs
Experimental tracking	structured runtime logs plus metrics.jsonl, config.json, and benchmark reports	that comparisons relied on preserved machine-readable artifacts rather than console impressions
Benchmark authority	bounded suites with fixed trainer-step budgets and saved outputs	that shortlist and finalist decisions came from comparable runs
Statistical caution	repeated-seed finalist validation and explicit claim boundaries	that methodological rigor remains separate from overclaiming

Table 2.1: Methodological evidence chain.

2.2 COMPREHENSIVE VALIDATION METHODOLOGY

Because the project was constrained to a single consumer GPU, validation began with feasibility rather than with final accuracy alone. The first question was whether the complete reinforcement-learning pipeline could run, remain stable, and preserve enough headroom to avoid frequent catastrophic failures. The methodology therefore tracked several validation dimensions:

- Runtime performance, especially throughput and step time.
- Response behavior, including response length and truncation rates.

- Memory behavior, including peak VRAM, average VRAM, allocator pressure, and phase-level hotspots.
- Runtime stability, including OOM-recovery events and behavior under longer diagnostic runs.
- Benchmark comparability, using standardized outputs across configurations.

Runtime logging recorded the operational metrics of the trainer, benchmark suites produced standardized outputs, and a dedicated profiling stack inspected memory hotspots and phase-level behavior. Under an 8 GB budget, performance and stability could not be separated. A configuration that looked faster in a short window but produced frequent truncation or OOM backoff in a longer diagnostic run could not be accepted as a reliable operating point.

System validation followed an evidence hierarchy:

1. The relevant regression tests had to pass.
2. The modified system had to survive the benchmark workflow without breaking comparability conditions.
3. The resulting metrics had to show a measurable advantage in throughput, step time, or stability.

This prevented the project from treating isolated profiler gains as sufficient proof of improvement. The benchmark methodology served as the bridge between implementation work and experimental conclusions. Rather than comparing configurations informally, the project used repeatable suites and control scripts to standardize inputs, consolidate metrics, and determine whether a candidate configuration could replace the current baseline.

Model evaluation initially used GSM8K because it offered a compact, verifiable reasoning task aligned with the scope of the project. The same evaluation contract was then kept for the broader normalized math-dataset path through three checks:

- Final-answer correctness, verified through deterministic comparison with the target answer.
- Compliance with the expected response format, so that evaluation checked more than whether the right number appeared somewhere in the response.
- Reward and training dynamics across runs, including variance, outliers, instability, and temporal evolution.

The rule-based verifier fit the hardware constraint because it avoided the need for a separate learned reward model while preserving clear reward semantics. Format compliance still mattered because the generated answer had to respect the structure required by the task and the verifier. Reward behavior also required separate analysis, since sparse binary rewards and group-relative comparisons can generate unstable dynamics even when the implementation itself is technically correct.

2.3 GLOBAL PLANNING

The project was planned around a global workload of 600 hours, divided into 300 hours for Computer Engineering work and 300 hours for Data Science work. This baseline served as the reference framework for the schedule, the task distribution, and the budget model. It does not imply that the work happened in a perfectly sequential order. In practice, implementation, debugging, benchmarking, and writing overlapped repeatedly.

For planning purposes, the work was organized into four internal phases that describe the project clearly from a technical point of view: foundation, core implementation, optimization and validation, and documentation and review. The foundation phase covered problem definition, scope delimitation, theoretical orientation, feasibility analysis, and setup of the development environment. The core implementation phase concentrated most of the engineering effort: native training-loop construction, GRPO and reward logic, memory-management mechanisms, benchmarking infrastructure, and the main feature additions. The optimization and validation phase focused on profiler analysis, Triton integration, benchmark comparison, and methodological checks. Finally, the documentation and review phase addressed the preparation of the two memory documents, final consolidation of results, and defense preparation.

The formal baseline is summarized in Table 2.2.

Internal phase	Planning block	Weeks	Computer Engineering hours	Data Science hours	Total hours
Foundation	Definition and initial planning	1-4	70	70	140
Core implementation	Development	5-10	170	110	280
Optimization and validation	Evaluation and consolidation	11-14	40	80	120
Documentation and review	Final review and writing	15-16	20	40	60

Total	Full project plan	1-16	300	300	600
--------------	--------------------------	-------------	------------	------------	------------

Table 2.2: Formal 600-hour planning baseline.

The project schedule is summarized in Figure 2.2. The Gantt chart expands the formal 600-hour planning baseline into the concrete work packages used by both FDPs, while preserving the five-stage structure expected for the project plan.

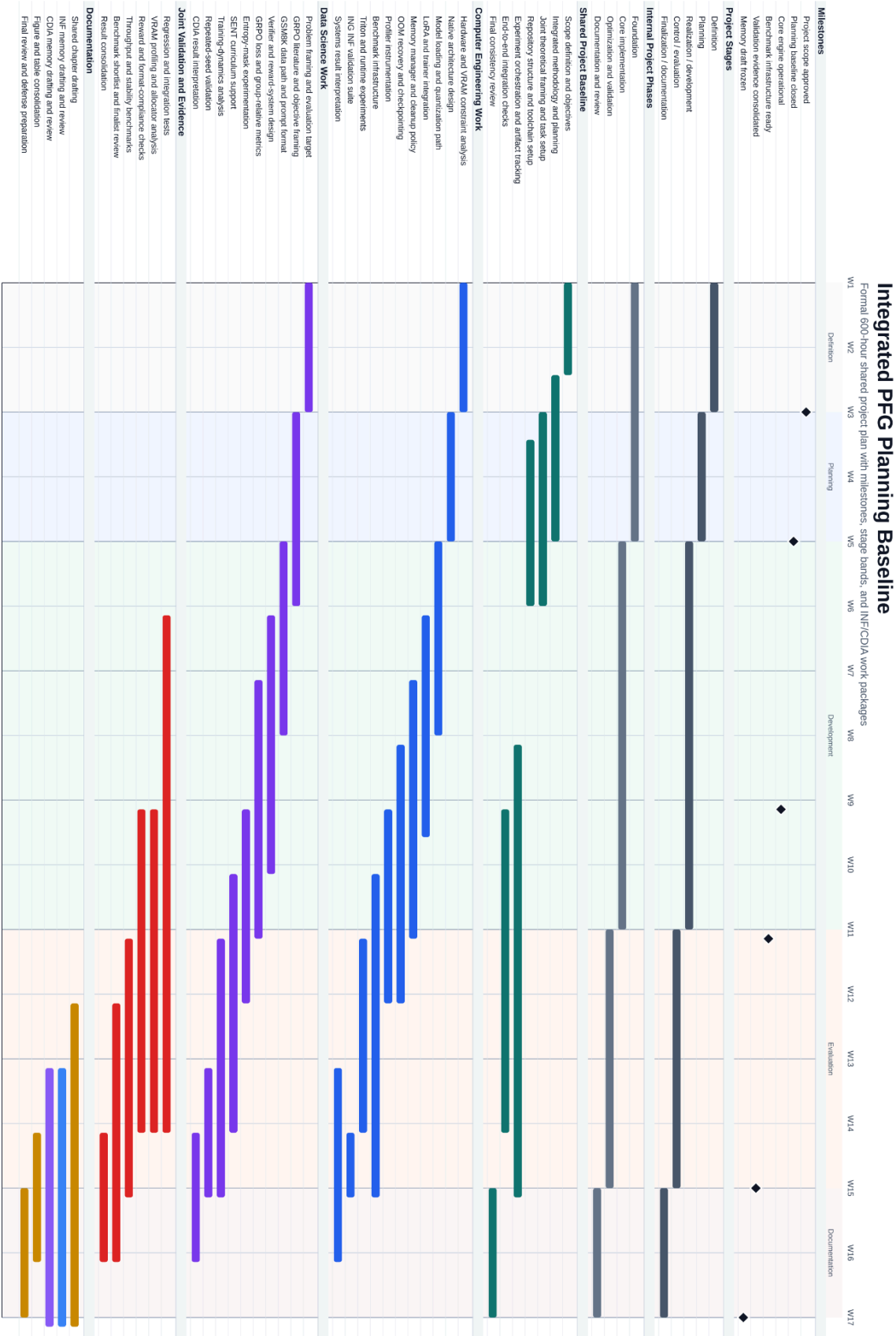


Figure 2.2: Integrated 600-hour FDP planning baseline.

In addition to the phase-based planning, the project was distributed by activity type in order to reflect the real balance between implementation and experimental work. The resulting allocation is shown in Table 2.3.

Activity block	Hours	Share
Implementation	210	35%
Profiling and optimization	120	20%
Experimental runs and validation	150	25%
Analysis and documentation	120	20%
Total	600	100%

Table 2.3: Planned work allocation by activity block.

This distribution reflects the character of the project. The dominant effort was technical construction and controlled validation. The implementation block covered the native training engine, memory manager, reward pipeline, benchmark tooling, and associated infrastructure. Profiling and optimization appeared as a distinct block because, under an 8 GB VRAM constraint, performance analysis and memory-behavior inspection were not optional refinements. Experimental runs and validation occupied a quarter of the total plan because each major claim required bounded benchmark execution, artifact review, and regression checking. The remaining time covered interpretation, chapter writing, synthesis of results, and final review.

The project also required an explicit critical-path interpretation:

1. The theoretical and methodological dependency came first: the GRPO formulation, the target task, the reward structure, and the system requirements had to be clear before implementation could converge.

2. The technical dependency followed: core engine components such as model loading, LoRA injection, memory management, and the trainer loop had to exist before useful benchmarks or quality diagnostics could be run.
3. The evaluative dependency came next: benchmark suites, structured logging, and validation tests had to mature before configuration decisions could be treated as final.
4. Only after those stages could the documentation be written with enough confidence to distinguish between safe claims and still pending ones.

The main bottleneck of the critical path was experimental time. Even when the implementation was correct, long-run reinforcement-learning experiments and controlled reruns on a single consumer GPU imposed a hard wall-clock limit. For this reason, the plan included implicit buffers for reruns, debugging after OOM incidents, and re-benchmarking after sensitive changes. In practice, that meant the project could not assume that every planned experiment would succeed on the first attempt or that every apparent optimization would survive the full validation workflow.

2.4 TASK DISTRIBUTION BY WORK AREA

Computer Engineering work concentrated on the system-level responsibilities of the project. These tasks included the design of the native architecture, management of the constrained VRAM budget, implementation of explicit memory-cleanup and checkpointing strategies, construction of OOM-recovery behavior, and development of profiling and benchmarking infrastructure. This area also covered the systems interpretation of Triton experimentation, low-level performance bottleneck analysis, and the engineering justification for avoiding higher-level abstractions that would have reduced runtime visibility.

Data Science work concentrated on the algorithmic and evaluative components. Its responsibilities included the formulation and verification of the GRPO-style objective, design of the rule-based reward and verifier path, preparation of the verifiable math-dataset workflow, integration of entropy masking and SENT curriculum support, and interpretation of reward behavior and model evaluation results. It also carried the methodological burden of defining how benchmark outputs, training dynamics, and reasoning-quality signals should be interpreted without overclaiming beyond the available experimental budget.

Not all tasks can be assigned cleanly to one area. The project also included shared work that had to be developed collaboratively and then distributed between both tracks in the planning model. These shared tasks included overall project definition, feasibility analysis, integrated planning, end-to-end integration testing, experimental orchestration, benchmark review, interpretation of cross-cutting results, and the writing of the common introductory and methodological sections. The fact that the two memories are distinct does not imply that the implementation process was divided into isolated

tracks. On the contrary, the project depended on continuous exchange between the system view and the algorithmic view.

Table 2.4 summarizes the formal distribution.

Phase	Computer Engineering tasks	Data Science tasks	Shared tasks
Foundation	hardware constraints analysis, native architecture criteria, memory-risk identification	problem framing, GRPO literature review, verifiable math-task evaluation framing, metric definition	scope definition, feasibility study, integrated planning, toolchain setup
Core implementation	model loading path, LoRA integration, memory manager, OOM recovery, profiler instrumentation, benchmarking infrastructure	GRPO loss implementation, verifier and reward logic, normalized dataset pipeline, entropy-mask logic, SENT support, evaluation scripts	end-to-end integration, experiment orchestration, debugging loops
Optimization and validation	throughput analysis, VRAM profiling, Triton experiments, benchmark comparison, runtime stability validation	training-dynamics analysis, benchmark interpretation, statistical checks, reward-behavior inspection, error analysis	acceptance criteria review, rerun prioritization, result consolidation
Documentation and review	Computer Engineering memory writing, systems diagrams, engineering justification, defense preparation	Data Science memory writing, methodology/results discussion, limitation analysis, defense preparation	final consistency review, shared introduction and methodology revision

Table 2.4: Task distribution by work area.

Figure 2.3 summarizes the task network behind that distribution. Unlike the Gantt chart, it does not represent calendar duration. It shows the dependency logic that connected planning, the two technical work areas, integrated validation, result consolidation, and final memory preparation.

Integrated PFG Task Network

Dependency map for the shared methodology chapter: planning, dual-track implementation, validation, and final consolidation

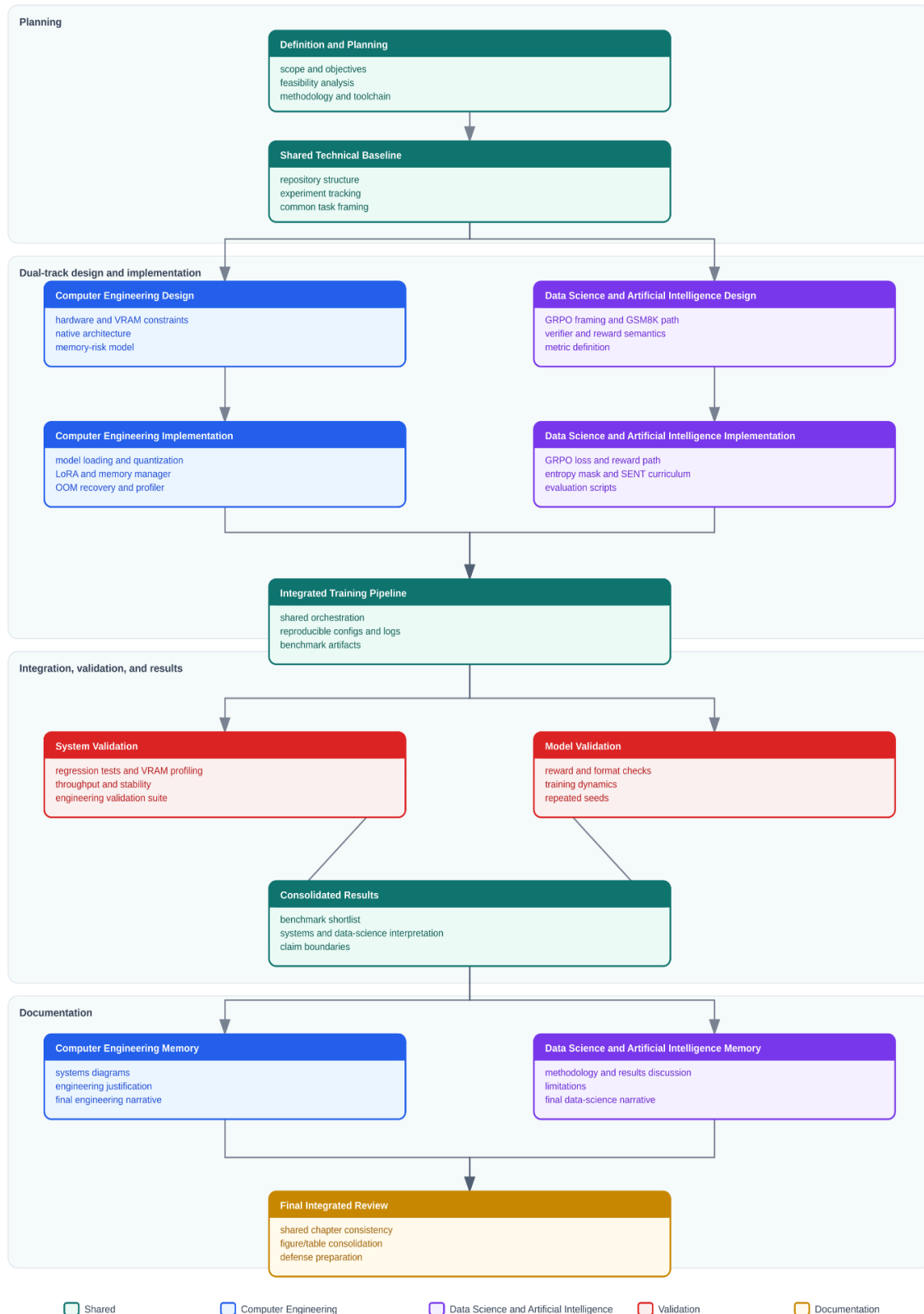


Figure 2.3: Integrated FDP task network.

2.5 BUDGET OVERVIEW AND COST DISTRIBUTION

The budget was prepared as a project resource model. The central hardware asset was the development workstation used throughout the project, with the NVIDIA RTX 3060 Ti as the main computational constraint. For budget presentation, the workstation is treated through the amortized project-use fraction of its approximate 1100 EUR reference acquisition cost, including the GPU as part of the same working machine. With a four-year useful life and a six-month project period, the attributable hardware cost is 137.50 EUR.

Direct software expenditure was effectively zero. The project relied on open-source or no-cost tools such as PyTorch, Transformers, Triton, bitsandbytes, pytest, Git, and GitHub. Weights & Biases was used when needed for experiment tracking, and the free plan covered that use. Electricity consumption is acknowledged as an operational factor, but it was not isolated from the household environment and therefore is not itemized as a separate amount.

The largest cost component is the planned dedication of work time. The human-resource estimate follows the roles performed during the project and keeps the 600-hour planning baseline established for the integrated work. Table 2.5 distributes those hours across the main roles involved in planning, systems engineering, model experimentation, evaluation, and documentation.

Role	Main responsibility	Computer Engineering hours	Data Science hours	Total hours	Rate	Cost
Project manager	Planning, coordination, scope control, and documentation schedule	35	35	70	30 EUR/h	2100 EUR
Systems engineer	Architecture, memory management, profiling, optimization, and testing infrastructure	170	20	190	35 EUR/h	6650 EUR
ML engineer and data scientist	Reward design, dataset preparation, training	20	170	190	35 EUR/h	6650 EUR

	configuration, and evaluation analysis					
Evaluation engineer	Benchmark execution, result validation, and experiment comparison	45	45	90	30 EUR/h	2700 EUR
Technical writer	Final documentation, revision, and defense preparation	30	30	60	25 EUR/h	1500 EUR
Total student labor	Integrated project workload	300	300	600	-	19600 EUR

Table 2.5: Human-resource cost distribution.

The remaining cost components correspond to the technical resources used during development. The workstation provides the material basis for the implementation and experiments, while the software stack adds no direct licensing cost. Table 2.6 consolidates the itemized budget.

Cost block	Amount
Human-resource cost	19600 EUR
Amortized development workstation including RTX 3060 Ti	137.50 EUR
Software and libraries	0 EUR
Weights & Biases	0 EUR
Electricity	not itemized
Total itemized budget	19737.50 EUR

Table 2.6: Itemized project budget.

3. COMPUTATIONAL THEORY FOUNDATIONS

The computational constraints that shape the system start with the hardware itself. Transformer post-training on a single 8 GB GPU is expensive because memory hierarchy, tensor lifetime, data representation, and decoder-only generation all interact within a narrow device budget. The role of this foundation is to define those constraints before moving to the optimization methods and toolchains that work within them.

3.1 GPU ARCHITECTURE AND CUDA PROGRAMMING MODEL

For transformer workloads, raw peak floating-point operations per second (FLOPS) are only part of the story. RooflineBench, a benchmark suite for measuring decoder performance against compute and bandwidth limits, shows that the effective limit depends on operational intensity and memory bandwidth, especially once decoding-heavy execution pushes the hardware into a memory-bound regime [6]. Figure 3.1 grounds this in empirical data: the theoretical comparison demonstrates how workloads are forced onto the memory-bound diagonal at lower operational intensities, while the measured performance confirms that modern models on consumer hardware operate within this constrained regime. The same effect appears in consumer-GPU fine-tuning: sequence length, precision mode, and optimizer state change throughput because they change how much data must be read, rewritten, or kept resident during the step [4]. On a constrained consumer GPU, advertised FLOPS matter less than the amount of work that can be sustained before memory movement becomes the limiting factor [4] [6].

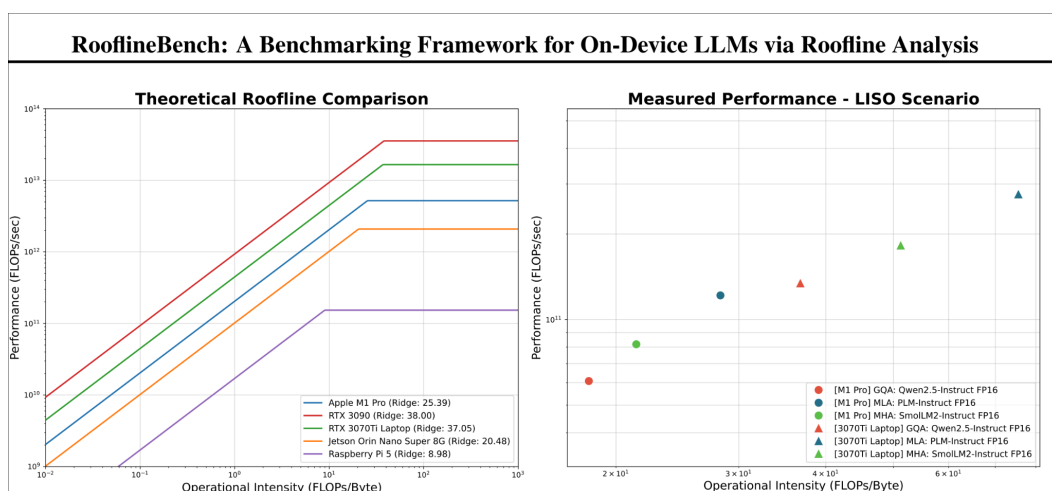


Figure 3.1: Roofline model and measured decoder performance on constrained hardware, extracted from RooflineBench [6].

The Compute Unified Device Architecture (CUDA) programming model exposes that question through a hierarchy of memories and execution scopes. At the storage level, the relevant path runs from global memory through L2 cache to on-chip shared memory and finally to registers [7]. Each level is smaller and faster than the previous one, so efficient execution depends on using the closer levels for data that will be reused and avoiding unnecessary round trips to device memory. Figure 3.2 maps this physical storage hierarchy, illustrating the funnel data must navigate, from the expansive GPU dynamic random-access memory (DRAM), through the shared L2 cache, and down into the tightly coupled unified data caches and register files.

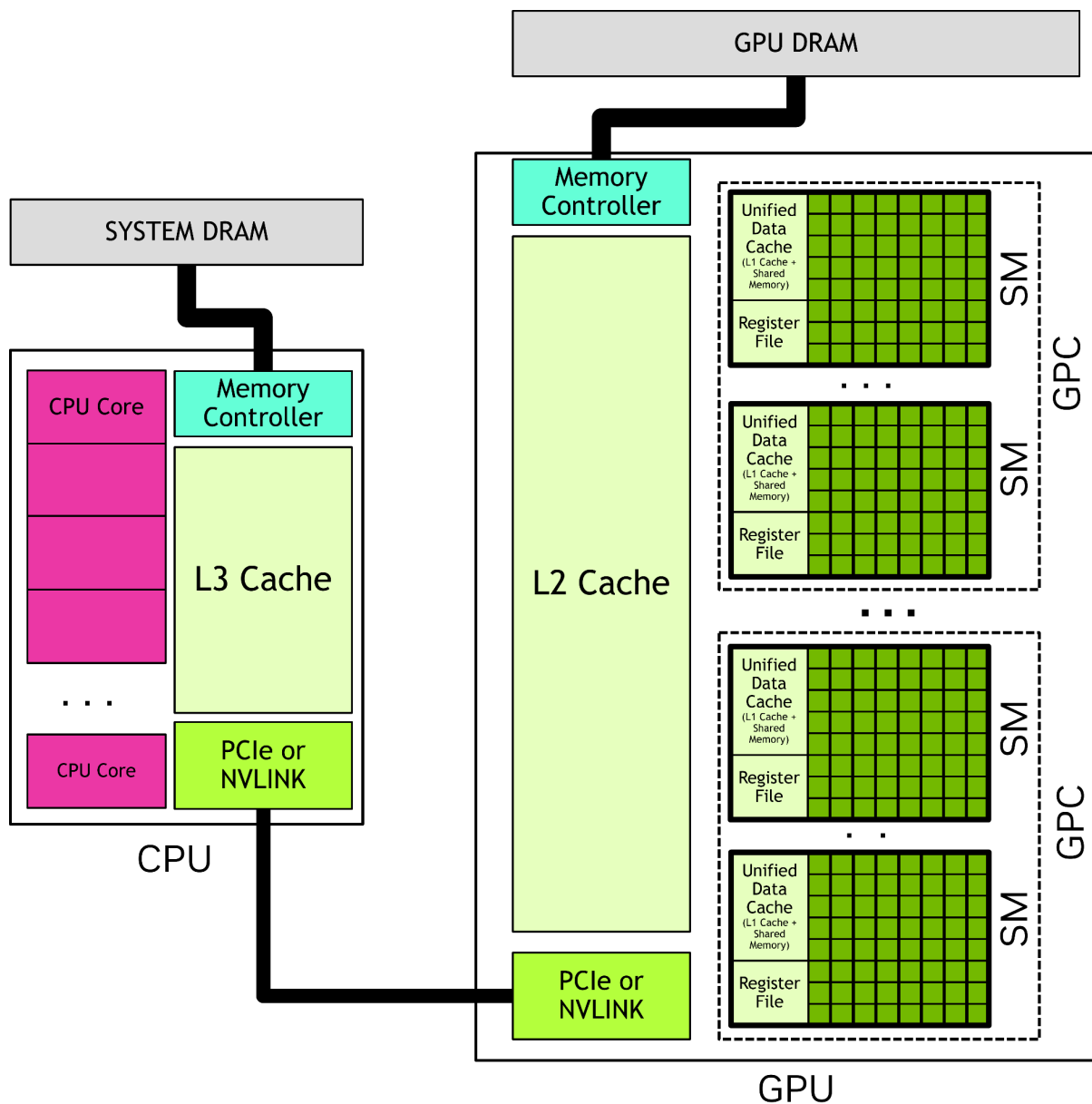


Figure 3.2: GPU memory hierarchy from device memory to registers, extracted from the CUDA Programming Guide [7].

At the execution level, threads are organized into warps of 32, then into blocks, then into grids, with shared memory visible at block scope and registers private to each thread [7]. Figure 3.3 visualizes this corresponding software topology, showing how the overarching grid distributes encapsulated thread blocks across the physical hardware outlined above. This hierarchy matters because every movement across it has a cost. Re-reading from global memory, spilling live values out of registers, or forcing wide synchronization points can erase much of the benefit of fast matrix multiplication.

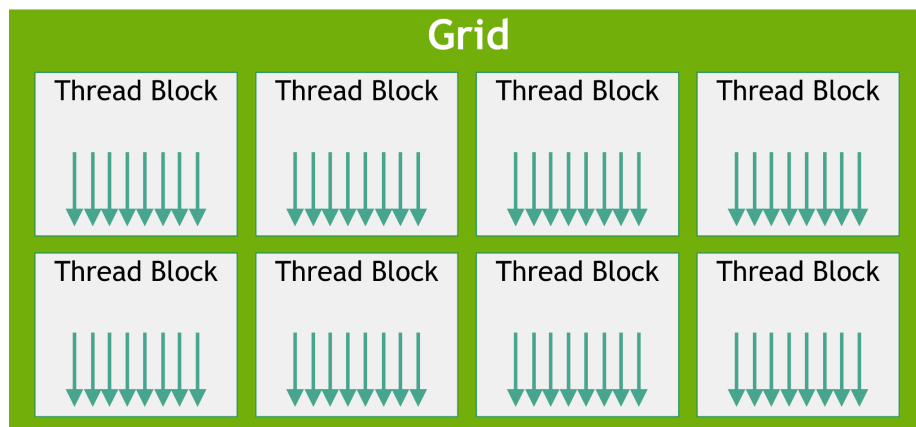


Figure 3.3: CUDA grid, block, warp, and thread hierarchy, extracted from the CUDA Programming Guide [7].

Deep learning kernels are therefore usually tuned for sustained throughput rather than isolated latency. Good performance depends on keeping tensor cores busy, overlapping memory movement with compute, and limiting the amount of work that falls back onto slower parts of the hierarchy [7] [8]. The profiling literature on 8 GB consumer GPUs shows the same effect from a higher level: sequence length, optimizer state, and precision mode all change throughput and VRAM pressure in visible ways because they alter how much data must be carried through the step and when that data is needed [4]. In practice, the CUDA model is not background detail. It explains why performance on local hardware is so sensitive to execution order and intermediate state.

3.2 MEMORY MANAGEMENT IN DEEP LEARNING SYSTEMS

Memory pressure in deep learning does not come from one tensor class. It comes from the interaction of weights, gradients, optimizer states, activations, key-value (KV) caches, and temporary workspaces, and that interaction changes across phases of the loop. The GNMR dynamic-precision

study gives a precision-side version of the same problem: low-precision execution is not uniformly safe across all operators, so stable training may require operator-local precision recovery rather than a single global format [9]. The controlled RTX 4060 study reaches the same conclusion empirically: changing sequence length or optimizer strategy changes both throughput and VRAM occupancy even when the model itself is fixed [4]. A useful memory model has to track tensor lifetime as well as parameter count. Table 3.1 summarizes the main sources of pressure and the trade-offs attached to them.

Source or mechanism	Memory pressure	Main trade-off or caveat
Weights, gradients, and optimizer states	Long-lived training state that can dominate the baseline footprint before activations or caches are considered.	Precision and optimizer choices can reduce the footprint, but low-precision execution is not uniformly safe across all operators [9].
Activations	Intermediate values from the forward pass that must either be retained for the backward pass or rebuilt later.	Retaining them reduces recomputation but raises the peak; releasing them lowers memory use but adds work later in the step, regardless of whether the implementation uses checkpointing, selective recomputation, or a custom phase schedule [10] [11] [4].
KV caches and allocator layout	Decode-time cache growth and phase changes between inference and training.	Conservative reservation wastes memory, but dynamic growth can fragment the logical cache and hurt kernel behavior and throughput [12]. Fragmentation can also make nominal free memory diverge from usable memory and become a large part of reserved memory near the peak [13].
Temporary workspaces	Phase-local buffers that compete with	Planned allocation and buffer reuse can separate a stable virtual layout from physical allocation, materializing memory only when needed. The same principle generalizes

and memory pools	long-lived state inside the same small budget.	beyond serving once buffer reuse and aggressive release become structural design choices [12].
Offloaded state	Working-set portions moved to central processing unit (CPU) or pinned host memory to increase effective capacity.	The gain is paid back through transfer cost, latency, and scheduling complexity. RooflineBench reports roughly linear degradation once automated offloading serves tensors from pinned host memory or ordinary CPU memory rather than device-resident buffers [6], although Reinforcement Learning from Human Feedback (RLHF) systems can still use offloading to stay within the memory envelope [13].

Table 3.1: Main memory-pressure sources and trade-offs in constrained transformer training.

These pressures interact in practice. Fragmentation, activation lifetime, planned allocation, and offloading are different ways of handling tensor lifetimes under a narrow peak. On small hardware, the cost of carrying state forward can exceed the cost of rebuilding or moving some of it, but every reduction in device residency changes the bottleneck somewhere else.

3.3 QUANTIZATION AND DATA REPRESENTATION

Quantization begins as a representation problem before it becomes a training recipe. The first distinction is between low-precision compute formats such as 16-bit floating point (FP16) and BFloat16 (BF16) and lower-bit storage schemes used to shrink memory residency. Mixed-precision training made FP16 practical at scale, while BF16 became attractive because its wider exponent range reduces overflow and underflow risk [9]. On consumer GPUs, however, numerical safety and wall-clock efficiency do not always point in the same direction. The RTX 4060 profiling study found that BF16 degraded throughput relative to FP16 under otherwise similar settings [4].

The second distinction is between storage precision and compute precision. Quantized Low-Rank Adaptation (QLoRA) states that the base weights can be stored in 4-bit NF4 format and dequantized into a higher-precision computation format when the layer is used [11]. That split is central on limited hardware. A model may fit because the stored weights are compact, while the actual arithmetic still happens in BF16 or FP16. Reinforcement Adaptive Mixed-Precision Quantization (RAMP) frames the same constraint from the deployment side: a 7B model in FP16 is already outside the range of an ordinary 8 GB card, so the storage format determines feasibility before any training logic begins [14].

This is where NF4, 4-bit integer (INT4), and double quantization matter computationally. QLoRA defines NF4 as a low-bit format tailored to zero-centered, normally distributed pretrained weights and uses double quantization to shrink the metadata needed to reconstruct them [11]. That does not make the runtime "pure 4-bit." It produces a hybrid path in which memory traffic falls because the stored tensors are smaller, but the execution still pays for dequantization and higher-precision math. From a systems perspective, that hybrid character is the point, not a flaw.

The literature also makes clear why nominal bit-width is an incomplete metric. D2Quant, a weight-only quantization study, shows that some matrices are structurally harder to quantize than others and that weight-only compression can still cause activation drift [15]. RAMP reports that mixed-precision layer assignments can improve footprint and quality, but they also introduce format switches, kernel fragmentation, and register-pressure changes across the stack [14]. In other words, the real cost exceeds bits per weight alone. It also includes metadata, dequantization work, memory traffic, and kernel compatibility.

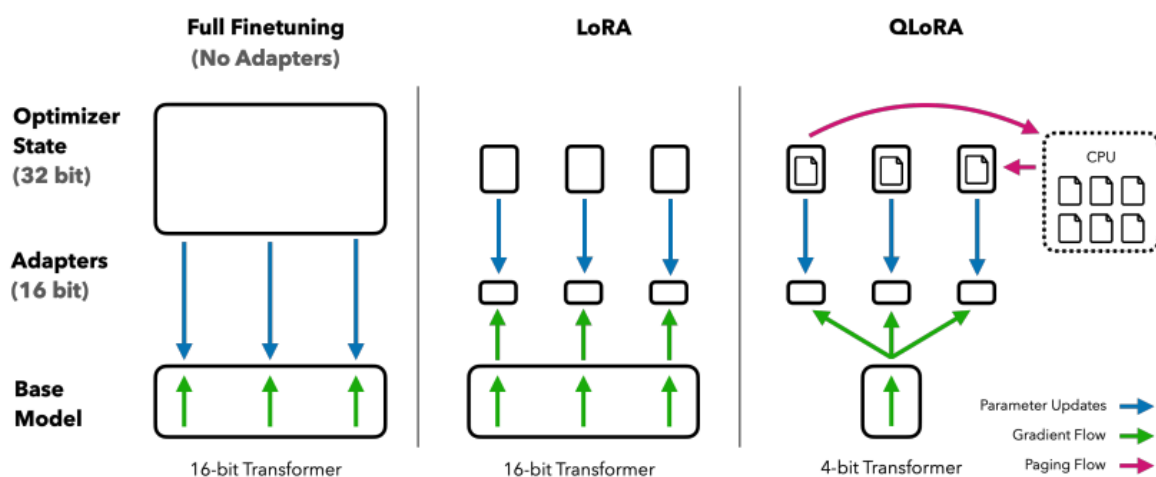


Figure 3.4: Full finetuning, LoRA, and QLoRA memory paths, extracted from QLoRA [11].

That is why quantization belongs in the computational foundations chapter. It changes how much state can live on the device, how often tensors must be moved, and which execution paths remain viable under bandwidth pressure [11] [15] [14]. The next chapter reviews the specific low-VRAM methods that build on these representation choices.

3.4 COMPUTATIONAL BOTTLENECKS OF TRANSFORMER ARCHITECTURES

The transformer remains the main computational skeleton of current language models, and self-attention remains the part whose cost grows fastest with sequence length. In a standard decoder

block, each token produces query, key, and value projections. The attention score tensor then compares the query positions against the key positions, so its logical size grows with the product of the sequence length by itself. This is the computational reason behind the usual $O(n^2)$ attention cost. On local hardware, that quadratic term quickly stops being abstract. Longer contexts increase intermediate state, memory movement, and the chance that one phase of the training loop will hit the VRAM ceiling before the others.

RooflineBench separates decoder-only execution into two phases with very different bottlenecks [6]. Table 3.2 keeps that distinction explicit.

Phase	Execution pattern	Dominant bottleneck	Consequence on constrained hardware
Prefill	Processes the input sequence in parallel.	It can move closer to the compute-bound side of the roofline.	The GPU has more opportunity to use its compute units efficiently, although longer inputs still increase intermediate state and memory movement.
Decode	Generates tokens one at a time and repeatedly fetches weights and KV-cache entries.	It becomes memory-bandwidth bound; RooflineBench estimates roughly one floating-point operation per byte for 16-bit decoding [6].	Compute units can remain underused even when the model is large on paper, and response length becomes a direct pressure point for throughput and VRAM.

Table 3.2: Prefill and decode bottlenecks in decoder-only execution.

Figure 3.5 visualizes this execution bottleneck, showing how decoding workloads are trapped under the memory bandwidth roofline.

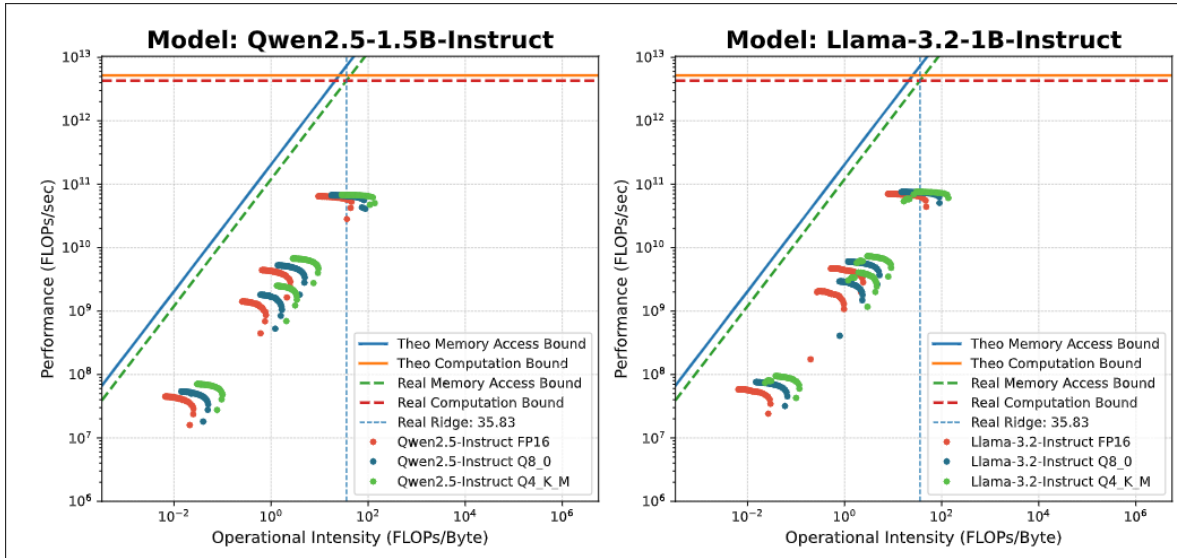


Figure 3.5: Decode workload trapped under the memory-bandwidth roofline, extracted from RooflineBench [6].

The cost of attention is therefore not limited to the number of multiply-add operations. It also depends on the size and lifetime of the intermediate tensors created around the score computation, masking, softmax normalization, and value aggregation. A mathematically equivalent formulation can place very different pressure on the memory hierarchy depending on when those intermediates are materialized and how often they must be reread. Memory access pattern is part of the cost of attention itself, not a separate implementation detail handled after the algorithm is chosen.

The decoder-only architecture adds a second persistent pressure point through the KV cache. vAttention, an attention-cache management approach that treats cache allocation as a virtual-memory problem, notes that the KV cache already accounts for a large fraction of inference memory, grows one token at a time, and is hard to size perfectly in advance because final decode length is unknown [12]. RooflineBench treats the same cache as part of the memory traffic that drags decode workloads toward the bandwidth-bound side of the roofline [6]. On small hardware, the cache is not a side structure. It helps determine feasible batch size, feasible response length, and achievable throughput.

This becomes even more important in reinforcement-learning settings, where rollout generation can dominate runtime. The sparse-rollout literature describes a genuine memory wall: as reasoning traces grow longer, the expanding KV cache forces smaller rollout batches and lowers utilization [16]. Compression can recover capacity, but the paper also shows that naive compression changes the

effective rollout distribution and can destabilize training [16]. In this setting, memory management directly shapes optimization behavior.

Taken together, these foundations explain the constraints that the rest of the memory has to solve:

- bandwidth-bound decoding;
- quadratic attention growth;
- persistent KV state;
- narrow margins for activations and temporary buffers.

The next chapter reviews the current optimization methods and local-first stacks built to cope with those constraints.

4. STATE OF THE ART

If Chapter 3 defined the constraints, this chapter reviews the methods used to work within them. The emphasis now is comparative rather than foundational: which techniques reduce memory pressure, which software stacks package them, which runtime paths accelerate the critical kernels, and where the current ecosystem still falls short for local post-training on a single consumer GPU.

4.1 SINGLE-GPU MEMORY REDUCTION TECHNIQUES

The state of the art for local post-training is built around a small set of recurring memory interventions. Each one attacks a different part of the footprint. Activation recomputation reduces transient activation peaks, low-bit loading shrinks the frozen backbone, paged optimizer strategies smooth update-time spikes, and offloading extends the feasible working set beyond device memory. What matters in practice is not the existence of these techniques in isolation, but how they combine near the 8 GB boundary.

Gradient checkpointing remains one of the most widely used ways to reduce activation memory. The basic exchange is straightforward: store fewer intermediate activations during the forward pass and recompute some of them during the backward pass. QLoRA treats this as important enough to mention explicitly when motivating paged optimizers, since memory spikes during checkpointed backpropagation can still trigger out-of-memory failures on one machine [11]. The RLHF memory study reaches a compatible conclusion from another angle: checkpointing can reduce memory use, but it does not erase fragmentation effects and it helps most when the peak really occurs in training rather than earlier phases [13].

Low-bit loading addresses a different component of the problem: static model residency. QLoRA established the dominant pattern for this setting by storing the frozen base in 4-bit NF4, dequantizing into a higher-precision compute format at execution time, and training only LoRA adapters [11]. This changed what could be run locally. Once the backbone can sit in memory without exhausting the card, the optimization target moves from "can the model load at all?" to "can the full training step survive its transient peaks?"

Optimizer strategy becomes decisive for exactly that reason. QLoRA introduced paged optimizers to prevent memory spikes from causing failures during finetuning [11]. Later consumer-GPU profiling showed that the choice is not merely defensive. On an RTX 4060, paged optimizers improved throughput and made 2048-token runs feasible under an 8 GB budget [4]. Near the memory boundary, optimizer choice changes both feasibility and throughput, so it belongs in the systems discussion rather than being treated as a minor training detail.

CPU offloading is useful, but it is best understood as a fallback capacity mechanism rather than an ideal baseline. The RLHF memory study reports that offloading can reduce effective memory use, even though it may also change fragmentation behavior [13]. RooflineBench complements that result by showing the performance cost once execution has to serve tensors from slower tiers beyond VRAM [6]. Offloading buys room by spending bandwidth and latency. It is often worth doing, but it rarely leaves the system in the same operating regime.

Viewed together, these methods form a layered toolkit. Checkpointing targets activations. Low-bit loading targets the frozen weights. Paged optimizers target transient update spikes. Offloading targets whole-device capacity. This division of labor is the core pattern behind current low-VRAM training practice.

That division is easier to compare directly than through a long catalogue. Table 4.1 summarizes the intervention families that matter most for this project and the reason none of them solves the full local reinforcement-learning problem on its own.

Technique family	Main pressure relieved	Main gain on constrained hardware	Main local cost or limit
Gradient checkpointing	Activation peaks during backward/update	Lowers transient training-phase memory	Adds recomputation cost and does not remove fragmentation by itself
4-bit NF4 loading with double quantization	Persistent backbone residency	Makes a 1.5B reasoning model fit with room left for rollouts and updates	Still requires higher-precision compute and dequantization work
Paged optimizer strategies	Optimizer-state spikes during updates	Smooths update-time memory pressure and can preserve feasibility at longer sequences	Adds runtime and implementation complexity around state movement
CPU offloading	Whole-device capacity limit	Extends the feasible working set beyond VRAM	Pays back the gain through PCIe bandwidth, latency, and less predictable runtime behavior

Table 4.1: Memory-reduction intervention families for constrained training.

4.2 CONSUMER-GPU FINE-TUNING STACKS

The practical route into low-VRAM fine-tuning is now dominated by integrated software stacks rather than isolated techniques. In current practice, most local workflows combine model loading, quantization, adapter injection, optimizer setup, and orchestration through a small number of high-level libraries. The most common baseline is the combination of Transformers, Parameter-Efficient Fine-Tuning (PEFT), and bitsandbytes, where bitsandbytes supplies the low-bit loading and optimizer primitives used by many local workflows. This stack has become the default reference because it provides a direct path from a pretrained model to a quantized LoRA or QLoRA run without requiring a custom runtime. Consumer-GPU profiling work reflects that norm explicitly: the RTX 4060 study uses Transformers through the Trainer application programming interface (API), PEFT for adapter integration, and bitsandbytes for low-bit execution [4].

The success of this stack is easy to understand. It reduces engineering cost, lowers the entry barrier for experimentation, and makes a wide range of configurations available through standard APIs. Its limitations are equally clear once the hardware margin becomes small. The more orchestration is delegated to framework layers, the harder it becomes to observe the exact sequence of allocations, releases, cache growth, and temporary buffers created during each phase of the loop. On a large GPU this may be acceptable or even desirable. On an 8 GB card, that loss of visibility can become a practical systems problem because the difference between a stable run and an out-of-memory failure may be a small number of hidden tensors or a poorly timed phase transition.

The same tension appears in reinforcement-learning-oriented abstractions. Transformer Reinforcement Learning (TRL) and related training layers reduce the cost of assembling a reinforcement-learning fine-tuning workflow on top of the Hugging Face ecosystem. This is valuable because GRPO-style experimentation involves more than ordinary supervised fine-tuning: rollout generation, response scoring, grouped sampling, masking, and repeated policy updates all need to be coordinated. High-level libraries reduce that burden, but they also encourage a more opaque execution model in which inference-like phases and training phases are tightly interleaved behind a common interface. That design is productive for general experimentation, but less attractive when the main objective is to understand and control memory-critical behavior step by step.

Recent low-VRAM frameworks make this trade-off even more visible. Unsloth, a fine-tuning stack aimed at faster low-VRAM adaptation, documents a workflow for LoRA, QLoRA, and GRPO-like training under constrained memory, including the use of gradient checkpointing, fast inference backends, and low-bit execution [17] [18]. This indicates a shift beyond simple supervised adaptation; the goal is now to make reasoning-heavy post-training practical on consumer hardware. Compact-model research follows a similar logic. Tina, a compact-model training study, demonstrated that a 1.5B model can achieve significant reasoning gains through efficiency-focused training rather than massive infrastructure [5].

These stacks matter because they compress a large amount of systems knowledge into a usable workflow. They also expose the central limitation of high abstraction under strict VRAM constraints. Convenience and control do not increase together indefinitely. At some point, making a workflow easier to run also makes it harder to inspect. On a machine with ample memory, that trade-off is often favorable. On a single consumer GPU, it can decide whether the source of a failure is visible, reproducible, and correctable.

Table 4.2 makes that trade-off explicit for the main software routes considered in the project.

Stack or route	Main strength	Main limitation near 8 GB VRAM	Relevance to this project
Transformers + PEFT + bitsandbytes	Fastest standard path from pretrained model to low-VRAM fine-tuning	Tensor lifetime and hidden framework buffers are harder to inspect phase by phase	Baseline ecosystem reference, but not enough control for the final engine
TRL and similar reinforcement-learning abstractions	Reduces reinforcement-learning workflow assembly cost	Interleaves rollout and optimization behavior behind a broad training interface	Useful comparative reference, but weaker for phase-level observability
Unslloth-style low-VRAM workflows	Strong practical optimization for compact-model LoRA / GRPO workflows	Prioritizes convenience and optimized defaults over direct runtime control	Important state-of-the-art evidence that low-VRAM reinforcement learning is feasible, but not a substitute for native control
Native PyTorch engine	Maximum visibility over generation, scoring, update, cleanup, and recovery phases	Highest implementation and debugging burden	Chosen because observability and memory policy control were first-order project goals

Table 4.2: Consumer-GPU fine-tuning stacks and their local-control trade-offs.

4.3 OPTIMIZED KERNELS AND RUNTIME PATHS

Once the model fits and the orchestration stack is chosen, performance depends heavily on the efficiency of a small number of critical paths. In transformer workloads, attention kernels and generation backends dominate that discussion. The most influential reference in this area is FlashAttention, an exact attention-kernel family that reframed attention optimization around memory traffic rather than only around arithmetic complexity. Its contribution was to show that exact attention can become substantially faster if the implementation minimizes reads and writes between slow high-bandwidth memory (HBM) and fast on-chip memory through an input/output-aware design [10]. This result mattered because many earlier attempts to accelerate attention had reduced nominal FLOPs without delivering equivalent wall-clock improvements.

Later work has refined that argument. FlashAttention-4 shows that, on newer hardware, the bottleneck can shift away from matrix multiplication itself and toward shared-memory traffic and other functional units that do not scale as quickly as tensor-core throughput [8]. The broader systems lesson is that optimization is hardware-relative. A kernel that is efficient on one generation of GPUs is not automatically optimal on another. For constrained local training, this matters because the relevant question is not whether a kernel is state of the art in the abstract, but whether it reduces the specific bottleneck of the target machine.

Triton has become important in this context because it lowers the barrier to custom kernel specialization. Its attraction is not that it supersedes low-level CUDA work in every case, but that it allows selective optimization of narrow, high-impact paths from within a Python-centered development environment. That makes it especially attractive for projects that need some low-level acceleration without abandoning the flexibility of a research codebase. At the same time, the literature around optimized attention kernels makes one point clear: custom kernels must justify themselves empirically. FlashAttention-4 reports gains over a Triton baseline on Blackwell hardware [8], which does not weaken Triton as a tool. It shows that a kernel language is not a result by itself. A concrete implementation still has to improve throughput, latency, or memory behavior on the relevant execution path.

Generation backends deserve separate attention because reinforcement-learning post-training is not dominated only by backward computation. Rollout generation can become one of the main runtime and memory costs, especially when grouped sampling and long responses are involved. Work on sparse rollouts describes this as a memory wall: as reasoning traces grow longer, KV-cache growth constrains rollout batch size and harms utilization [16]. This changes the meaning of runtime optimization. Faster kernels help, but so does any backend choice that reduces generation overhead enough to keep grouped rollouts feasible.

The practical state of the art in constrained runtime optimization is therefore selective, not monolithic. FlashAttention represents the benchmark for IO-aware attention. Triton shortens the path

to focused kernel specialization. Fast-generation backends help with rollout throughput but may introduce their own memory-management trade-offs. The recurring pattern is that local post-training does not benefit equally from every acceleration layer. The useful path is usually the one that improves the critical bottleneck without making memory behavior too opaque to control.

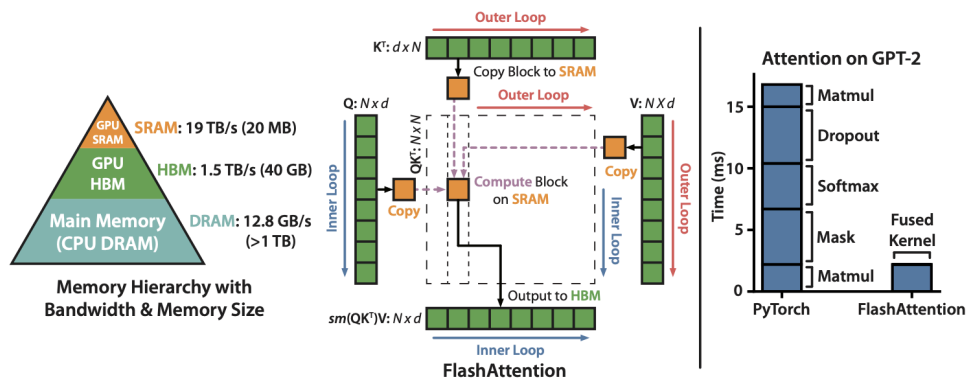


Figure 1: **Left:** FLASHATTENTION uses tiling to prevent materialization of the large $N \times N$ attention matrix (dotted box) on (relatively) slow GPU HBM. In the outer loop (red arrows), FLASHATTENTION loops through blocks of the $\mathbf{K}$ and $\mathbf{V}$ matrices and loads them to fast on-chip SRAM. In each block, FLASHATTENTION loops over blocks of $\mathbf{Q}$ matrix (blue arrows), loading them to SRAM, and writing the output of the attention computation back to HBM. **Right:** Speedup over the PyTorch implementation of attention on GPT-2. FLASHATTENTION does not read and write the large $N \times N$ attention matrix to HBM, resulting in an 7.6 $\times$ speedup on the attention computation.

Figure 4.1: Tiled IO-aware attention compared with the naive attention path, extracted from FlashAttention [10].

4.4 GAP IN CURRENT TOOLING FOR LOCAL POST-TRAINING

The literature offers a substantial set of techniques for reducing memory, accelerating attention, and packaging low-VRAM workflows. Even so, a clear tooling gap remains when the target is local reinforcement-learning post-training on a single consumer GPU. The problem is not the absence of optimization methods. It is the absence of a development model centered on phase-level control under a strict memory ceiling.

The gap has three parts:

1. **Granularity.** Most existing stacks optimize broad classes of behavior: low-bit model loading, automatic adapter insertion, framework-managed rollout generation, or generic memory-saving policies such as checkpointing and offloading. Those abstractions are useful,

but they do not necessarily expose the exact moments when memory pressure rises. In local post-training, those moments matter. Generation, scoring, frozen-policy computation, backward propagation, optimizer update, checkpoint save, and error recovery do not place the same demands on VRAM. When a system operates close to the hardware boundary, feasibility depends on controlling the transitions between those phases and on releasing memory aggressively once each phase is complete.

2. **Observability.** High-level stacks are good at making workflows easier to launch, but they are less effective at showing why a run failed near the OOM boundary. The literature contains useful partial answers. RLHF memory studies show that explicit allocator hygiene, including cache clearing between phases, can reduce fragmentation substantially [10]. Sparse-rollout work shows that memory-saving interventions can change not only feasibility but also training dynamics [13]. What remains missing is a small, inspectable engine in which these decisions are not side effects of a framework backend but direct parts of the runtime design.
3. **Local-constraint mismatch.** Much of the surrounding ecosystem assumes either that more hardware can be added, that offloading overhead is acceptable, or that some loss of runtime transparency is an acceptable price for convenience. Those assumptions are reasonable in general-purpose tooling. They are less convincing when the goal is to study and control GRPO-style post-training on one 8 GB GPU. In that regime, a reference-model-heavy design, a deeply layered orchestration stack, or an overly automatic memory strategy can make the system harder to debug than to optimize.

This gap explains the role of a native single-GPU training engine. Such a system does not replace the broader ecosystem; it exposes the training loop at the level where constrained execution must actually be managed. A native engine can separate generation from optimization, make memory cleanup explicit, keep adapter behavior directly visible, and support selective acceleration only where benchmark evidence justifies it. Mainstream frameworks tend to hide these details behind broader abstraction. A granular engine moves in the opposite direction, but that is the adaptation required by the hardware constraints reviewed here.

5. SYSTEM ARCHITECTURE AND DEVELOPMENT

5.1 SYSTEM ARCHITECTURE DESIGN

5.1.1 Architectural objectives and global structure

The system was designed as a native training engine for a narrow operating regime: reinforcement-learning post-training of a 1.5B reasoning model on a single GPU with 8 GB of VRAM. That constraint shaped the architecture from the beginning and ties the design to objectives O1, O3, and O4: implementing the local engine, managing the memory boundary, and preserving fine-grained runtime control. The full execution path had to be visible and controllable enough to survive repeated runs near the memory boundary. The resulting structure separates startup, model preparation, data loading, rollout generation, reward computation, optimization, checkpointing, benchmarking, and profiling into explicit components coordinated by one central training loop.

At a high level, the execution path begins in the command-line entry point, which loads the default configuration, applies runtime overrides, initializes logging, checks device readiness, and transfers control to the trainer. The trainer then assembles the runtime from specialized subsystems: the model loader prepares the quantized backbone and tokenizer, the LoRA module injects the trainable adapters, the memory manager controls cache release and checkpointing policy, and the checkpoint manager persists the training state. During execution, the same trainer also coordinates the verifier, entropy-based selective masking, benchmarking hooks, and optional profiler hooks. This organization keeps the critical path short enough to inspect while avoiding a monolithic script with hidden side effects.

The main architectural decision was to treat the training loop as a composition problem rather than as a single framework call. Rollout generation, policy evaluation, loss computation, optimizer stepping, metric logging, and recovery actions place different demands on memory and on temporary tensor lifetime. Hiding all of them behind a general-purpose abstraction would have reduced development effort, but it would also have made the system harder to profile and harder to stabilize under pressure. The architecture therefore favors explicit control over a smaller code surface.

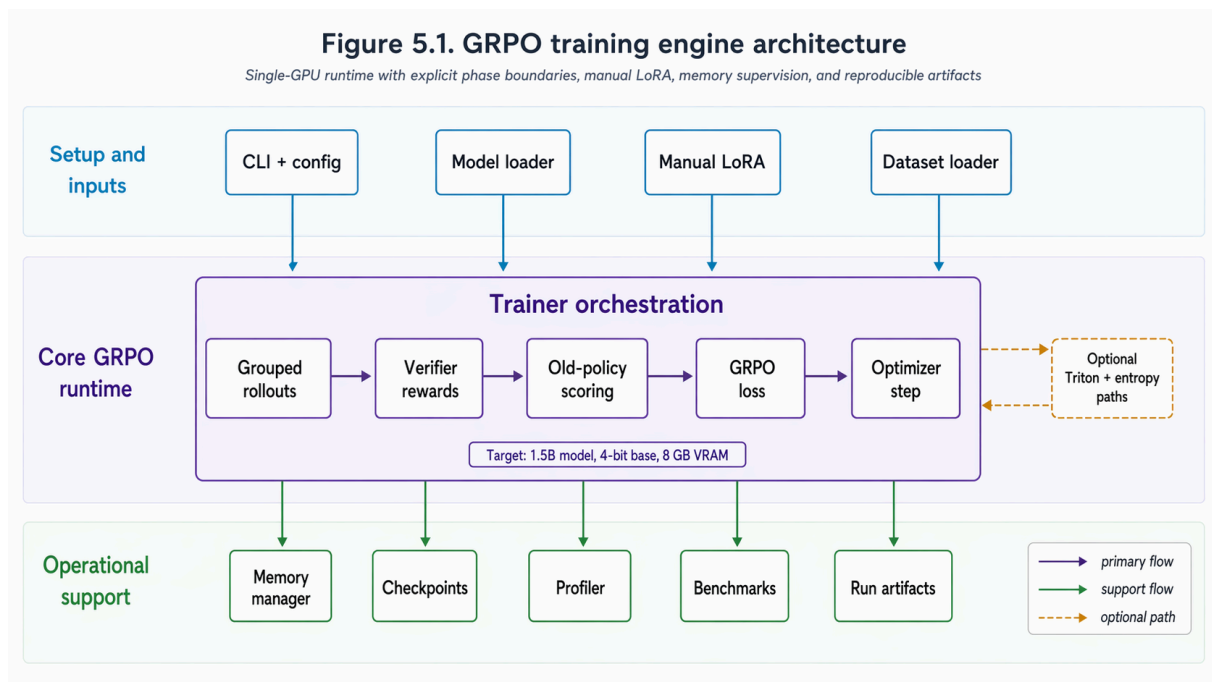


Figure 5.1: GRPO training engine architecture.

The component boundaries follow that same logic. The runtime is organized around five central responsibilities, summarized in Table 5.1. Around that core, configuration, logging, benchmarking, and profiling act as operational support layers rather than as detached utilities. They are part of the architecture because they determine whether the system is observable and reproducible under the same constrained conditions in which it is supposed to operate.

Component	Main responsibility	Why it exists in this architecture
Model loader	Load tokenizer and 4-bit backbone with the effective compute dtype	Establishes a compact, predictable starting footprint
Manual LoRA subsystem	Inject, enable, disable, enumerate, and export adapter weights	Keeps the trainable surface small and directly inspectable
Memory manager	Track VRAM, clear cache, update checkpointing policy, expose peaks	Makes runtime pressure visible and actionable near the OOM boundary

Trainer / orchestrator	Coordinate generation, verification, old-policy scoring, update, logging, and retries	Centralizes the phase transitions that actually determine feasibility
Checkpoint manager	Save full checkpoints and compact LoRA exports, restore interrupted state	Preserves continuity and replay-safe recovery under constrained execution

Table 5.1: Core architectural responsibilities of the training engine.

5.1.2 Phase boundaries and data flow

The data flow reflects the same choice. The first version of this path was built around GSM8K because it provided a compact, verifiable baseline. The later architecture keeps the same execution contract while generalizing the input side:

1. A normalized mathematical example enters through the data loader.
2. The example is tokenized and expanded into grouped prompts.
3. Generation produces candidate responses and the temporary cache state associated with them.
4. Verification converts the sampled text into reward values.
5. The trainer builds the tensors required for old-policy log probabilities, response masking, relative-advantage computation, and loss evaluation.
6. The transient training states are consumed or explicitly cleared before they can remain alive across unrelated phases.

The same data layer also supports broader verifier-compatible mathematical datasets through the normal dataset interface. Once examples are normalized into the expected question, answer, split, and metadata fields, they can be saved as local train, validation, and test splits, processed by the same SENT preprocessing workflow for semantic-entropy-based ordering when curriculum ordering is needed, and then loaded by `train.py` without changing the trainer. This keeps dataset preparation, curriculum preparation, and training separated while preserving one runtime contract for the trainer.

Seen at the level of one training step, the architecture is easier to justify. Each transition changes both the shape and the residency time of the active tensors, as summarized in Table 5.2. A design that leaves these transitions implicit makes it difficult to know where memory is really being spent. A phase-explicit architecture makes it possible to reason about when allocations should peak, when cleanup should happen, and which operations deserve separate profiling.

Phase	Main active state	Memory implication	Architectural consequence
Prompt preparation	Prompt batch and tokenized inputs	Cheap to hold compared with later states	Kept as a stable entry point for the step
Generation	Grouped responses and temporary cache state	Can become the dominant short-term cost, especially as the response budget grows	Isolated as a visible phase with its own profiling and cleanup points
Verification and reward assignment	Sampled texts, parsed answers, reward values	Lower GPU pressure than generation or backward, but the state determines later training tensors	Kept separate from optimization so reward behavior remains inspectable
Old-policy scoring, masking, and advantage computation	Log-probability tensors, response masks, relative advantages	Transient tensors can push the run over the memory boundary if they survive too long	Consumed promptly and cleared explicitly
Backward and update	Activations, gradients, adapter parameters, optimizer state	Training-oriented pressure differs from inference-like generation pressure	Managed with distinct memory rules and recovery decisions

Table 5.2: Phase boundaries and memory implications in one training step.

The system is organized around phase boundaries rather than around a single end-to-end black box. Generation, scoring, old-policy computation, optimization, checkpoint save, and benchmark evaluation all use the same model, but they do not use it in the same way. Some phases are inference-like and dominated by cache growth or sequence length. Others are training-oriented and dominated by activations, gradients, and optimizer state. Keeping those phases visible in the architecture gives the runtime a place to apply distinct memory rules instead of assuming that one

generic policy is suitable everywhere. It also gives the recovery logic a concrete structure. When the system approaches the OOM boundary, the question is rarely whether the whole run is too large in the abstract. The useful diagnosis is which phase crossed the limit, what state can be preserved, and which part of the schedule actually needs to change.

5.1.3 Reuse across training, benchmarking, and profiling

The architecture therefore follows a layered but asymmetric structure. The startup and configuration path is thin, because it exists mainly to assemble the runtime and define the execution contract. The core path is heavier and more stateful because it must coordinate the transitions that actually determine feasibility. Around it sit benchmarking and profiling paths that reuse the same runtime rather than reimplementing it. This reuse matters. If benchmarks, diagnostics, and profiling were built on a different execution path, they would be less trustworthy as evidence about the behavior of the real training loop.

5.2 ARCHITECTURAL DESIGN DECISIONS

5.2.1 Native control over framework abstraction

Three design decisions define the structure of the engine more than any individual optimization. The first is the choice of native PyTorch over high-level orchestration layers such as Hugging Face Trainer or TRL. The second is the choice of manual LoRA instead of PEFT. The third is the decision to treat memory control as a primary architectural concern instead of as a late optimization pass. Together, these choices explain why the system looks closer to a compact runtime than to a standard library-driven fine-tuning script.

Choosing native PyTorch was mainly a decision about visibility. The project required direct control over generation, old-policy computation, backward passes, cache release, micro-batching, and recovery after OOM events. Those requirements are hard to satisfy cleanly when several execution phases are hidden inside higher-level abstractions. Writing the loop at lower level increased implementation and debugging effort. That cost was accepted because the project needed a system whose behavior could be inspected phase by phase. On the target hardware, observability was more important than reducing the amount of handwritten orchestration code.

5.2.2 Manual LoRA and adapter visibility

The decision to implement LoRA manually follows the same reasoning. PEFT is effective for standard parameter-efficient fine-tuning, but it also introduces another layer of indirection between the code and the adapter behavior. Here the adapter path had to remain visible enough to answer concrete systems questions: which modules receive LoRA layers, how gradients flow through quantized base

layers, how adapters are disabled during base-policy passes, and how a future fused path could interact with those layers. Manual LoRA increased the maintenance burden, but it produced a simpler dependency surface and a more direct relationship between the runtime and the code that defines the trainable path.

5.2.3 Memory control as an architectural constraint

Explicit memory control became a first-class requirement because the project operates near the failure boundary by design. An 8 GB GPU leaves little room for opaque buffers, accidental tensor retention, or phase transitions that keep stale allocations alive longer than necessary. For that reason, memory management is not confined to one isolated module. It informs the trainer structure, the checkpointing policy, the logging strategy, and the way configuration is exposed. Feasibility depends on controlling when memory is acquired, when it is released, and which mechanisms react when the margin becomes too small.

5.2.4 Why the design stays narrow

These decisions also explain what the architecture does not try to be. It is not a general-purpose reinforcement-learning framework, a distributed training system, or a wrapper over every mainstream post-training stack. The design is deliberately narrower. It targets one model family, one hardware tier, and one style of local post-training. The data path is broader than the original GSM8K baseline, but it remains bounded to verifier-compatible mathematical datasets with deterministic splits and stable prompt formatting. That narrow scope makes the system more legible. It also makes the engineering contribution clearer, because each structural choice can be evaluated against a concrete operational constraint rather than against an abstract idea of flexibility.

Design boundary	What it avoids	What it protects
One model family and one hardware tier	A broad compatibility layer with behavior that is difficult to inspect	A direct mapping between code decisions and the 8 GB execution regime
Verifier-compatible mathematical datasets	An open-ended data interface with unstable prompt and answer formats	Deterministic splits, stable prompt formatting, and reproducible reward computation

Native local orchestration	A common framework interface that merges data handling, rollout execution, optimization, logging, checkpointing, and evaluation	Phase-level control over tensor residency, cost attribution, and recovery behavior
----------------------------	---	--

Table 5.3: Scope boundaries and protected engineering properties.

This narrowness also distinguishes the architecture from a more conventional framework-driven design. In a broader stack, generality is often achieved by merging several concerns into a common training abstraction and differentiating runs mostly through configuration. That is efficient when the main objective is feature coverage across many settings. It is less appropriate when the main objective is to understand one constrained execution path in detail. Here the architecture gives up some genericity in exchange for better control over residency, better attribution of runtime cost, and a clearer mapping between code structure and hardware behavior.

The same trade-off appears in the relationship between engineering overhead and debugging cost. At first sight, using more specialized local components seems to increase complexity, because more code must be written and maintained directly. In practice, on a resource-constrained system, hidden complexity can be more expensive than visible complexity. A hidden allocation, an implicit phase transition, or a backend decision made far from the trainer can cost more time than a longer local implementation if it turns a stable run into an intermittent failure. The architecture was therefore optimized less for minimal source code and more for diagnosability under pressure.

5.3 CORE ENGINE COMPONENTS

5.3.1 Model loading and quantized startup

The model loader is the first of the core components because it defines the baseline memory footprint for everything that follows. Its role is to load the tokenizer and the base language model in a 4-bit configuration suitable for local training, while also resolving the effective compute dtype and basic tokenizer settings. This component is intentionally small. It does not try to hide the quantized loading path behind several compatibility layers. Its job is to establish a compact, predictable starting point for the rest of the runtime. That separation is useful because it isolates one of the most consequential choices in the project: a frozen quantized backbone with a trainable adapter path layered on top.

5.3.2 Adapter path and manual LoRA layers

The LoRA subsystem sits directly on top of that loaded backbone. Its central abstraction is a manual wrapper around supported linear layers or bitsandbytes-backed 4-bit layers; the latter are quantized linear layers supplied by the low-bit loading library. The wrapper keeps trainable low-rank matrices, applies the configured scaling and dropout, and can route eligible operations through selective Triton paths when that is justified. Beyond the layer itself, the subsystem also provides the logic to inject adapters into the model, enumerate adapter parameters, enable or disable adapters for different passes, and save or merge adapter weights. This makes the adapter path visible both as a mathematical mechanism and as a runtime object that the trainer can control explicitly.

5.3.3 Memory supervision during execution

The memory manager forms the architectural bridge between static model design and dynamic execution. It exposes lightweight telemetry, cache-clearing behavior, memory warnings, and dynamic checkpointing control. Its role is to report how much memory is in use and to shape how the system responds to that information. The module can clear cache on a schedule, reset peak statistics, print detailed telemetry, and update checkpointing policy when pressure crosses configured thresholds. For that reason the memory manager remains part of the engine core rather than an auxiliary tool layer. The training loop depends on it for operational stability and for debugging convenience.

5.3.4 Trainer as coordination center

The training orchestrator is the real center of the system. It composes the other components into a complete GRPO execution path and governs the transitions between phases. Its responsibilities split into two groups:

- During setup, it loads the model, injects LoRA, creates the optimizer and scheduler, initializes verifier and entropy mechanisms, attaches checkpointing and benchmarking support, and prepares optional profiler hooks.
- During execution, it handles grouped generation, reward computation, response masking, old-policy log-probability caching, training micro-batches, metric updates, OOM backoff, checkpoint cadence, and final weight export.

This component is where architectural decisions become visible as runtime behavior.

The OOM path is part of that architectural role, not an isolated error handler. The trainer is the only component that sees the active phase, the current micro-batch schedule, the live memory signal, the prepared rollout state, and the checkpoint context at the same time. For that reason it is also the natural place to coordinate recovery rather than delegating it to a generic outer wrapper. A constrained runtime can recover coherently only if the same component that governs phase

transitions can also decide what work has already succeeded, what state must be preserved, and which later action would be unnecessarily destructive. Recovery therefore remains part of the main execution path because only the trainer has enough context to make those decisions correctly.

5.3.5 Checkpointing, recovery, and export

The checkpoint manager closes the core set by handling persistence and recovery. It can save full checkpoints containing model, optimizer, scheduler, and metadata state, and it can also save LoRA-only weights as a much smaller export path. On resume, it restores state and relocates optimizer tensors to the model device so that interrupted runs can continue without manual repair. This dual role matters on constrained hardware. Full checkpoints preserve complete training continuity, while LoRA-only saves provide a lighter artifact for later evaluation or inference.

Its architectural importance is broader than ordinary persistence. On this hardware, interruption and continuation are part of normal engineering reality rather than rare administrative events. A useful checkpoint therefore has to preserve more than weights. It has to preserve enough execution context that a resumed run still remains in the same operational regime as the interrupted one. That includes the effective configuration, the optimizer and scheduler state, and the surrounding metadata that make later profiler traces and benchmark comparisons interpretable. Persistence and runtime continuity were designed together from the start.

5.3.6 Dependency order and centralization

Although each of these components has a distinct responsibility, the design avoids treating them as isolated services. Their value comes from how they fit together. Quantized loading reduces the static footprint, manual LoRA keeps the trainable path small and inspectable, the memory manager governs pressure-aware behavior, the trainer coordinates phase transitions, and the checkpoint manager preserves both continuity and compact exports. The result is a runtime architecture in which the major engineering choices remain legible instead of being dispersed across several framework layers.

This interaction is easier to see if the components are read in execution order rather than as a static list:

1. The model loader establishes the initial memory budget and execution dtype.
2. The LoRA subsystem defines the only trainable part of the network and therefore the scope of gradient flow.
3. The trainer builds the phase sequence around those two decisions.
4. The memory manager observes the resulting runtime and reacts when the current policy becomes unsafe.
5. The checkpoint manager preserves the resulting state and allows the same run to be resumed or exported without rebuilding the whole context from scratch.

The system functions as a strict dependency chain rather than as a set of independent services. Decisions made during the initial stages, such as memory allocation or data formatting, directly establish the operational boundaries for all subsequent processes.

That dependency chain also explains why some responsibilities remain centralized instead of being distributed further. The trainer is intentionally the coordination center because it is the only place where model state, batch state, reward state, memory state, checkpoint state, and benchmark state are all visible at once. Moving too much of that coordination outward would have made the code look more modular on paper while weakening the only location where cross-phase decisions can be made with full context. For this system, excessive decomposition would have produced more interfaces, but not better control.

5.4 CONFIGURATION, LOGGING AND OPERATIONAL SUPPORT

5.4.1 Configuration as execution contract

The operational support layer was designed to make the engine configurable and observable without turning it into an opaque configuration framework. The main configuration backbone is a nested dataclass-based schema with separate sections for the model, LoRA, GRPO, entropy masking, curriculum learning, training behavior, and experiment tracking. The command-line entry point starts from a default preset tailored to the 8 GB target and then mutates the relevant nested fields in response to explicit runtime flags. This approach keeps the normal path short while still allowing controlled variation during benchmarks and diagnostic runs.

This configuration structure also acts as a boundary between stable defaults and experimental overrides. The default preset defines the operating assumptions under which the rest of the architecture was built, while the CLI surface exposes only the knobs that are meant to vary across runs. That matters for reproducibility. The project does not depend on a large collection of loosely documented JSON files scattered across the repository. Instead, the configuration model remains close to the runtime code that consumes it, and the checkpointing subsystem can serialize or restore the effective configuration as part of the training state.

5.4.2 Logging and runtime observability

Logging is treated in the same practical way. The logging utilities define a shared namespace, custom verbosity levels, and a handler adapted to long-running progress bars, so console output remains readable during training. More important than the mechanics is the role logging plays in the architecture. It is one of the tools used to understand where the runtime is spending time, when the memory margin is shrinking, which policy changes were applied, and how a run reached a failure.

Structured metric logging and optional Weights & Biases integration extend that observability beyond the console by preserving machine-readable traces of the execution.

5.4.3 Recovery and profiler integration

Checkpointing and recovery support also belong to the operational layer because they connect persistence with runtime continuity. The trainer coordinates the checkpoint manager with the current training state, output directory layout, and final export path so that both interrupted and completed runs remain recoverable. Recovery is built into the execution model from startup through the end of training.

The optional profiler hooks complete this layer by giving the trainer a way to emit phase-level, per-step, and anomaly-oriented telemetry during execution. The profiler subsystem is separate enough to remain optional during normal runs, but close enough to the trainer to capture the transitions that matter most under memory pressure. Profiling is an instrumented view of the same runtime that performs the training.

5.4.4 Operational baseline and reproducibility

The operational environment itself is intentionally modest. The system assumes a CUDA-capable NVIDIA GPU, the PyTorch and Transformers stack, bitsandbytes for low-bit loading, optional Triton for selected kernel paths, FastAPI for the optional service interface, and Weights & Biases for experiment tracking depending on the run mode. That dependency baseline is still substantial, but it is narrower than the combination of high-level training frameworks that would otherwise be required to obtain equivalent behavior. The architecture therefore keeps the support layer large enough to be usable and observable, but small enough that its behavior can still be traced from the code.

Keeping these support mechanisms close to the runtime also has an architectural purpose. For training on constrained hardware, reproducibility involves more than just logging hyperparameters. It depends on capturing the specific memory state and tensor lifecycle of the execution environment, since these factors determine whether a configuration will successfully run or exceed the available VRAM. The relevant execution context includes:

- the exact output layout;
- the metric stream;
- the checkpoint cadence;
- the logging granularity;
- the profiler state used to interpret a run.

If these elements are external to the execution path, they become easier to lose or to desynchronize from the training state. By keeping them attached to the runtime, the system makes the run easier to reproduce and easier to audit after the fact.

5.5 TESTABILITY AS A DESIGN REQUIREMENT

5.5.1 Why testability shapes the design

Testability was treated as an architectural requirement because the project replaces several mature abstractions with direct implementations. Once the training loop, adapter path, checkpointing, and memory behavior are brought under explicit local control, correctness can no longer be delegated to the surrounding ecosystem. The architecture had to remain modular enough that individual responsibilities could be exercised in isolation and then recombined in higher-level integration tests.

5.5.2 Coverage across components and execution modes

The resulting test structure follows the system boundaries closely. Table 5.4 groups the main coverage areas by the kind of contract they protect.

Test area	Examples of covered behavior	Why it matters	Hardware dependency
Loader and adapter contracts	Model loading, LoRA behavior, adapter enumeration, fake bitsandbytes fixtures	Checks important adapter and loader logic without requiring real 4-bit kernels in every test environment	Usually lightweight; heavier checks cover real loading paths
Algorithm and data contracts	GRPO algorithm logic, normalized dataset loading, old-policy cache handling, verifier behavior, entropy masking	Protects the correctness of the training inputs, reward path, and objective-related state	Mostly software-level

Runtime and observability contracts	Memory management, profiler APIs, metric behavior, checkpoint metadata	Keeps the operational mechanisms aligned with the constrained execution model	Mixed
Substrate-dependent contracts	Triton correctness, CUDA-dependent behavior, generation under realistic runtime conditions, full trainer integration	Validates behavior that cannot be represented faithfully with pure CPU scaffolding	Hardware-dependent

Table 5.4: Test coverage aligned with architectural boundaries.

This split between lightweight and hardware-dependent tests follows directly from the architecture. Some contracts can be verified as ordinary software behavior: configuration parsing, checkpoint metadata, adapter enumeration, reward logic, or dataloader transformations. Others depend on the execution substrate itself: memory telemetry, Triton correctness, profiler output, or generation behavior under realistic runtime conditions. Treating both categories explicitly keeps the validation claim precise. It avoids the false impression that a constrained-system training engine can be validated completely without the constrained system on which it runs.

5.5.3 Regression protection for architecture-critical paths

Performance and memory validation also have a distinct role inside this design-for-testability approach. Files such as the Triton performance tests and the memory-manager tests do not replace the later benchmark chapters, but they do support the architecture by protecting critical assumptions from silent regression. They check that performance-oriented paths remain executable and that VRAM-related control logic behaves coherently under the scenarios it was built to manage. The test suite is an internal component of the architecture and a primary mechanism for checking system reliability. These checks help preserve operational integrity as the execution environment changes or optimization work continues.

The architecture was built around explicit control, and that control only improves the system if the controlled parts remain correct as they change. A manual LoRA path, a custom checkpointing policy, or a profiler-integrated trainer loop provides more leverage than a higher-level abstraction only if regressions can be detected before they distort benchmark conclusions or long-run behavior. The testing layer serves a stabilizing role that is architectural, not merely procedural. It keeps the engine from drifting away from the precise behavior that justified the low-level design in the first place.

6. MEMORY OPTIMIZATION AND VRAM MANAGEMENT

6.1 MEMORY CONSUMPTION ANALYSIS

6.1.1 RTX 3060 Ti (8 GB) VRAM budget

The central engineering constraint of the system is not simply that the target machine has a consumer GPU. The real constraint is that the full GRPO training workflow must execute inside the effective memory budget of a single RTX 3060 Ti, where the usable margin is smaller than the nominal 8 GB figure suggests. This is the core evidence for objective O3: managing limited GPU memory robustly rather than merely fitting the model once. In the benchmark logs used to characterize the accepted runtime, the device is reported as providing about 7.7 GB of VRAM to the process. This difference matters. Once driver overhead, allocator behavior, and transient phase spikes are taken into account, the engine is not operating with a comfortable reserve. It is operating inside a narrow margin where design choices that appear secondary on larger systems become decisive.

This constraint changes the meaning of optimization. On a larger machine, memory efficiency can be treated as a way to improve utilization after feasibility has already been established. Here memory efficiency is what makes feasibility possible in the first place. The project then does not frame VRAM management as a later improvement over an otherwise complete implementation. The implementation itself was shaped around VRAM from the beginning: model size, quantization path, adapter design, checkpointing policy, micro-batch sizing, retry semantics, and profiler instrumentation all serve the same requirement of keeping the training loop alive under a hard single-GPU limit.

The practical consequence is that the memory budget cannot be reasoned about only in terms of static parameter count. The relevant unit is the full step. Generation, verification, old-log-probability preparation, backpropagation, optimizer updates, and checkpoint decisions do not stress memory in the same way or at the same moment. VRAM is treated as a dynamic budget instead of a static value. Effective management requires tracking when allocations occur and how long tensors remain resident. To avoid crashing, the system must clear these transient states before the next spike in memory pressure.

6.1.2 Component-level footprint: model, activations, gradients, optimizer, KV cache

The main memory consumers in the runtime are structurally different from one another, and they cannot all be reduced by the same intervention. The frozen backbone is the largest persistent object in the system, which is why the project adopts 4-bit loading in the first place. Without that reduction, the model would consume too much of the card before generation or training even began. The LoRA

adapters, by contrast, are small enough that they do not define feasibility on their own. Their importance lies in restricting the trainable path so that gradients and optimizer state remain bounded. In setup, the engine reported 112 injected LoRA layers and 4,358,144 trainable parameters, about 0.39% of total model parameters. That ratio illustrates why adapter-based fine-tuning was essential for this hardware tier.

Activations and gradient-bearing tensors are the next major source of pressure. Unlike the frozen backbone, they are not always resident. They appear and disappear across the training phases, but their peaks are large enough to determine whether a step succeeds. This is why gradient checkpointing remains relevant even after quantization and LoRA have already reduced the static footprint. Quantization lowers the cost of keeping the base model resident. It does not eliminate the transient cost of forward and backward execution on long responses.

The optimizer contributes a different kind of burden. It is not as visible as the backbone or as intuitively obvious as activations, but it creates pressure exactly where a constrained system is most fragile: during updates and state maintenance. Both the literature and the local benchmark evidence show that optimizer choice is not a minor training detail in this regime. Paged optimizers were introduced in QLoRA specifically to mitigate memory spikes during fine-tuning [11], and consumer-GPU profiling later showed that they can also improve throughput while keeping long-sequence runs feasible within an 8 GB budget [4].

The final major consumer is the KV cache created during rollout generation. In reinforcement-learning post-training, this is especially important because the system must generate grouped responses before it can train on them. Long outputs therefore stress memory even before the backward pass begins. This is one reason why the project does not interpret response length as a purely algorithmic hyperparameter. Response length is also a memory-management choice, because it changes both the size of the rollout state and the pressure carried into later phases of the step. The accepted runtime keeps a bounded response cap for that reason. The empirical justification for the chosen cap is developed later in the benchmark chapter.

Table 6.1 condenses the operating-envelope evidence into one place. It shows how each response-cap regime changes speed, truncation pressure, and recovery behavior.

Response-cap regime	Evidence type	Step-time signal	Response / truncation signal	Recovery signal	Practical reading

512	40-step diagnostic	33.41s average, 26.26s median	421.67 average response length; truncation on 19/40 steps; majority truncation on 17/40; full truncation on 7/40	1 OOM backoff	Lower immediate cost, but high truncation pressure in a longer window
768	Accepted 40-step diagnostic	27.59s average, 25.45s median, 63.78 tok/s average	440.49 average response length; truncation on 9/40 steps; majority truncation on 7/40; full truncation on 3/40	0 OOM backoffs	Intermediate cap used in the accepted operating profile
1024	Matched sweep plus preserved train log	156.41s in the matched response sweep, 15.33 tok/s	Longer response budget, but a much narrower margin	two explicit early backoffs in the preserved benchmark log	Largest response budget, but narrowest operating margin

Table 6.1: Response-cap regimes, speed, truncation pressure, and recovery behavior.

6.1.3 Critical operating margin near the OOM boundary

The phrase "near the OOM boundary" describes a specific operating condition in this project, not a vague concern about stability. In the accepted training regime, the system often runs with only a few gigabytes of free headroom after setup, and that headroom must absorb phase-specific peaks whose size depends on response length, grouped sampling, gradient accumulation, active checkpointing state, and the current micro-batch schedule. Under those conditions, two runs with the same nominal configuration can behave differently if sequence realization, retry history, or transient allocator state changes the exact moment at which a peak occurs.

This is why the project does not treat OOM as a binary distinction between success and failure. The more useful distinction is between three operating regimes:

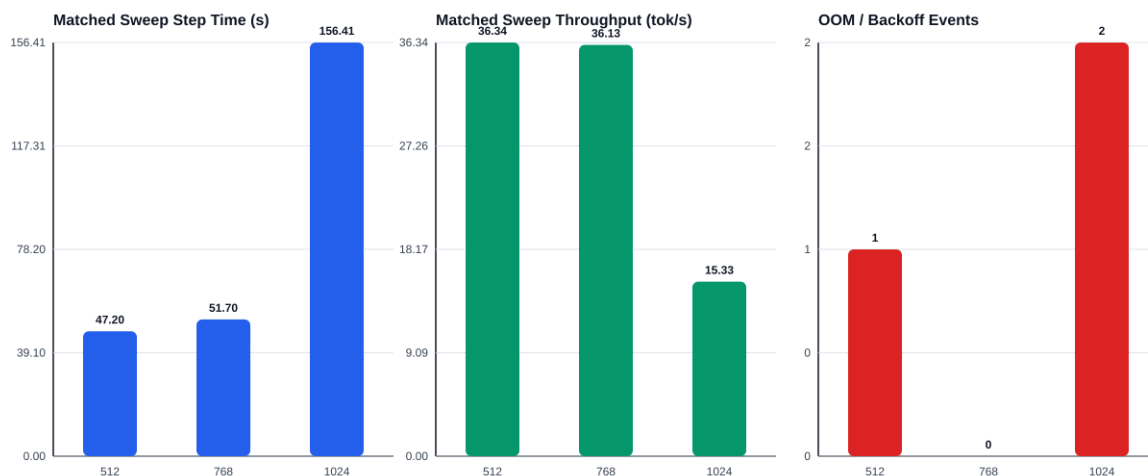
- Stable margin: the current micro-batch schedule and checkpointing policy are sustainable.
- Narrow margin: the step still completes, but a small change in sequence length or allocator state may cause a failure.
- Breached margin: the runtime must adapt immediately or abort.

Engineering for the middle band is one of the main contributions of the system. Many high-level stacks implicitly assume that one should either stay far away from the limit or accept occasional failures. On a consumer GPU, that is too expensive. Staying permanently far away from the limit wastes already scarce throughput, while accepting repeated failures destroys reproducibility and wall-clock efficiency.

The benchmark artifacts make this narrow-margin behavior concrete. In one older 1024-response benchmark run, the engine backed off twice at the beginning of training, moving from `gen_batch=4/train_batch=4` effectively down to `2/2` and then `1/1`. That result shows what breached margin looked like on the target hardware. By contrast, the preserved 768 diagnostic kept both generation and training micro-batches stable at 4 throughout the run and completed 40/40 steps without OOM backoff. Average speed alone is therefore not enough. On this hardware, the relevant question is whether the configuration preserves a survivable operating margin across whole steps.

Response Cap Trade-offs

Matched sweep timing is paired with preserved diagnostic and train-log OOM evidence to show where the 1024-token regime becomes fragile.



Sources: raw/project_experiments_and_results.md and 2026-04-15-benchmark-artifacts/benchmarks/output/ing_inf/triton_on_response_len_1024/train.log

Figure 6.1: Response-cap trade-offs across the preserved evidence.

Figure 6.1 combines the matched response-length sweep with preserved diagnostic and train-log recovery evidence. It provides a compact view of how the response cap changes the memory envelope and recovery burden. Chapter 8 returns to the same evidence to justify the adopted default more directly.

6.2 QUANTIZED BACKBONE AND PRECISION STRATEGY

6.2.1 4-bit NF4 loading in the runtime

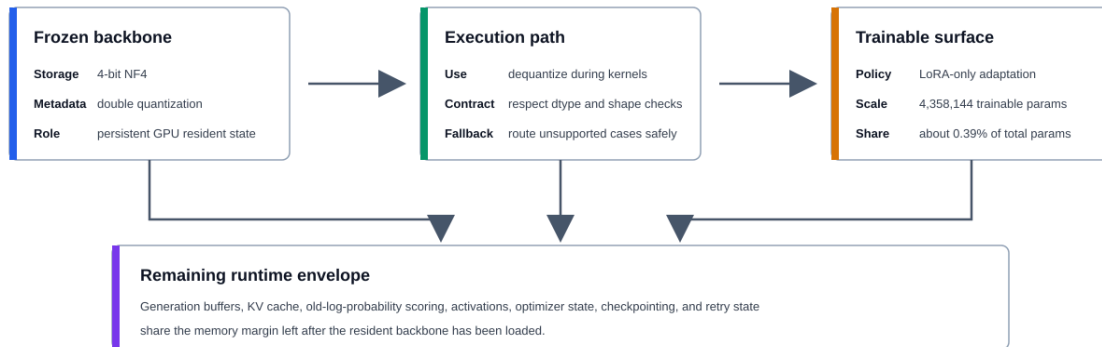
The first role of quantized loading in this system is startup feasibility. The training loop begins by loading the tokenizer and the full base model before any rollout, reward computation, or update work can even be attempted. That loaded backbone then remains present across the whole run while generation buffers, activations, cached log probabilities, optimizer state, and checkpoint-side tensors appear and disappear around it. On a single RTX 3060 Ti, that makes the backbone the fixed part of the memory budget. If that fixed part is too large, every later control mechanism becomes secondary because the runtime starts from an already exhausted baseline.

For that reason the loader keeps the frozen base in a bitsandbytes-backed 4-bit NF4 configuration, using the same library-backed quantized loading path reviewed in the low-VRAM stack discussion, and treats that choice as part of the runtime envelope rather than as an isolated model-format preference [11]. The practical effect is that the engine establishes a compact resident model first and then layers the rest of the training step on top of it. This is consistent with the broader architecture presented earlier: the model loader is intentionally narrow, and its main job is to create a predictable baseline footprint on which the rest of the runtime can depend. That predictability leaves room for grouped generation, old-policy preparation, adaptive checkpointing, and retry logic without forcing all of those mechanisms to compensate for an oversized startup state.

The memory benefit is also structural. The backbone is the one large object that the runtime cannot simply clear between phases. Activations can be recomputed. Caches can be emptied. Temporary rollout state can be moved or released. The loaded base model cannot. Quantizing that persistent object therefore changes the shape of the whole budget more than many local optimizations elsewhere in the step. The resulting design is asymmetric by intent: the backbone is stored in the most compact form the runtime can use reliably, while the trainable path is reduced by other means, adapter-only optimization. Quantized loading therefore remains a memory-strategy question even after the underlying quantization ideas have been introduced earlier.

Quantized backbone and precision split

The runtime compresses the large resident object while keeping the small trainable path inspectable and recoverable.



Design rule: compress the large persistent object, keep the small trainable path simple, and reserve headroom for phase peaks.

Figure 6.2: Precision split used by the runtime.

Figure 6.2 shows the practical split behind Section 6.2. The base model is compressed because it is the large object that stays resident, while the update path remains adapter-only so gradients, optimizer state, and retry logic stay within a small and inspectable trainable surface.

6.2.2 Double quantization: savings and added cost

In this runtime, double quantization matters for the same reason as the main 4-bit loading choice: the project cannot afford to discuss memory savings only at the level of nominal weight width. What stays resident on the device includes not only the compressed weights but also the auxiliary information required to use them. If that auxiliary state grows enough, the headline gain of low-bit loading becomes less useful in practice. Preserving double quantization is therefore a way of protecting the end-to-end residency reduction rather than accepting a partial saving on paper and then losing part of it to reconstruction metadata [11].

That point becomes more important when the training step is viewed as an envelope instead of a snapshot. The resident backbone must coexist with rollout KV cache, grouped completions, reward-side tensors, cached old-policy values, checkpointing decisions, and occasional retry overhead. The relevant question is not whether the model can be loaded once under ideal conditions. The relevant question is whether it can remain loaded while the rest of the system passes repeatedly through its higher-pressure phases. Double quantization contributes to that margin even though it does not remove a visible peak by itself. Its effect is to lower the constant background occupancy against which every later peak must be absorbed.

A compressed resident backbone still has to be used by real kernels. During execution, the runtime pays for dequantization and for the constraints that come with bitsandbytes-backed 4-bit layers. Those constraints propagate outward. The manual LoRA wrappers have to sit on top of quantized base layers. Selective Triton paths have to respect that underlying representation or fall back cleanly when they cannot. Testing has to cover fake and real 4-bit paths so loader and adapter logic do not silently diverge between environments. Double quantization is a design commitment that shapes the surrounding implementation. On this hardware, that commitment is still favorable because a little more arithmetic and integration work is cheaper than starting every step with a larger resident footprint.

6.2.3 Storage precision vs compute precision in fine-tuning

The decisive runtime issue is not the abstract distinction between storage precision and compute precision, but how that distinction is used to divide responsibilities inside the system. The frozen backbone is compressed because it dominates persistent residency. Computation is not forced into the same low-bit format because the execution path still has to support grouped generation, repeated forward passes, adapter updates, and occasional replay after failure. Those are different pressures, and the runtime addresses them with different precision choices rather than trying to solve every problem with one global format policy.

This split also clarifies why the project does not pursue blanket quantization of every trainable component. The accepted operating profile keeps the major saving where it matters most, in the resident base model, while leaving the trainable LoRA path in a simpler update regime. The adapters account for a very small fraction of total parameters, so compressing them aggressively would recover far less memory than compressing the backbone, yet it would make the update path harder to inspect and maintain. In a runtime already balancing checkpointing, backoff, and selective kernel specialization, that trade is poor. The system therefore uses low-bit storage where it changes feasibility and avoids forcing extra complexity into the smaller trainable surface.

The same logic applies to retry behavior and long-run stability. When a step has to be retried, the engine must preserve enough numerical reliability that replay does not become another source of drift or fragile behavior. A compressed backbone combined with a higher-precision compute path is a workable compromise for that requirement. It reduces the fixed memory cost without making every forward and update phase depend on an aggressively quantized arithmetic path. Consumer-GPU profiling supports that broader reading: low-VRAM fine-tuning remains feasible only when quantized residency, optimizer choice, sequence length, and precision configuration are treated as one coupled operating profile rather than as independent switches [4]. The precision strategy in this runtime follows the same logic. It is a specific design for keeping the persistent model small enough, the trainable path simple enough, and the full execution loop stable enough to run near the 8 GB boundary.

6.3 GRADIENT CHECKPOINTING AND VRAM-DRIVEN ACTIVATION POLICY

6.3.1 Computation-memory trade-off

Gradient checkpointing remains one of the central memory controls in the system because it addresses a different problem from quantized loading. Quantization shrinks the persistent backbone. Checkpointing shrinks the temporary activation burden by trading recomputation for residency. QLoRA already treated paged optimizers and checkpointing as complementary tools for surviving memory spikes during fine-tuning [11]. The local system extends that logic into a more dynamic policy suited to the phases of reinforcement-learning training.

This trade-off is not purely theoretical. The RLHF memory study found that checkpointing can reduce memory pressure effectively when the main peak occurs in the training phases, while also showing that it does not erase fragmentation effects by itself [13]. That observation matches the project well. The runtime has to manage both activation burden and allocator behavior. Checkpointing is therefore necessary, but it is not sufficient on its own.

Because the project operates close to the OOM boundary, the computation cost of checkpointing is accepted as part of normal operation. This would be a poor default on hardware where memory is abundant and throughput is the only target. On the RTX 3060 Ti, it is a rational exchange. Losing some time to recomputation is preferable to losing the entire step to a memory spike that then forces a destructive retry path.

6.3.2 Selective checkpointing in practice

The project does not use checkpointing as a simple on-or-off flag applied blindly to the whole model. The runtime exposes selective checkpointing controls so the mechanism can be applied at a granularity closer to the actual memory bottlenecks. Not all modules contribute equally to peak activation pressure, and a constrained system should avoid paying recomputation cost where little memory is saved in return.

The memory manager reflects this by iterating over checkpointable modules, selecting the subset governed by the configured policy, and applying checkpointing flags dynamically. This design integrates checkpointing into active runtime control instead of treating it as a one-time startup setting. The system can then adjust memory behavior during training, including sudden spikes in VRAM demand. Checkpointing can be activated, deactivated, or updated in response to observed pressure instead of only to what the configuration predicted in advance.

This selective design also fits the general engineering stance of the project. The point is not to prove that checkpointing should always be used everywhere. The runtime exposes enough control for the mechanism to be applied where it helps the most. That is consistent with the broader preference for

selective rather than ideological optimization that also appears in the Triton story and in the OOM-recovery path.

6.3.3 vram_auto thresholds, update interval, and activation logic

The runtime goes beyond static checkpointing and uses a VRAM-driven policy identified in the configuration as `vram_auto`. In the benchmark-backed configuration artifacts, this policy is parameterized by an enable threshold of 0.82, a disable threshold of 0.72, and an update interval of 10 steps. Checkpointing is not toggled opportunistically at every minor fluctuation. The controller is designed with hysteresis and periodic reevaluation so that it does not thrash between states.

This design became more important after the adaptive OOM-recovery refactor. The preserved OOM commit note shows that the system now preserves optimizer-step peaks before making `vram_auto` decisions, specifically so that spikes created during gradient clipping or `optimizer.step()` are not hidden from the controller. This closes a common blind spot in memory management: if the controller observes only the calmer parts of the step, it can conclude that checkpointing is unnecessary precisely when the dangerous peak is still being missed.

The result is a checkpointing policy that behaves more like a pressure-sensitive controller than like a compile-time toggle. That fits this hardware regime because the memory load seen by the runtime depends on response length, grouped sampling, micro-batch schedule, and retry history. Under those conditions, a fixed checkpointing stance is often too rigid. The `vram_auto` controller is the project's answer to that rigidity.

vram_auto checkpointing hysteresis

Two thresholds prevent rapid toggling when measured GPU memory pressure fluctuates near the limit.

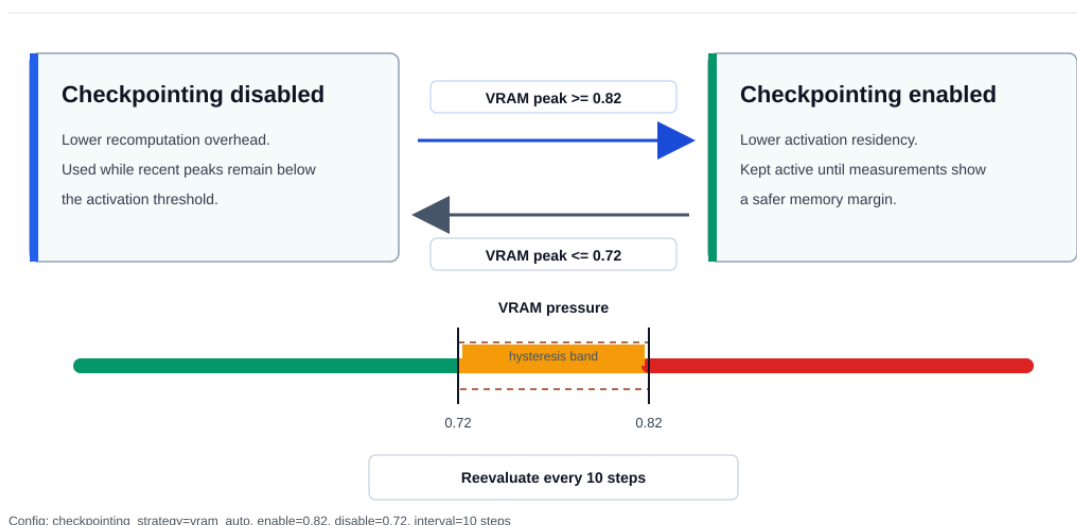


Figure 6.3: vram_auto checkpointing hysteresis in the accepted configuration family.

Figure 6.3 translates the preserved config values into the control logic they imply. The important engineering feature is the gap between the enable and disable thresholds: checkpointing is activated under pressure, but it is not disabled again until later measurements show a clearly safer margin.

Table 6.2 summarizes the accepted memory-control surfaces discussed so far.

Control surface	Accepted or preserved setting	Operational purpose
Backbone storage	4-bit NF4 with double quantization	Keep the persistent model footprint inside the single-GPU budget
Trainable surface	LoRA-only adaptation; observed trainable surface 112 layers, 4,358,144 parameters (0.39%)	Bound gradients and optimizer state to an adapter-only path
Rollout grouping	group_size = 4	Preserve GRPO group-relative behavior without crossing the stable memory envelope
Response cap	max_response_length = 768 in the accepted preset	Hold a workable line between truncation pressure and OOM fragility
Checkpointing policy	vram_auto	React to observed pressure rather than fixing one static checkpointing stance
vram_auto thresholds	enable at 0.82, disable at 0.72, update every 10 steps	Add hysteresis so checkpointing does not thrash

Table 6.2: Accepted memory-control surfaces.

6.4 MEMORY MANAGER AND RUNTIME RELEASE POLICIES

6.4.1 Cache clearing, fragmentation control, and peak tracking

The memory manager is the operational center of the runtime because it turns memory policy into concrete behavior. Its responsibilities include:

- clearing cached GPU memory between pressure-sensitive phases;
- collecting VRAM telemetry during execution;
- warning when usage approaches dangerous levels;
- tracking peak statistics as well as current statistics;
- controlling checkpointing decisions when the runtime is under pressure.

The module exposes both current statistics and peak statistics because the current memory state alone is not enough to understand a constrained reinforcement-learning training step. The highest pressure reached within a phase is often more informative than the memory still resident after that phase has partially unwound.

Cache clearing is especially important in this setting because the project is not a single uniform forward-backward loop. It alternates between phases with different allocation patterns, and that makes fragmentation more dangerous. The RLHF memory study is particularly useful here. It found that large memory gaps can emerge between reserved and allocated memory because fragmentation accumulates across phases, and that calling `empty_cache()` after the right phase can substantially reduce that overhead [13]. The local system follows the same insight in spirit: memory cleanup is treated as a part of the mechanism that keeps the next phase from inheriting a harmful allocator state.

This is why allocated memory and the effective operational cost of the allocator must be distinguished carefully. A step can fail not only because active tensors are too large in the abstract, but because cached and fragmented blocks make the next large request impossible to satisfy. Peak tracking and explicit cleanup are the two tools the project uses to make that behavior visible and partially controllable.

6.4.2 CPU-backed intermediate state and staged GPU transfer

The newer recovery design no longer treats prepared rollout state as something that must remain fully resident on the GPU throughout retries. The commit 30ad07e OOM-recovery refactor changed the retry lifecycle in four steps:

1. Successful prepared rollout state is preserved instead of being discarded after a later failure.

2. Prepared rollout tensors are moved to CPU memory so they do not remain in the hottest VRAM tier.
3. Old-log-probability and training slices are streamed back to the GPU only when the retry needs them.
4. The retry operates on preserved state rather than regenerating or recreating the full GPU-side rollout.

The change shows that memory management requires both tensor compression and strict control over data placement. The architecture must coordinate where tensors are stored and the exact timing of their transfer back to the device to keep the training loop within VRAM limits.

This policy is especially valuable when long sampled rollouts are involved. In a naive retry design, the system would regenerate or fully recreate large GPU-side rollout state whenever an OOM occurred later in the step. That is expensive in throughput terms and self-defeating in memory terms, because the retry is forced to rebuild exactly the kind of oversized state that already contributed to failure. CPU-backed prepared state breaks that loop. It makes later recovery strategies technically possible because successful earlier work can be preserved without forcing the same tensors to stay resident in the most constrained memory tier.

From a systems perspective, this is a better use of the CPU-GPU boundary than generic offloading. The project is not trying to offload everything indiscriminately. It moves specific intermediate state out of the hot memory tier so that later retry logic has a preserved base to work from. That selective movement fits the rest of the architecture better than a broad offload-first policy would. Data placement determines what state can survive a failure, while the decision rules for using that preserved state are part of the recovery mechanism itself.

6.4.3 Aggressive recovery and memory-pressure responses

The memory manager also underpins the more aggressive responses needed when ordinary cleanup is not enough. A constrained runtime needs a graded response ladder rather than one reaction for every rise in memory pressure:

- warn when usage approaches a dangerous region;
- clear cache more forcefully when ordinary cleanup is insufficient;
- reset peak statistics after policy changes so later decisions are not based on stale pressure readings;
- feed the trainer with usage information for checkpoint decisions and recovery actions;
- support execution-schedule changes when cleanup and checkpointing no longer provide enough margin.

Some conditions require only cleanup. Others require checkpoint activation. Others require a change in the execution schedule.

This layered response is preferable to a simplistic "run until failure, then shrink everything" policy. The benchmark evidence shows why. In fragile long-response settings, a coarse recovery path that only reduces the whole workload can save the run, but at a steep throughput cost. The memory manager does not by itself decide the full recovery semantics, but it provides the measurements, cleanup hooks, and state transitions that make a more selective policy implementable. It is therefore the substrate on which the full recovery mechanism depends.

6.5 PHASE-AWARE OOM RECOVERY

6.5.1 Detecting failure by phase

The most important memory-management evolution in the project is that OOM handling is now phase-aware. Earlier designs naturally tend toward a whole-step interpretation of failure: a step fails somewhere, the system shrinks the workload, and the step is replayed from the beginning. The training step is now split into rollout generation, old-log-probability preparation, and training/update phases, so a failure can be localized to the part of the step that actually crossed the limit.

This affects both correctness and efficiency. If a training-phase OOM forces the runtime to regenerate the full rollout, the retry pays again for work that had already succeeded and may not even be the source of the failure. More importantly, if generation includes stochastic sampling, replaying the whole step can change the data on which the retry is supposed to act. The new design avoids that by preserving rollout state and retrying only the failing phase. Architecturally, that aligns the recovery path with the structure of the training loop.

6.5.2 Independent generation and training micro-batch backoff

Once failures are phase-localized, the obvious next step is to stop treating micro-batch adaptation as a single global scalar. The newer design does exactly that. Generation and training micro-batches now back off independently, and they can also regrow independently when headroom returns. This is a concrete project mechanism, not a generic reference to "dynamic batch reduction."

The configuration artifacts show the governing parameters of this policy in the accepted system family: initial and maximum generation/training micro-batches, a backoff factor of 0.5, a probe interval of 12, a cooldown of 4, and a probe-usage threshold of 0.75. At runtime, the policy behaves as a control loop:

1. The runtime localizes the failure to the generation side or the training/update side.

2. It reduces the affected micro-batch independently instead of shrinking the whole step by default.
3. The reduction follows the configured 0.5 backoff factor within the generation and training micro-batch bounds.
4. The runtime waits through the configured cooldown of 4 steps before treating the schedule as safe to probe again.
5. Every 12 steps, it tests whether regrowth is safe when observed usage stays below the 0.75 probe threshold.
6. If the probe succeeds, the affected micro-batch can grow again instead of remaining permanently conservative.

The runtime therefore does more than shrink after failure and stay permanently conservative. It periodically tests whether safe regrowth is possible, so the policy adapts in both directions.

The benchmark artifacts show why this is useful. In the older 1024-response benchmark family, both generation and training were forced downward quickly after early failures. In newer follow-up runs, the logs show more nuanced behavior: training-phase failures can reduce only `training_micro_batch`, and successful later steps can recover some of that lost size when the observed headroom permits it. That refinement matters on a single consumer GPU. A system that can only shrink is stable but pessimistic. A system that can shrink and later regrow is better aligned with the real, fluctuating memory profile of reinforcement-learning training.

6.5.3 Replay-safe retries, regrowth probes, and resume consistency

The most technically demanding part of the new recovery design is not the backoff itself. It is the insistence that retries and resumes remain replay-safe. During flush-time OOM handling, the trainer captures and restores the accumulation window transactionally, including:

- gradients;
- accumulation counters;
- optimizer state;
- scheduler state;
- replay metadata;
- torch CPU/CUDA random-number-generator (RNG) state across visible devices.

It also persists adaptive micro-batch state and replay-related RNG state in checkpoints so resumed jobs continue from the last safe schedule instead of reverting to defaults.

This is a substantial improvement over a naive retry path. Without transactional restoration, a retry can silently double-count part of a batch, change dropout masks, regenerate different sampled responses, or mutate optimizer state before the rerun actually begins. All of those effects are

especially damaging in a project whose contribution depends on profiler-backed and benchmark-backed comparison. If retries change the computation semantically, then memory recovery also corrupts comparability.

The replay-safe design prevents that drift. It treats recovery as part of the algorithmic integrity of the runtime rather than as a crude operational patch. The project shows that OOM can be survived while preserving the meaning of the interrupted training step as closely as possible.

Table 6.3 summarizes the recovery contract.

Recovery mechanism	Visible behavior	Why it matters under 8 GB VRAM
Phase-local failure detection	Distinguishes rollout generation, old-log-prob preparation, and training/update failures	Avoids rerunning successful work after a later-phase OOM
Independent generation / training backoff	Generation and training micro-batches shrink separately	Keeps one fragile phase from forcing a global overreaction
Regrowth probes	The schedule can increase again when headroom returns	Prevents the runtime from becoming permanently pessimistic after one spike
CPU-backed prepared rollout state	Prepared rollout tensors are preserved off-device and streamed back in slices	Prevents retries from rebuilding oversized GPU state unnecessarily
Replay-safe transactional restore	Gradients, accumulation counters, optimizer state, scheduler state, and RNG state are restored before retry	Preserves comparability and prevents silent semantic drift

Checkpointed adaptive state	Recovery state and replay base state survive resume boundaries	Keeps interrupted jobs aligned with uninterrupted execution
--------------------------------	---	--

Table 6.3: Phase-aware OOM recovery contract.

The recovery design reflects the logic of the memory subsystem. Optimization depends on combining low-bit model residency and bounded state with pressure-sensitive checkpointing and explicit cleanup. CPU offloading for intermediate tensors and resume semantics support that design by protecting state during retries. The engine fits within a single GPU because the runtime manages memory pressure actively. Chapter 8 evaluates these choices through benchmark analysis, using empirical evidence to avoid relying on intuition alone.

7. TRAINING PIPELINE IMPLEMENTATION

The training runtime executes one complete optimization step through a phase-explicit pipeline that keeps generation, pre-computation, and backward/update work separated closely enough to remain inspectable on a single 8 GB GPU. The project implements GRPO-style post-training through a runtime that keeps grouped rollouts, temporary decode state, frozen-policy scoring, and adapter-only optimization under direct control even though those operations do not place the same demands on memory or on execution mode [2] [16] [19].

7.1 END-TO-END PIPELINE STRUCTURE

The architectural structure described earlier becomes operational here as a repeatable step contract. Each iteration begins from a prompt batch that has already been formatted, grouped, and transferred into the runtime under the same working assumptions as the accepted local execution path. The central issue is how one batch is transformed as it passes through the step. The pipeline matters because each stage consumes a different representation of the same underlying example and prepares the exact state required by the next stage.

The complete step is organized around three phases. The first is rollout generation. The second is pre-computation, where responses are verified, filtered, transformed into grouped rewards, and converted into the tensors needed for optimization. The third is the training/update phase, where the adapter parameters receive gradients and the optimizer state advances. This division lets the runtime apply different execution rules at each stage. Generation is inference-like and dominated by decode behavior and cache growth. Pre-computation is a mixed stage in which text outputs are converted into compact numerical state. Training is the only phase in which activations, accumulated gradients, and optimizer state all need to coexist on the device.

The main execution handoff is between representations rather than between abstract modules. The generation phase emits sampled response tokens, decoded text, and response-local masks. Pre-computation consumes those artifacts and reduces them to a smaller training bundle: verified rewards, old log probabilities, centered advantages, and the masking tensors that define where loss is allowed to flow. The update phase then operates only on that compact bundle instead of carrying the full textual and decode-time state forward. This reduction in representation size is one of the practical reasons the pipeline remains legible. The later phases do not need all the objects created by the earlier ones, and forcing them to keep those objects alive would blur both the memory profile and the logical meaning of the step.

Execution mode also changes across those handoffs. Generation runs under evaluation-style semantics without gradient tracking. Pre-computation mixes verification logic, mask construction, and frozen-policy scoring, which means it still depends on model outputs but is not itself an update

phase. Training and update then re-enter the mode in which gradients are accumulated on adapter parameters and optimizer state is allowed to advance. This sequence matters because it gives the runtime a predictable order for allocation, cleanup, and metric interpretation. Step-local telemetry becomes easier to read when decode-heavy work, transformation-heavy work, and backward-heavy work are not collapsed into one blended block.

The end-to-end pipeline is a typed flow of state. Prompt batches become grouped textual rollouts. Grouped rollouts become verified and scored numerical bundles. Those bundles become adapter updates and step metrics. The rest of the chapter can then focus on what each phase does to the state it receives and what constraints that imposes on the next phase.

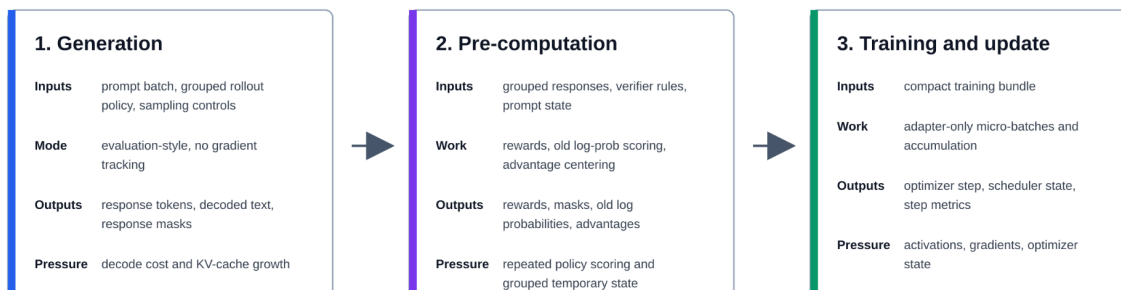
Table 7.1 is the compact version of that contract. It shows what each phase consumes, what it emits, and why the runtime keeps them separated instead of hiding them in one opaque trainer call.

Phase	Dominant model mode	Main inputs	Main outputs	Main pressure point
Generation	Evaluation-style, no gradient tracking	Prompt batch, grouped rollout policy, sampling controls	Response tokens, decoded text, response masks	Decode cost, KV-cache growth, response-length pressure
Pre-computation	Mixed inference / transformation path	Responses, verifier rules, prompt state, grouped structure	Rewards, response-only masks, old log probabilities, centered advantages	Repeated policy scoring and temporary grouped state
Training and update	Training mode on adapter parameters only	Compact training bundle from pre-computation	Gradient updates, optimizer step, scheduler step, step metrics	Activations, gradients, optimizer state, micro-batch sizing

Table 7.1: Three-phase training-step contract.

Explicit three-phase training pipeline

Each step narrows the representation before gradients and optimizer state are allowed to coexist on the GPU.



Sources: /src/grpo/trainer.py

Figure 7.1: Explicit three-phase training pipeline in the local runtime.

Generation, pre-computation, and update work do not only differ conceptually; they consume different tensors, run under different model modes, and expose different memory risks.

7.2 GENERATION PHASE

The generation phase starts from normalized mathematical prompts already formatted in the same conversational style expected by the target model. Before any optimization tensors are built, the runtime follows four steps:

1. The data path preserves the original example identity, keeps prompt formatting stable, and can present examples in a SENT-ordered semantic-entropy curriculum instead of plain dataset order.
2. Once the batch reaches the trainer, each prompt is expanded into a rollout group. In the accepted operating profile this grouped sampling remains at `group_size = 4`, which is large enough to support group-relative rewards while still fitting within the hardware envelope of the target system.
3. Generation runs as a dedicated inference step rather than as a side effect of training. The runtime enters an evaluation-style context, disables gradient tracking, and produces multiple candidate responses per prompt.
4. The trainer removes the prompt tokens from the generated sequences before later phases consume them. The artifacts passed forward are response tokens, response masks, and

decoded text, not the full concatenated prompt-plus-response sequence as seen internally by the model.

This contract keeps the decode path cleanly separated from the later backward phase and reduces the amount of accidental state carried forward. Integration tests cover prompt-token removal directly because including prompt tokens in the response path would contaminate both scoring and loss masking.

The most important systems consequence of this phase is that response length immediately becomes a runtime resource decision. Every additional generated token extends the decode horizon and enlarges the working set associated with grouped rollouts. The long-response reinforcement-learning literature describes the same pressure from another angle: as rollout traces grow, KV-cache residency and decode-time bandwidth costs start to dominate throughput and eventually constrain the feasible rollout batch [16]. The local implementation therefore treats sampling controls, response caps, and dataset compatibility as part of pipeline design.

Although the project can route parts of generation through optional Triton-assisted paths, the pipeline does not assume that one generation backend is always dominant. The operational requirement is simpler: generation must return grouped responses correctly, must leave the model in a recoverable state for the subsequent phases, and must avoid retaining decode-time state longer than necessary. The benchmark-backed comparison between generation backends is treated separately from the execution-path description.

This requirement became more visible once the same pipeline was used beyond the initial benchmark dataset. Harder math datasets can be structurally supported by the loader and still be poor workloads for the accepted 768-token envelope if most sampled completions run into the cap. The trainer does not need a different execution path for those datasets. The same normalized split format, prompt contract, SENT cache compatibility checks, and training loop apply; the question of whether a dataset is a good learning target belongs to the experimental analysis rather than to a separate trainer architecture.

7.3 PRE-COMPUTATION PHASE

Once grouped responses are available, the pipeline moves into a conversion stage that transforms textual rollouts into compact optimization inputs. The order matters because each object produced here is consumed either by scoring or by the later backward pass:

1. The verifier extracts candidate final answers from each response using a cascading set of lightweight rules, normalizes numerical forms, and compares the extracted answer with the ground truth. This avoids the memory cost of a learned reward model while keeping the reward semantics tied directly to the task.

2. The verifier treats incomplete or truncated outputs conservatively. Incorrectly rewarding a cut-off completion would damage both the learning signal and the later incident analysis.
3. The response text is paired with response-only masks that define where loss is allowed to flow. Prompt positions are excluded from that path, and active truncation masking can also remove truncated completions from the loss contribution so that clearly incomplete samples do not distort the grouped update.
4. The generated text is reduced to the objects that the optimization phase can consume: rewards, response tokens, attention masks, and response-local supervision masks.
5. The trainer computes old log probabilities for the sampled response tokens because the clipped objective depends on a comparison between the current policy and an older policy state [3]. Instead of densely recomputing every prompt-response pair, prompt prefill can be computed once, the prefix state can be reused across the grouped responses attached to that prompt, and response scoring can then proceed over the expanded group.
6. After scoring, the trainer computes group-relative advantages. In the implemented variant, rewards are centered within each rollout group instead of being left as independent scalar values, which keeps the update tied to relative performance inside the local group [2] [19].
7. When truncation-aware masking is enabled, the same grouping logic can exclude invalid or masked samples before the centered advantages are finalized.

Tests compare the cached prompt path with dense references and verify that the resulting old log probabilities produce the same loss values. This matters because the grouped rollout structure would otherwise multiply prompt-side work unnecessarily. The output of the pre-computation phase is therefore no longer tied to raw decode artifacts. It is a compact, phase-ready training bundle that can be consumed in micro-batches during the only part of the loop where gradients are allowed to flow.

7.4 TRAINING AND UPDATE PHASE

The training phase is intentionally narrower than the earlier generation stage. Only the adapter parameters are updated, while the quantized backbone remains frozen in the QLoRA-style adaptation pattern that makes local feasibility possible in the first place [11]. It sharply reduces the amount of trainable state that must coexist with activations, cached old-policy scores, and optimizer buffers during the backward pass. The consequence is that the runtime can spend its limited VRAM budget on the phase-local tensors that actually change from step to step instead of on a full-model training surface.

Execution within this phase is micro-batched. The trainer does not assume that the grouped rollout tensors produced earlier can be consumed safely in one large backward pass. The update path is therefore organized as an ordered runtime sequence:

1. The compact training bundle is sliced into smaller units, and those slices are processed sequentially. This preserves the semantics of the larger grouped step while keeping the live activation footprint under tighter control.
2. When optional reference-style logits or entropy-mask logic are enabled, they are integrated here rather than during generation, because this is the only phase where the loss surface is assembled explicitly.
3. Several micro-batches may contribute normalized gradients before a flush occurs, so the optimizer step is not tied one-to-one to every micro-batch.
4. When a flush is due, the trainer clips gradient norms, advances the optimizer, updates the scheduler, and clears gradients before the next accumulation window begins.
5. Once the update is complete, the trainer records step-local metrics and prepares the system for the next batch. Step time, tokens per second, reward statistics, truncation behavior, micro-batch state, and VRAM telemetry are all part of this finalization step.

Integration tests cover the less obvious corner cases of this behavior, including partial accumulation windows at the end of an epoch and the need to rescale those windows correctly before stepping. Without that care, a narrow-memory pipeline would gain feasibility at the cost of silently changing the effective optimization scale from one part of the run to another.

The update phase also preserves the distinction between the main operating preset and optional code paths retained for completeness. The implementation can compute reference logits and KL-style penalties when configured, and it verifies that these frozen-policy passes really run with adapters disabled where required. The default operating configuration avoids a heavy always-on reference-model path because the primary engineering target is still stable local execution.

The result is that one training iteration ends as a bounded sequence whose internal states can still be inspected after the fact. The pipeline keeps broader algorithmic hooks available, but the main path remains the one that fits the hardware regime cleanly.

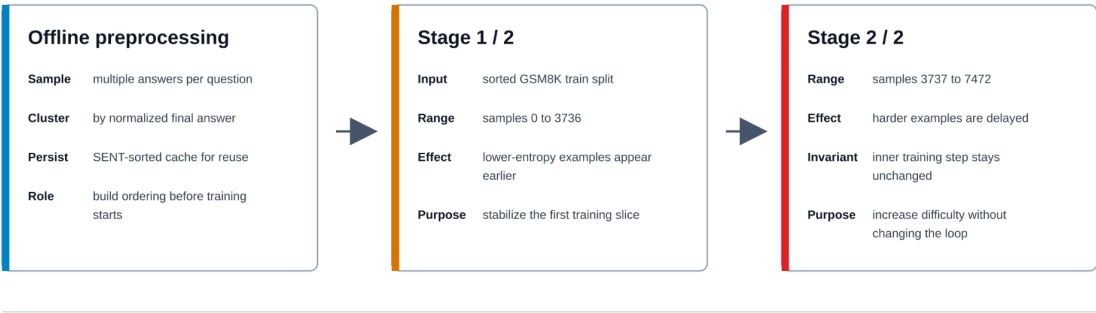
7.5 OPTIONAL PIPELINE ENHANCEMENTS

Two optional mechanisms modify the pipeline without changing its basic phase structure. The first is entropy-based selective training. In this path, token-level entropy is computed from the model logits over the response region, and a binary mask is constructed so that only the selected response positions contribute to the loss. Prompt positions are excluded by construction, and alignment tests exist precisely because a selective-loss mechanism becomes dangerous if its mask shape or token offsets drift away from the tensors used by the GRPO loss. The optional Triton path can accelerate parts of this calculation, but the conceptual role of entropy masking inside the pipeline is simpler: it changes which response tokens carry gradient, not how the three main phases are sequenced.

The second mechanism is curriculum ordering through semantic entropy. SENT is prepared offline rather than inside the main training step. A separate preprocessing workflow samples multiple responses per question, clusters them by normalized final answer, estimates semantic entropy, and saves a cache with compatibility metadata. During training, the data loader can validate that cache against the active prompt format, tokenizer context, and model identity, then expose the dataset in staged difficulty order. Stage advancement happens outside the inner training step, so the core pipeline remains unchanged. What changes is the order in which examples arrive and, consequently, the difficulty profile seen across epochs.

SENT curriculum ordering

Offline preprocessing creates a sorted cache; the dataloader exposes it as two staged slices from easier to harder examples.



Sources: /src/data/gsm8k_loader.py, /scripts/preprocess_sent.py

Figure 7.2: SENT curriculum staging as used by the runtime.

Figure 7.2 shows the useful part of SENT for this explanation: not the full cache internals, but the way offline preprocessing produces a sorted artifact that the dataloader later exposes as staged slices of the active training split.

These two mechanisms were integrated as modifiers rather than as foundations of the system. That design choice is intentional. The project could not allow curriculum support or selective masking to become preconditions for basic feasibility, because the whole runtime still had to remain understandable and recoverable when those features were absent, disabled, or invalidated by configuration. In practical terms, the three-phase pipeline remains the stable core. Entropy masking and SENT alter what happens within that core, but they do not redefine it. Their speed, stability, and benchmark consequences are therefore evaluated later as engineering choices rather than being treated here as prerequisites for understanding the execution path.

8. PERFORMANCE ENGINEERING, TRITON, AND BENCHMARK-DRIVEN OPTIMIZATION

Performance engineering in this project was constrained by a condition that changes the meaning of optimization itself: the target runtime had to remain usable on a single 8 GB consumer GPU while still executing grouped GRPO-style training, old-policy scoring, and adapter updates. Objective O2 required end-to-end computational efficiency rather than isolated kernel speed. Under that constraint, a local speedup is not automatically useful. A change may improve one kernel while increasing decode fragility, it may reduce one benchmark window while worsening long-response behavior, or it may look attractive only because it silently shifts work into a later phase. The accepted operating profile therefore emerged from a narrower discipline. Profiling was used to identify where time and memory pressure were accumulating, Triton was introduced only where specialization had a plausible payoff, and promotion decisions were taken through bounded benchmark evidence rather than through intuition alone.

The resulting engineering story is narrower than a simple "Triton made everything faster" narrative. Some Triton paths survived and became part of the accepted stack. Some remained useful only as optional fast paths. Others consumed substantial development time without earning a place in the default operating profile. That asymmetry is part of the main technical result. On constrained hardware, good performance work is not defined by how many specialized kernels are written, but by whether the final system becomes faster without becoming harder to trust.

8.1 PERFORMANCE-ANALYSIS METHODOLOGY

The first performance decision was methodological rather than low-level: benchmark output was treated as the promotion authority. Console impressions remained helpful during debugging, especially when a kernel crashed immediately or when an edit produced an obvious local regression, but they were not strong enough to justify keeping or rejecting an optimization. The system instead relied on a layered measurement process. Short suites were used to screen candidate configurations. Matched comparison runs were used to compare nearby variants under controlled conditions. Longer diagnostics were used to test whether a short-window gain survived once response lengths, allocator state, and retry behavior had time to evolve.

This distinction mattered because the project was operating close to two boundaries at once. One boundary was wall-clock cost. A broad search over Triton toggles, response lengths, group sizes, gradient accumulation settings, and adapter variants would have consumed too much time if every candidate had required a long run. The second boundary was measurement distortion. Deep profiling itself can change runtime behavior on small machines by adding trace overhead, retaining extra metadata, or changing allocator timing. For that reason, profiling and benchmarking were separated into different lanes with different goals instead of being collapsed into a single measurement workflow.

The throughput-oriented lane relied on the training harness and comparison scripts. These runs recorded step time, tokens per second, completion status, observed OOM recovery, and peak VRAM. They identified which complete runtime configurations actually survived bounded end-to-end measurement. The instrumentation-oriented lane used PyTorch traces and the dedicated VRAM profiler. That profiler wrapped a short training run, recorded CUDA memory history, preserved a snapshot, and exposed a visualization path so allocator state and transient peaks could be inspected after the run. Throughput runs established what was winning or failing; profiling runs explained where the cost or fragility was accumulating.

Another important methodological choice was to keep the benchmark windows bounded but still structurally meaningful. A trivial smoke test can be too short to reveal the real cost of grouped generation. A very long run can be more faithful, but it is too expensive to repeat across many variants. The project therefore mixed three time scales. The shortest time scale was kernel or path-level validation, often enough to catch obvious correctness or dispatch failures. The middle time scale was the benchmark suite, typically long enough to smooth out startup noise and expose consistent step-time behavior. The longest time scale was the targeted diagnostic run, used when a short-window result looked promising but still needed to prove that it would survive a more realistic workload.

This methodology also prevented a narrower but very common tuning error: optimizing for the fastest incomplete metric. Throughput by itself was not sufficient. A candidate that looked faster because it truncated responses more aggressively, because it disabled a stabilizing mechanism, or because it required repeated OOM backoff to finish was not automatically a better operating point. Completion, comparability, and recoverability remained part of the acceptance standard alongside raw speed. Triton was not kept because it sounded lower-level, and a benchmark was not accepted because its headline number looked attractive. A change had to survive the full measurement chain.

Benchmark-driven promotion workflow

Candidate optimizations advance only after surviving progressively stricter measurement windows.

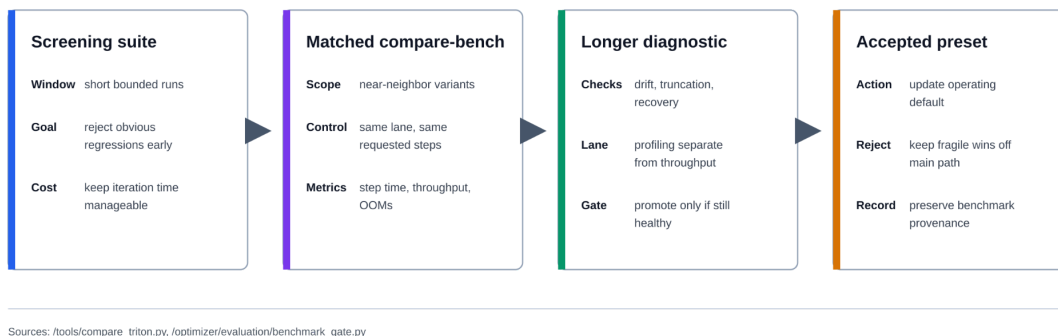


Figure 8.1: Benchmark-driven promotion workflow used for performance decisions.

Figure 8.1 condenses the measurement policy behind the chapter. Short suites screen candidates, matched comparisons test nearby variants, longer diagnostics check whether the gain survives, and only then does a setting become part of the accepted preset.

8.2 BOTTLENECK IDENTIFICATION IN THE NATIVE ENGINE

The dominant bottleneck was generation. This conclusion is consistent with the broader literature on long-response reasoning models, where decode-side cost and KV-cache residency can dominate runtime once grouped or repeated rollouts are required [16]. The local evidence made the same pattern concrete. Even after the backbone had been quantized and the trainable surface had been narrowed to LoRA adapters, generation continued to absorb a large share of wall-clock time. That is an important result because it shows what the earlier engineering chapters did and did not solve. Quantization and VRAM control made the runtime feasible. They did not make decode cost disappear.

The benchmark families exposed this bottleneck from several angles. In the long-response Triton matrix, the strongest end-to-end stack was not the one with the most aggressive generation specialization. It was the one that kept Triton in the non-generation kernels while leaving generation on the torch path. In the shorter generation suites, torch generation repeatedly matched or outperformed Triton generation for the tested grouped settings. In the response-length sweeps, the workload became sharply more expensive as the cap increased, even when the rest of the configuration was held fixed. These results all point in the same direction: decode-side work was

expensive enough to deserve special attention, but also sensitive enough that the most specialized path was not always the best full-system choice.

The second bottleneck pattern was phase asymmetry. The runtime did not behave like a uniform forward-backward loop. Generation stressed prompt expansion, cache growth, and grouped sampling. Old-log-probability computation stressed repeated policy scoring over grouped responses. The update phase stressed activations, loss construction, and adapter-side matrix operations. Because the phases were different, local optimizations often failed to transfer cleanly. A change that improved one phase could leave another untouched, or could shift the performance peak to a more fragile point in the step.

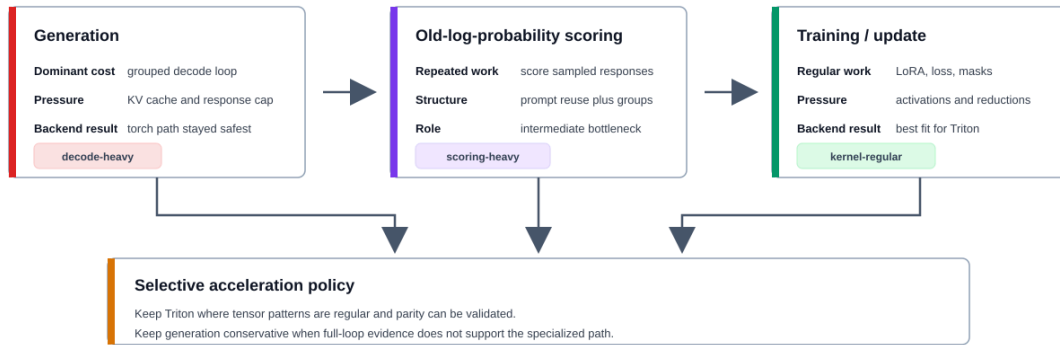
The old-log-probability path is a good example. It is not the headline bottleneck in the same way generation is, but it still matters because grouped GRPO-style optimization requires comparing the current policy against a previous policy state. That means the runtime must score sampled responses after generation has finished. Once prompt reuse and grouped expansion enter the picture, this path becomes more regular than raw generation but still large enough to influence total step time. It therefore sits in an intermediate zone: expensive enough to justify careful implementation and useful benchmarking, but not the phase that ultimately determined the global default.

The update phase introduced a third kind of cost. Here the dominant operations were not decode loops but repeated reductions and matrix operations over tensors whose shape was more stable. This phase was a better candidate for Triton specialization because the same computational patterns appeared repeatedly: fused LoRA execution, GRPO loss construction, entropy-based masking, and related reduction logic. In other words, the runtime contained two kinds of hotspots. One kind, especially generation, was expensive but also structurally difficult to stabilize. The other kind, especially update-side kernels, was both expensive and regular enough to justify more aggressive specialization.

Response length tied these bottlenecks together because it altered the cost of the generation phase more directly than it altered the more regular update-side kernels. As the cap increased, grouped decoding stayed active for longer, KV-cache residency grew, and the whole step became harder to interpret through isolated local speedups. Response length changed the relative weight of the main hotspot strongly enough that benchmark conclusions about generation backends and selective specialization could not be treated as independent from the chosen cap.

Bottleneck structure in the native engine

The slowest path was not the easiest one to specialize; generation dominated time but update-side work was more regular.



Sources: benchmark interpretation in Chapter 8 and response-cap evidence summarized in Chapter 6.

Figure 8.2: Bottleneck structure in the native engine.

Figure 8.2 separates the three relevant pressure points. Generation is the dominant wall-clock cost but is also the most sensitive backend path. Old-log-probability scoring is repeated enough to affect step time, while the update phase contains the more regular tensor work where selective Triton specialization is easier to justify.

The response length sweep does not only compare three policy settings. It reveals how quickly the bottleneck profile tilts toward decode-side cost once the rollout horizon is extended. A shorter cap can make generation look artificially manageable inside a bounded window, while a longer cap can make a promising kernel path appear worse because the runtime spends much more of the step in decode-heavy behavior. Response length is therefore part of the measurement context for the bottleneck itself, not just a separate memory-management choice.

The same reasoning later applied to broader dataset workloads, but the engineering conclusion is narrower. A dataset can be structurally supported by the runtime and still be a poor fit for the accepted response cap. That observation does not make dataset selection part of the Triton optimization story; it only reinforces why response length, prompt format, and split handling must be measured under the same operating profile used by the training loop.

Under those conditions, selective acceleration became easier to justify than a universal acceleration policy. The update-side kernels still benefited from structured specialization because their tensor patterns remained comparatively regular across the tested response regimes. Generation did not behave as cleanly. Its cost moved with the response cap, the grouped rollout structure, and the current backend path all at once. That interaction explains why the project could accept strong

non-generation Triton paths while still refusing to treat the most aggressive generation specialization as the automatic default.

8.3 TRITON AS A SELECTIVE ACCELERATION STRATEGY

8.3.1 Why Triton was introduced at all

Triton entered the project because the torch path was easy to trust but did not always expose enough control over hot kernels, while raw CUDA would have imposed too much implementation cost for a solo engineering project already operating under a strong wall-clock constraint. Triton occupies an intermediate layer. It provides direct control over kernel structure, launch geometry, and memory access patterns, but does so at a level that is still practical to iterate on in a research-engineering setting. That middle position was the main reason it was chosen. The project needed a tool that could express specialized kernels without turning every optimization into a full CUDA engineering effort.

This decision also matched the structure of the runtime. Several expensive paths involved repeated tensor operations with stable patterns: low-rank adapter projections, loss reductions, entropy computations, and decode-side cache handling. These are the kinds of operations where a specialized kernel is at least plausible. The project was not trying to use Triton as a general replacement for the rest of the stack. It was trying to use it where specialization could plausibly reduce repeated overhead in the most expensive parts of the loop.

8.3.2 Why Triton was not treated as a universal answer

The same properties that make Triton attractive also limit where it should be used. Specialized kernels depend on shape assumptions, dtype contracts, and hardware behavior more strongly than the plain torch path. Unsupported shapes, narrow edge cases, and dispatch rules are not incidental details. They are part of the real cost of adopting Triton. The local implementation makes this explicit. Triton support exists across several subsystems, but those paths retain fallbacks because the project could not afford to let specialized kernels become single points of failure.

This is especially important on a consumer GPU. A datacenter-oriented narrative can assume that the main challenge is extracting more parallel efficiency from a large, stable hardware budget. That was not the situation here. The tighter constraint was preserving a runtime that still behaved coherently when the hardware margin was small. A specialized kernel that saves time but narrows the operating envelope too much can still be a net loss. That possibility shaped the entire Triton strategy.

8.3.3 Why selective acceleration became the final policy

The benchmark evidence converged toward a selective policy instead of a universal one. Triton-off baselines were clearly slower in the relevant engineering suites, so abandoning Triton entirely would have meant discarding real gains. At the same time, forcing Triton generation on by default was not supported by the strongest long-response results, nor by the shorter grouped generation comparisons. The most credible reading of the evidence was therefore asymmetric. Triton was worth keeping, but its value was concentrated in the non-generation kernels and in a narrower set of structured fast paths.

This outcome is stricter than a blanket acceleration policy. Selective acceleration requires admitting that some expensive engineering work did not become a universal win. In this project, that selectivity made the final operating profile defensible. The accepted stack uses specialized kernels where benchmarking justified them, falls back where the plain path remained safer or faster, and preserves the ability to compare both modes cleanly.

8.4 CUSTOM TRITON KERNELS IN THE PROJECT

8.4.1 Kernel families and engineering intent

The Triton work in the project formed four main families. The first family covered LoRA execution. The second covered GRPO loss construction. The third covered entropy-mask creation. The fourth covered paged-KV generation support. These families differ in both ambition and stability. The update-side families operate on comparatively regular tensor shapes and remain close to the core of the optimization step. The generation-side family is structurally more complex because it has to preserve output semantics, cache correctness, and prompt expansion behavior while also competing against an already strong reference implementation.

What unifies these families is not that they are all equally successful, but that they were all chosen for a reason. Each one targeted a path where the runtime was either demonstrably expensive or structurally repetitive enough to justify specialization. That is an important distinction. The project did not start from the assumption that every operation needed a Triton variant. It started from a narrower question: where does the runtime repeat enough structured work that the cost of a custom kernel can plausibly be recovered? Figure 8.3 gives the overview that the following subsections unpack: each kernel family is shown with its target phase, validation gate, and final acceptance status.

Triton kernel families and acceptance status

The final stack keeps specialized kernels only where correctness gates and full-loop evidence justify them.



Figure 8.3: Triton kernel families and final acceptance status.

Figure 8.3 summarizes this selective result. LoRA and GRPO-loss specialization belong to the accepted default path, entropy masking remains a conditional accelerated lane inside the accepted masking regime, and paged-KV generation is implemented and validated but not promoted as the default generation backend.

8.4.2 Fused LoRA execution

The LoRA kernels targeted a very specific pattern. The runtime uses a frozen low-bit backbone and updates only adapter parameters, so adapter-side operations are repeated across every optimization step. This makes LoRA execution both frequent and structurally regular. The fused Triton path combines base projection behavior with adapter-side matrix operations and also supports the dequantization context associated with the 4-bit backbone. This fusion targets redundant overhead in the core execution path. The design prioritizes training performance, reducing the cost of the operations that the runtime repeats throughout the training phase.

This family was also a natural place to invest engineering effort because it sits in a friendlier zone of the design space than grouped generation does. Adapter execution is central to the training phase, but its dataflow is more predictable. That makes it easier to reason about input validation, block sizing, and fallback conditions. In practical terms, this means LoRA was one of the Triton areas where specialization was more likely to survive the full acceptance process.

The benchmark evidence supports that interpretation indirectly. The best accepted stack kept Triton enabled overall while disabling Triton generation, which means the surviving value had to come largely from the update-side and structured non-generation paths. Fused LoRA belongs to that surviving set of useful specialization work even if it is not the most visible subsystem from a benchmark-report perspective.

8.4.3 Fused GRPO loss

The GRPO loss was another strong specialization target because it combines repeated reductions with a stable computational contract. The fused loss path accepts policy logits, target ids, old log probabilities, grouped advantages, clipping parameters, and the relevant masks, then computes the loss through a specialized forward path while preserving a torch fallback for unsupported inputs. From an engineering perspective, this is a good Triton candidate because the operation repeats frequently, sits directly on the hottest part of the update phase, and already has a well-defined reference implementation for parity testing.

The value of this kernel family is not only raw speed. It also concentrates a complex reduction pattern into one explicit implementation boundary. That can improve reasoning about what the update phase is actually doing, provided the fallback path remains available and the parity tests remain trustworthy. This is one of the recurring advantages of selective specialization in the project. A specialized path was useful only when it improved performance without making the semantics harder to verify.

The tests around this subsystem reflect that requirement. The Triton loss must match the torch GRPO loss numerically under supported conditions, and it must also fall back cleanly for unsupported cases such as incompatible dtypes. This is exactly the sort of behavior that matters more on constrained hardware than on larger systems. A specialized loss kernel that crashes or silently diverges in narrow edge cases makes the whole training loop harder to trust.

8.4.4 Entropy-mask kernels

The entropy-mask path is smaller in scope than the loss or generation subsystems, but it is still a meaningful specialization target because it performs repeated token-level entropy work over structured tensors. The Triton implementation accelerates entropy histogramming, thresholding, and minimum-token enforcement. These operations are regular enough to benefit from custom kernels when the entropy-mask path is active.

This subsystem also exposes a limit in the performance record. Earlier project notes suggested that disabling entropy masking could improve throughput, but those notes came from a different stage of the runtime. They do not belong in the accepted evidence base for the final thesis narrative.

The final performance-suite reading is therefore narrower. Once the accepted benchmark record was fixed, the question was no longer if an earlier masking-off lane had looked faster in isolation. The relevant question was whether the masking-enabled path kept enough regular non-generation work to justify selective acceleration and a tested fallback path.

The conclusion is conditional. Entropy-mask acceleration can stay only as a limited part of the masking-enabled non-generation stack. Its value comes from being a localized, parity-tested specialization. What does not survive is the older claim that `entropy_mask_off` was a validated faster operating choice. The performance suite supports a smaller conclusion: if masking stays enabled in the accepted stack, this lane can be accelerated, but that does not reopen the masking decision at method level.

8.4.5 Paged-KV generation support

The paged-KV generation path was the most ambitious Triton effort in the project. It implements prompt-cache prefilling, grouped cache expansion, decode-time cache updates, and paged attention support for grouped decoding. Conceptually, this path targeted the most attractive possible prize. If grouped generation could reuse prompt state efficiently and decode through a well-specialized cache structure, then the dominant bottleneck of the runtime might become much cheaper.

That promise explains why this part of the system consumed a large amount of engineering time. It addressed the hardest hotspot with the most structurally ambitious solution. At the same time, it also inherited the hardest correctness burden. Generation is not like an internal training reduction where a small numerical mismatch can sometimes be caught through a scalar comparison. A generation bug can alter output text, break masking alignment, or make benchmark comparisons meaningless without always failing loudly. That risk forced the paged-KV work to meet a higher standard than many update-side kernels.

The tests for this subsystem reflect that burden. They check cache expansion semantics, parity against torch-based prefix-cache behavior on small Qwen-style models, mixed prompt-length decode consistency, and runtime policy decisions around tail micro-batches. This is a large validation surface for one subsystem, but it was necessary. The paged-KV path was trying to accelerate the most expensive phase of the system, and that phase was also the least tolerant of semantic drift.

The final benchmark outcome was mixed. The paged-KV and Triton-generation work produced technical knowledge and some bounded local gains, but the strongest end-to-end defaults did not retain Triton generation as the universal path. This is an honest boundary of the hardware regime: the hardest hotspot was also the one least willing to accept aggressive specialization as a universal default.

8.4.6 FALLBACKS, INPUT CONTRACTS, AND PRACTICAL MAINTAINABILITY

Integrating custom kernels required consistent fallback support across all families. This was a practical necessity for running Triton code within the training loop, not just a defensive safety measure. Specialized kernels depend on shape and dtype contracts, and those contracts are easier to violate in a research training engine than in a tightly fixed production inference service. Fallback behavior therefore became part of the performance design, not an afterthought attached after implementation.

This also affected maintainability. A project with only one specialized path per subsystem would have looked cleaner on paper, but it would have been far more brittle in practice. The coexistence of Triton and torch paths made benchmarking slower to interpret, yet it also made the results more trustworthy. The system could compare both modes, route unsupported cases safely, and keep the final operating profile narrower than the total amount of implemented specialization work. That is one of the main reasons the Triton effort can be defended as serious engineering rather than as low-level experimentation for its own sake.

Table 8.1 condenses the outcome of that selective acceptance process.

Kernel family or lane	Correctness evidence	Performance evidence	Final status
Fused LoRA path	Covered by the Triton correctness and performance suites plus the surviving Triton-on default lane	Value survives indirectly because the accepted stack keeps non-generation Triton enabled	Accepted as part of the default non-generation Triton path
Fused GRPO loss	Numerical parity and fallback tests against the torch reference	Treated as a repeated update-side win worth keeping	Accepted in the default Triton stack

Entropy-mask kernels	Mask parity tests under threshold and percentile modes	Kept only within the final performance-suite boundary: a conditional acceleration path inside the masking-enabled stack, without relying on older entropy_mask_off speed claims	Accepted conditionally within the masking-enabled non-generation stack
Paged-KV / Triton generation	Dedicated generation correctness tests for cache expansion, mixed prompt lengths, and runtime policy decisions	Bounded local gains existed, but the strongest full-loop defaults did not keep it as the universal path	Implemented and validated, but not kept as the default generation backend
Compare-bench default path	Structured compare-bench marks both compared commits as valid recovered-OOM runs	Newer commit slightly faster (27.72s vs 28.74s) but with more recovered OOM events (5 vs 1)	Supports viability of adaptive recovery, not a simplistic monotonic-speed claim

Table 8.1: Selective Triton acceptance outcomes.

8.5 CORRECTNESS VALIDATION BEFORE PERFORMANCE ACCEPTANCE

Performance evidence in this project was meaningful only because correctness validation came first. A specialized kernel that changes semantics is not an optimization. It is a different algorithm hiding behind a faster benchmark number. This was especially important in a project whose main contribution depends on explicit runtime control and benchmark-backed comparison. If the specialized path silently changes what the system computes, then the benchmark ceases to be a trustworthy authority.

The validation strategy therefore followed a short gate sequence:

1. Reference parity came first. The fused GRPO loss had to match the torch GRPO loss numerically under supported conditions, the entropy-mask path had to reproduce the reference mask behavior including threshold-based cases, and the paged-KV generation path had to preserve decoded outputs, token-level masks, and mixed prompt-length behavior in targeted tests.

2. Unsupported or edge conditions were checked next. Some tests were there not to prove speed, but to prove that the system rejected or rerouted unsafe cases correctly.
3. Generation received a stricter semantic check because the damage from a silent mismatch was larger. The paged-KV tests verify that grouped cache expansion clones or shares state in the intended way, that mixed prompt lengths are handled consistently, and that tail micro-batch logic can disable Triton when the grouped shape becomes too narrow to justify the specialized path. These tests protect semantics and show that the system does not assume Triton is always appropriate just because a Triton path exists.
4. Performance was accepted only after the correctness gates had passed. For the loss, entropy-mask, and paged-KV families, the specialized path had to show an improvement over the reference path under hardware-backed conditions. This requirement kept the project from collapsing into a catalogue of implemented kernels. The project did not keep a kernel because it compiled, because it passed a smoke test, or because it looked sophisticated. It kept a kernel only if it first behaved like the reference path and then survived actual performance measurement.

This standard also explains why some large engineering efforts did not become default behavior. The paged-KV and Triton-generation work was not rejected because it lacked technical depth. It was limited because the full acceptance chain was stricter than local implementation success. Not every specialized kernel won, but the project had a defensible mechanism for deciding which ones deserved to stay.

8.6 BENCHMARK-DRIVEN OPTIMIZER DESIGN

The optimizer was designed as a benchmark-governed control loop, not as a broad hyperparameter search script. Its working style was inspired in part by Karpathy's autoresearch project, which frames autonomous experimentation around a deliberately small single-GPU setup, a fixed training budget, a comparable metric, and a repeated keep-or-discard loop [20]. That pattern was relevant here less as a literal implementation template than as a methodological cue. The useful lesson was to keep the search loop narrow enough that each attempt remains attributable, measurable, and easy to reject when the evidence is weak.

The local optimizer therefore adopts the spirit of autoresearch but not its exact operating model. This project does not hand one freely editable training file to an agent and let it optimize against a single validation scalar. Instead, it keeps a tracked frontier, proposes one bounded hypothesis at a time, benchmarks the candidate against the current reference, and promotes only when the candidate survives the hard gates for throughput, completion, correctness, and recoverability. The adaptation matters because the local problem is different: the target is conservative refinement of a fragile GRPO runtime on one 8 GB consumer GPU, not open-ended overnight improvement of a smaller training script.

Agent-guided search also entered late. By the time the optimizer loop was formalized, the main engineering decisions had already been made manually: the native PyTorch structure, the LoRA-only training surface, the phase-explicit recovery path, the benchmark harness, and the broad Triton-versus-torch boundaries. The remaining uncertainty no longer centered on first-order design choices. It centered on smaller runtime interactions whose effect was real but difficult to rank from intuition alone.

That narrow residual search space is where the autoresearch inspiration fit well. The agent was not used to set the main systems direction. It was introduced after the obvious manual gains had already been taken, and it was limited to a constrained surface of backend toggles, cache and cleanup policies, lane-specific adjustments, and other combinations that were easy to overlook but still measurable. Benchmark evidence still decided whether any apparent improvement was real enough to survive.

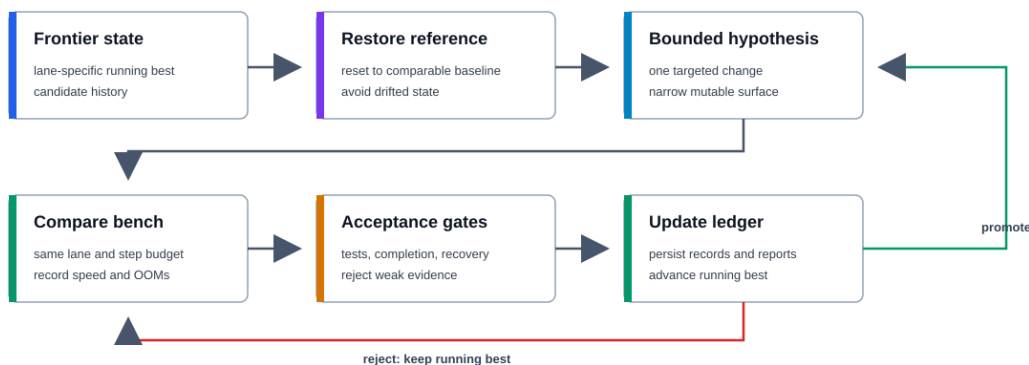
Two design decisions define this loop. The first is strict sequentiality. The optimizer evaluates one hypothesis at a time and avoids bundling many interacting changes into a single attempt. This makes attribution cleaner. If a candidate improves, the likely cause is easier to identify. If it regresses, rollback is immediate and understandable. The second is lane separation. Triton-on and Triton-off modes maintain separate frontiers and running-best states. This prevents one mode from contaminating the other and avoids a common benchmarking error in which changes are compared across substantially different backend assumptions.

The search surface was also bounded deliberately. Some parameters were locked rather than kept under permanent reconsideration. The stable rollout grouping remained 4, and entropy masking remained enabled even though earlier local notes had encouraged reopening that choice. This narrowing of the mutable surface is one of the strongest parts of the optimizer design. It shows that the project was not using automation to evade engineering judgment. The optimizer was allowed to search where the uncertainty remained real. It was not allowed to reopen decisions that had already been justified by the broader operating logic.

The acceptance mechanism reinforced that discipline. The reducer applies hard gates before promoting a candidate: the benchmark report must be sequential and complete, the relevant tests must pass, and recovered OOM behavior is treated as a constrained condition rather than ignored. The candidate and frontier states are then written into a ledger, the running best is restored or advanced according to the result, and reports are regenerated from those durable records. The optimizer acts as a persistent control plane, not a temporary experiment launcher. This structure is necessary because small benchmark windows often produce local performance gains that are misleading. Documenting why these gains were rejected ensures the optimization history remains justifiable.

Benchmark-governed optimizer loop

Each candidate must be attributable, comparable, and gate-checked before it can replace the running best.



Sources: /optimizer/reducer.py, /optimizer/records.py, /optimizer/evaluation/benchmark_gate.py

Figure 8.4: Benchmark-governed optimizer loop.

Figure 8.4 summarizes the optimizer as a conservative control loop: each candidate starts from a comparable reference, passes through benchmark and acceptance gates, and is either promoted into the ledger or rejected while the running best is preserved.

This design is especially relevant for the Triton story. Triton creates many tempting avenues for local improvement: changing a block size, enabling a generation fast path, preferring one specialized kernel over another, or forcing a backend where the auto-policy would have routed elsewhere. Some of these interventions can produce real gains in very short windows. The optimizer acted as a gatekeeper to prevent local performance gains from becoming permanent defaults too quickly. By requiring specific validation for each change, the system balanced aggressive hardware optimization with a conservative selection process.

8.7 OPTIMIZER WORKFLOW IN PRACTICE

In practical use, the optimizer begins from the current frontier and the running best for the active lane. If the working state has drifted away from the running best, the loop restores the reference before forming a new bounded hypothesis. This avoids comparing a candidate against an accidental or partially degraded baseline. The candidate is then benchmarked against the reference through the comparison harness, typically over a fixed short window, with VRAM sampled externally and outputs written in machine-readable form.

The workflow also guards against order effects. A small consumer-GPU benchmark can be influenced by warmed caches, initialization asymmetry, or the fact that one run paid an early setup cost that the next one did not. The optimizer therefore allows rerunning the same pair in reversed order when a result looks suspicious. This rigor is required to manage the noise inherent in short benchmarks. It is the necessary cost of using noisy data without assuming that the results are cleaner than the actual environment allows.

This workflow turned benchmark results into the accepted operating preset in a fairly direct way. Triton remained enabled overall because the Triton-off baselines were clearly slower in the relevant engineering suites. Triton generation was disabled by default because the strongest long-response stack came from keeping Triton everywhere except generation, and the shorter generation suites repeatedly favored torch. `group_size = 4` remained the stable grouped setting because larger groups raised cost sharply without showing a better practical trade-off.

The limits of the workflow are also important. The optimizer works well when the mutable surface is narrow and when short-window results are treated as screening evidence rather than as final truth. It cannot fully solve interactions that appear only after much longer runs, and it cannot replace longer diagnostics when response-length pressure or recoverability may change over time. For that reason, the project used the optimizer to narrow the search space and to reject weak candidates early, but not to claim that a short-window winner had automatically become the best possible end-to-end profile.

This limit strengthens the design. A benchmark-governed optimizer is useful when it has a specific purpose. In this case, the system identified which changes should be adopted as local operating defaults. Such a narrow scope is required to maintain engineering discipline when hardware is restricted.

8.8 FINAL PERFORMANCE FINDINGS

Three main findings survived the full measurement process:

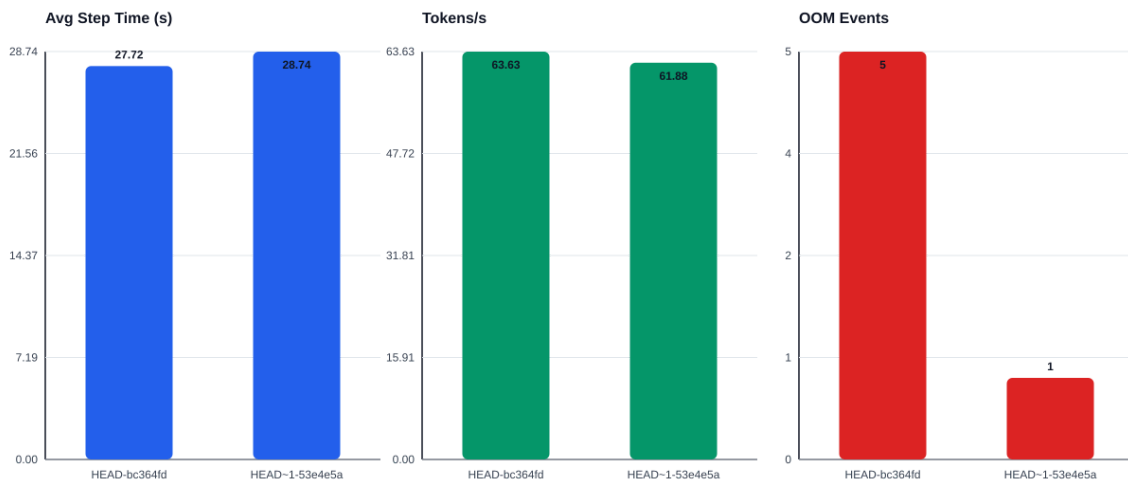
1. Triton was genuinely useful, but primarily through the non-generation kernels. In the imported 20-step engineering suite, the Triton-on baseline completed at roughly half the step time of the Triton-off baseline while using essentially the same card-level peak VRAM. This means Triton cannot be dismissed as cosmetic low-level work. It delivered real speed gains in the relevant operating regime.
2. The strongest Triton policy was selective rather than universal. In the long-response Triton matrix, the best full-stack result came from keeping Triton active except for generation. In the shorter grouped generation suites, torch generation repeatedly matched or beat the Triton generation path. This made the final decision relatively clear. The default should keep the

Triton paths that repeatedly paid off and disable the one that did not survive full-loop comparison strongly enough to justify being universal.

3. Some local optimization stories still had to be rejected as method-level defaults. Earlier entropy-masking impressions did not survive once the performance-suite boundary and the later optimized regime were taken seriously. Some tuned 1024 response-length runs were not catastrophic, yet the matched response-length evidence summarized in Figure 6.1 still showed that moving from 768 to 1024 created a much steeper fragility cliff. A slightly faster commit in the compare bench could also show more recovered OOM events than its slower predecessor. These cases explain why the project needed benchmark-gated selection instead of intuition-led tuning.

Compare-bench Triton-on Commit Snapshot

The imported compare bench preserved two nearby Triton-on commits and shows the cost of reading small speed gains together with recovery behavior.



Sources: /tools/compare_triton.py, compare_bench_20260415-204526.json

Figure 8.5: Compare-bench snapshot for two nearby Triton-on commits.

Figure 8.5 is useful because it prevents an overly simple reading of the compare bench. The newer commit is slightly faster, but it also recovers from more OOM events, which is exactly why the chapter treats benchmark promotion as a multi-signal decision rather than a single-number race.

This is where the large amount of Triton work can be stated most honestly. The project invested heavily in Triton because generation and update-side kernels were obvious performance targets, and because a native local engine would have been difficult to defend as a systems contribution without attempting serious specialization work. That investment did not produce a simplistic ending where

every Triton subsystem became part of the default stack. It produced something more useful: a measured understanding of where specialization paid off, where it remained optional, and where the torch path continued to dominate once the whole loop was considered.

On a larger machine with more abundant memory headroom and less pressure to preserve every margin of stability, a more aggressive specialization policy might well be sustainable. On a single 8 GB consumer GPU, the more important achievement was to determine which optimizations improved the actual operating profile rather than merely improving local measurements. The accepted configuration reflects that narrower standard. Triton remains part of the final system because it proved its value. It does not dominate the final system because the hardware regime did not reward specialization equally across all phases.

The final performance picture is coherent. The runtime became faster through selective specialization, not through blanket backend replacement. Kernel work was worthwhile because some of it survived, and because the rest still mapped the boundary between promising low-level ideas and trustworthy defaults. The optimizer loop was worthwhile because it prevented those two categories from being confused.

9. OPERATIONAL RESULTS, INCIDENTS, AND ENGINEERING LESSONS

9.1 FINAL OPERATING CONFIGURATION

By the end of development, the central question was no longer whether GRPO-style post-training could be made to run at all on a single consumer GPU. The more relevant question was what final operating profile could be defended as usable day to day. The accepted preset is the operational synthesis of objectives O1 through O4: a native local engine, selective efficiency work, robust memory control, and direct runtime governance. It keeps Triton enabled on the non-generation kernels, leaves generation on the torch path, uses grouped rollouts of four completions, accumulates gradients over sixteen micro-steps, caps responses at 768 tokens, and trains only LoRA adapters at rank 16 on top of a quantized 1.5B backbone. This preset allowed the system to complete bounded runs without immediate collapse, preserve a meaningful response budget, and keep recovery as an exception rather than as the dominant pattern of execution.

The final preset also changed what "good performance" meant in practice. Earlier in development, a lower step time could still hide an undesirable response regime, a narrower memory margin, or a backend choice whose behavior degraded once the workload left a short benchmark window. The final configuration treats these trade-offs more cautiously. It keeps the aggressive parts that proved useful, especially on structured update-side kernels, but it avoids forcing the generation path into a mode that was harder to defend under sustained use. The result is a runtime built to be run repeatedly, not just to produce attractive benchmark lines once.

The accepted preset makes the engine legible as a working system rather than as a collection of isolated optimizations. The quantized backbone keeps persistent residency under control, the adapter-only training surface bounds gradient and optimizer state, the selected response cap leaves enough room for substantive completions, and the final backend mix reduces the most visible fragility that appeared when generation was pushed more aggressively. In operational terms, the preset is the point at which the project stops looking like a prototype that happens to finish and starts looking like a narrow but coherent training runtime.

Table 9.1 summarizes the accepted profile and the immediate operational role of each decision.

Control surface	Accepted setting	Immediate operational purpose
Backbone storage	4-bit NF4 with double quantization	Keep the base model resident within the single-GPU budget

Trainable surface	LoRA only, rank 16	Bound gradients and optimizer state to a compact path
Rollout grouping	group_size = 4	Preserve grouped relative rewards without the cost explosion of larger groups
Gradient accumulation	16	Maintain an effective batch size that is still feasible under the available VRAM
Response cap	768 tokens	Avoid the strong truncation pressure of 512 while staying away from the 1024 fragility cliff
Triton policy	enabled except generation	Keep the structured kernel gains without inheriting the weaker full-loop generation behavior
LoRA adapter quantization	none	Preserve a valid, benchmark-backed adapter path after low-bit adapter variants proved weaker or invalid

Table 9.1: Accepted operating profile and operational purpose.

This operating profile also clarifies what the final system is not trying to solve. It is not a universal preset for arbitrary consumer GPUs, for arbitrary reasoning tasks, or for broader reinforcement-learning pipelines that can afford heavier regularization. It is a narrow operating point chosen for one hardware tier and one engineering objective: sustained local post-training near the memory boundary without turning the run into a sequence of avoidable failures. Specialization provides the context needed to interpret the technical results. By establishing a specific environment, the design keeps operational evidence identifiable and useful for further optimization.

The final preset also leaves several important limits in place. It does not remove the generation bottleneck. It does not create abundant safety margin. It does not make long-response experimentation cheap. It does not support a reference-model-heavy regularization path under the same budget. It does give the project a realistic default under which incidents can be analyzed as properties of one accepted system rather than as artifacts of a constantly shifting configuration space.

9.2 STABILITY, RECOVERY, AND THE PRACTICAL OPERATING ENVELOPE

The main operational result is not a single throughput number. It is that the engine reached a mode in which the complete training loop could be executed repeatedly on the target hardware without treating OOM as a routine end state. That distinction matters. On constrained hardware, many prototype systems can be made to work once, or can be made to finish under a short bounded run if the workload is chosen carefully. Far fewer remain usable once grouped generation, scoring, backward passes, and recovery behavior all interact under a more realistic window. The accepted preset belongs to that smaller set.

The clearest evidence for that claim comes from the preserved 40-step diagnostic with the accepted backend policy and response cap. That run completed 40/40 steps with an average step time of 27.59 seconds, a median of 25.45 seconds, an average throughput of 63.78 tokens per second, and no OOM backoff events. Generation and training micro-batch values also stayed at 4 throughout the run. Success at this stage meant maintaining stability near the hardware limit. The runtime operated close to the VRAM boundary without triggering the adaptive failures seen in earlier, more constrained attempts. This reliability mattered more than gaining extra memory headroom.

The response behavior in that accepted diagnostic also matters. The average realized response length was 440.49 tokens, with truncation visible but not dominant: 9/40 steps contained any truncation, 7/40 had majority truncation, and 3/40 reached full truncation. This configuration represents a trade-off between output length and memory availability. The system generates long responses while keeping truncation within manageable bounds. Although the hardware imposes real limits on each step, the results show that the model can still produce the completions required for reasoning tasks.

The alternative 512 diagnostic helps define the lower edge of that envelope. It also completed all 40 steps, but that headline success still concealed a weaker operating regime. The average step time was 33.41 seconds, the median was 26.26 seconds, and the average response length was 421.67 tokens. The run recorded 19/40 steps with some truncation, 17/40 with majority truncation, 7/40 with full truncation, and one recovered OOM backoff event. Under the later replay-safe recovery design, that backoff no longer means that the run itself became semantically unreliable. The more important signal is the drift within the schedule: the first twenty steps averaged 22.39 seconds, while the last twenty rose to 44.43. This is worse than a slightly slower but steadier configuration because the latter half of the run spends too much time in a truncation-heavy, degraded regime that no longer represents the intended workload cleanly.

The upper edge of the envelope is defined by the 1024 response-length family. That regime is not best described as impossible, because some tuned variants remained executable. It is better described as operationally fragile. In the matched response-length sweep, moving the cap from 768 to 1024 increased average step time from 51.70 to 156.41 seconds and dropped throughput from 36.13 to 15.33 tokens per second. The preserved train log for that regime recorded two explicit early

OOM backoffs, reducing both generation and training micro-batches until the run moved into a far more conservative schedule. This matters more than raw feasibility. A system can still execute under such conditions, but it no longer operates on a healthy margin.

These three response-length regimes define a useful practical taxonomy of operation:

Regime	Typical operational pattern	Practical interpretation
Stable	no OOM backoff, fixed micro-batches, bounded truncation, moderate step-time drift	acceptable default operating condition
Degraded	step completion still holds, but truncation or step-time inflation becomes recurrent	survivable, but not a strong default
Interrupted or fragile	adaptive backoff triggers early, micro-batches shrink, throughput collapses	feasible only as an exploratory or narrowly tuned case

Table 9.2: Practical operating regimes near the memory boundary.

The accepted preset belongs to the stable regime, while the 512 long diagnostic sits at the boundary between stable completion and degraded operation. It finishes the bounded window, but only after drift and truncation become recurrent enough to weaken its value as a default. The fixed 1024 sweep belongs to the fragile regime. This framing is more informative than a binary "works versus fails" distinction because it matches how the engine actually behaves near the memory boundary. Most operationally relevant questions in this project were not about absolute failure. They were about how much adaptation was required before the run stopped resembling the workload it was supposed to represent.

The compare-bench snapshot between two nearby commits, the new OOM management and the previous commit adds a second kind of stability evidence. Both runs completed all 50 requested steps under the accepted Triton-on default path and both were marked as `oom_recovered` rather than as outright failures. The newer run averaged 27.72 seconds per step and 63.63 tokens per second, while the previous one averaged 28.74 seconds and 61.88 tokens per second. The newer run also recorded more recovery events, 5 instead of 1, but that increase should not be read as a simple regression. Under the later policy, OOM handling became more explicit and more phase-local, so pressure that would previously have surfaced less clearly could now be detected and contained in different parts of the step. The broader intervention did not destroy throughput or force the runtime into a

qualitatively weaker schedule. The accepted default path remained viable across nearby revisions even when the recovery logic had to intervene repeatedly in different areas of the loop.

From an operator's point of view, this continuity changes how the engine is interpreted. In the earlier phases of development, an OOM signal often meant that the entire step had to be treated as suspect. In the later system, an OOM signal is still undesirable, but it is no longer automatically catastrophic. The runtime can preserve prepared state, back off only the failing phase, and continue under a narrower schedule while keeping the step semantically coherent.

That said, the accepted preset does not eliminate the cost of narrow-margin execution. It keeps the engine usable under pressure, but it does not make that pressure disappear. The imported `ing_inf` suite still reports card-level VRAM peaks at roughly 7.70 GB across many bounded runs, which is effectively the whole available device budget for the process. The accepted profile should therefore be presented as robust within a constrained envelope, not as evidence that the memory limit ceased to matter.

Seen from routine operation rather than from benchmarking, the accepted system presents a recognizable pattern. Most successful steps spend much of their visible runtime in a moderate VRAM band around three to four gigabytes, then surge higher at the more expensive points of generation or update before dropping back after cleanup. The engineering result is not the visible average alone. It is that the system can return from those surges without corrupting the step, collapsing the schedule, or forcing full reruns of already successful work. That is the practical behavior that distinguishes a viable local runtime from a merely clever one.

The accepted profile should therefore be understood as a controlled narrow-margin regime. It is not comfortable. It is not broadly portable. It is not free of adaptation logic. But it is stable enough that the remaining incidents can be treated as informative engineering events instead of as symptoms of an unfinished prototype.

9.3 MAIN INCIDENTS AND WHAT THEY REVEALED

9.3.1 Feasibility before optimization

The first major incident was not a single crash or bug. It was the broad realization that feasibility itself was a harder problem than optimization. Before the project could compare kernels, response caps, or backend policies, it had to reach a state in which the full loop could execute at all on the target hardware. This incident shaped the rest of the work more deeply than any later benchmark result. It forced the system away from a framework-first design and toward a phase-explicit runtime in which generation, scoring, update, cleanup, and recovery could all be reasoned about separately.

What this incident revealed is that constrained local post-training fails early when the execution model is still too abstract. Hidden buffers, opaque phase transitions, and generic training calls are tolerable on roomier hardware because the runtime has enough reserve to absorb some avoidable cost. Under an 8 GB budget, that same hidden cost is often the difference between a fragile prototype and a runnable engine. The operational lesson is that feasibility work has to reduce uncertainty before it reduces latency.

This early phase also clarified why some engineering decisions remained worthwhile even though they increased local implementation effort. Writing the runtime at a lower level and implementing the LoRA path manually created a larger surface for testing and debugging, but it also removed categories of ambiguity that would have been much harder to diagnose under repeated OOM pressure. The final system still carries the cost of those choices, but the operational record suggests that this was the right trade for the hardware regime involved.

9.3.2 Making sampled responses usable for training

A second incident emerged once the engine could run its basic loop: producing responses was not enough if those responses were not semantically usable by the verifier, the masking logic, and the grouped objective. This was operationally important because a constrained reinforcement-learning system can appear to run correctly while actually training on malformed or misaligned supervision. The risk is especially high when text generation, answer extraction, truncation handling, and response-only masking all interact in one pipeline.

The main resolution here was not a single optimization but a tightening of contracts across the generation and pre-computation stages. The verifier behavior had to become stricter, the distinction between prompt tokens and response tokens had to remain explicit, and the loss path had to avoid treating incomplete outputs as valid supervision by accident. Response-length trade-offs, truncation rates, and operator-visible degradation only become interpretable once the underlying supervision path is trustworthy.

What this incident revealed is that the operational quality of a local training engine depends as much on semantic hygiene as on memory hygiene. A system that fits in VRAM but quietly mis-scores or mis-masks its rollouts is not actually more usable than a system that crashes more honestly. The project avoided that failure mode by making the verifier path and the response contracts part of the tested runtime.

9.3.3 Repeated OOM pressure as a design condition

Persistent OOM behavior was the defining operational incident of the project. It appeared repeatedly and in different forms: some workloads exceeded the safe budget immediately, some failed later in training after a transient rise in sequence cost, and some completed only by shrinking the schedule to

a less representative operating point. What makes this incident central is that it was not solved by one optimization pass or one better hyperparameter choice. It became manageable only after several runtime policies started to work together.

The 1024 response-length evidence makes this especially concrete. In the matched sweep, the run remained executable, but only after explicit early backoffs cut generation and training micro-batches down to a much smaller schedule. The older bounded runs with non-optimal backend or accumulation choices show the same pattern from another direction. Some triton_off variants were drastically slower while still living close enough to the memory boundary to emit repeated high-usage warnings and elevated OOM-recovery counts. These cases show that low throughput and low stability are not opposite failure modes. Near the hardware limit they often reinforce each other.

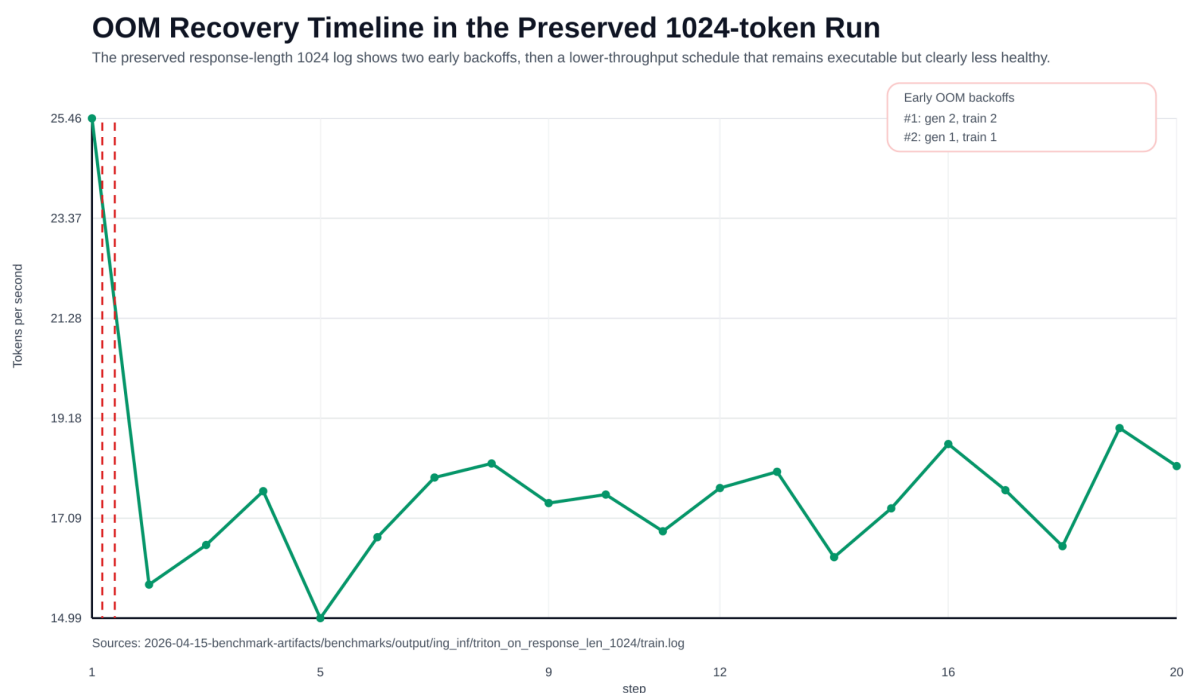


Figure 9.1: OOM-recovery timeline from the preserved 1024 response-length run.

Figure 9.1 uses one real preserved log to show what operational fragility looks like in practice: two early backoffs force the schedule downward, after which the run remains executable but at a clearly weaker throughput level.

The later recovery design changed the practical meaning of these incidents. Once failures became phase-local and retries became replay-safe, an OOM no longer implied that the entire step had to be discarded or semantically distrusted. This is one of the most important improvements in the final system. The engine still acknowledges that OOM is part of the environment, but it now contains that

condition with more discipline. Out-of-memory events remained a reality of the runtime, yet they ceased to define the overall structure of the run. By containing these failures as local incidents, the system kept a memory spike from compromising the integrity of the entire training step.

This incident also exposed a broader lesson about constrained systems work: memory margins are relational, not absolute. The device can report several gigabytes of free VRAM after one step and still remain vulnerable to a later failure because the next phase changes the tensor mix, the response realization changes decode cost, or allocator state shifts in an unfavorable way. The practical operating envelope therefore has to be described in terms of stable, degraded, and fragile regimes rather than in terms of one fixed notion of "enough free memory."

9.3.4 High step times and generation dominance

Even after the engine became stable enough to run, step time remained an operational incident in its own right. The raw ability to complete training steps was not sufficient if the wall-clock budget prevented meaningful iteration, especially in a project where benchmark-backed selection and repeated diagnostics were part of the method. The most persistent pattern in this area was generation dominance. Grouped rollout generation continued to absorb a large fraction of runtime even after the backbone was quantized and the training surface was reduced to adapters.

What this incident revealed is that local reinforcement-learning systems can shift their main bottleneck without becoming operationally comfortable. Quantization and adapter-based fine-tuning solved the residency problem well enough to make the engine feasible, but they did not solve the decode-side cost of long responses under grouped sampling. Improving performance in a specific kernel family does not always shorten the total step time. Since the bottleneck often remains in the generation phase, local gains can inadvertently push the runtime toward memory limits where the rollout becomes unstable.

The 768 token cap and the selective Triton policy address the generation bottlenecks identified in the runtime. This approach manages rollout costs to avoid the diminishing returns of applying custom kernels to operations that do not provide a clear speedup. By balancing decode latency against the required completion length, the system maintains stability while reserving specialized Triton paths for the most regular update-side tasks.

9.3.5 Broader dataset support and workload fit

The later move to broader math datasets exposed a different kind of incident. The engine could load and split Open-RS and DAPO-MATH-17K, and the training loop could start normally under the same 768-token cap. The problem was not structural support. The problem was that many sampled groups reached the response limit together. In those cases the run consumed generation time but produced

little useful update signal, because the grouped objective had no meaningful variation to compare inside the prompt group.

The incident separated three levels of feasibility that are easy to confuse:

1. The dataset can be supported by the code. In this case, the engine could load broader normalized math datasets, preserve the expected split structure, and start the training path without changing the trainer interface.
2. The run can fit inside memory. This condition shows that the accepted 768-token envelope and the existing recovery policies are sufficient to begin execution, but it does not prove that the resulting samples are useful.
3. The workload can produce usable training signal. This stronger condition fails when many sampled groups reach the response limit together, because the run spends generation time without producing enough reward variation inside the prompt group.

The first two conditions are necessary, but they are not enough for a practical local reinforcement-learning workflow.

The lesson is that generalized dataset support should not be confused with a guarantee of useful training. The architecture can expose a broader normalized math-dataset path, preserve train, validation, and test splits, and reuse the same SENT and trainer interfaces. It still needs the experimental side of the project to decide whether a candidate dataset produces usable rollouts and reward variation under the accepted response budget.

What this incident revealed is that local training efficiency depends on the compatibility between data, sampling policy, and hardware budget. On larger hardware, the same datasets could be handled by increasing the response cap. On the target GPU, that was not a stable option. The engineering contribution is therefore the ability to run and observe those workloads through one controlled pipeline, not a claim that every supported dataset becomes a good training target.

9.3.6 Profiling and Triton uncertainty

Two related incidents shaped the late engineering phase: profiling on constrained hardware could itself become intrusive, and Triton integration repeatedly produced ambiguity about whether a local win was a real end-to-end improvement. These issues directly affected which conclusions could be defended about the final operating stack.

Profiling had to be bounded because deep instrumentation changes the runtime. A short constrained run with memory tracing, snapshots, or heavyweight traces is useful for locating hotspots and fragmentation patterns, but it is not a neutral mirror of the production path. The practical resolution was to separate throughput-oriented benchmark runs from instrumentation-oriented diagnostic runs.

That distinction is now part of the operating method of the system. Measurement became another phase-sensitive activity, subject to the same caution as the training loop itself.

Triton uncertainty produced a different but related lesson. Implementing specialized kernels was not enough to justify them. Some paths remained genuinely useful and now belong to the accepted runtime. Others were only weak wins, edge-case wins, or wins that disappeared once the whole loop was measured under a realistic response regime. The project therefore learned to value selective acceleration over universal acceleration. This is a conservative result, but it is also the more robust one.

9.4 LESSONS FOR LOCAL NATIVE RL TRAINING

Several engineering lessons emerge from the final operating record:

1. Explicit control only matters when it is paired with enough discipline to interpret what that control is showing. Writing a native runtime is not an achievement by itself. It becomes worthwhile when the exposed phases, buffers, and policies can be validated, benchmarked, and revised without turning the system into a moving target. In this project, that discipline came from the interaction between tests, bounded benchmarks, longer diagnostics, and a refusal to promote local optimizations before they survived end-to-end comparison.
2. Constrained local reinforcement learning benefits more from layered mitigation than from dramatic one-shot fixes. No single intervention made the engine operational on its own. The decisive improvement came from combining bounded residency, explicit cleanup, adaptive recovery, and narrower training surfaces into one coherent runtime. The system works because these mechanisms form a stack, not because any one of them can be treated as a universal answer.
3. Response length is an operational systems parameter as much as an algorithmic one. In reasoning-oriented reinforcement learning, longer completions are tempting because they appear to preserve more room for the model to think. The local evidence shows that this intuition is incomplete. A longer cap can degrade throughput sharply, narrow the memory margin, and force enough schedule adaptation that the resulting training regime ceases to represent the intended workload cleanly. Conversely, a shorter cap can look efficient while quietly converting the run into a truncation-heavy regime with weaker practical value. The accepted 768 setting is important because it demonstrates that the best local operating point is often the one that avoids both extremes.
4. Dataset choice remains part of the operating envelope, not only part of the algorithmic setup. The same response-length problem appeared when the project moved to harder datasets. These datasets were not just harder to solve; they also produced longer rollouts. With a fixed response cap, the system had to distinguish between structural support and practical workload fit. Dataset choice therefore remained part of the experimental design,

while the system kept the execution contract general enough to load normalized math splits without rewriting the trainer.

5. Selective acceleration is often better suited to this hardware tier than backend replacement. The final system does not reject specialization; it depends on it in several important update-side paths. What it rejects is the assumption that every expensive phase should be pushed through the same low-level policy. Structured kernels and generation loops do not behave the same way, and the final runtime benefits from respecting that difference. This is likely to remain true for similar consumer-GPU systems where small changes in backend behavior can have disproportionate effects near the memory boundary.
6. The gap between "the run finished" and "the run represents a defensible operating default" is large. Short-window benchmarks are useful, but they can overvalue settings that later degrade under truncation pressure, allocator drift, or repeated recovery. Longer diagnostics are more expensive, but they reveal what the bounded suites cannot: whether a promising candidate remains usable after the easier early steps are gone. The final operating preset earned its place because it held up under both kinds of evidence.

The engine should not be understood only as a compact local alternative to larger training stacks. Its more specific contribution is to show that the engineering problem of local reinforcement-learning training is not exhausted by making the model fit in memory. The harder problem begins after that point: keeping the runtime inspectable, recoverable, and benchmarkable when it is already operating close to the limit. The final system does not remove that difficulty. It contains it well enough that the difficulty itself becomes analyzable instead of merely disruptive.

10. ETHICS

10.1 ETHICAL ISSUES IN LOCAL POST-TRAINING

The ethical question in this project is deliberately narrow: under what conditions is it responsible to make post-training methods cheaper, more accessible, and easier to reproduce on personally controlled hardware? The answer depends on two connected dimensions. One is social impact: access, misuse, sustainability, privacy, and inclusion. The other is professional engineering ethics: reliability, observability, truthful reporting, and responsibility for failure modes under constrained execution.

- **Environmental cost.** Training and adapting language models consumes electricity, water, hardware life, and indirect resources. Recent measurements also show that a large part of the footprint can come from development work around the final run: debugging, exploratory reruns, parameter search, and repeated validation [21]. A local workflow on already owned hardware can reduce dependence on rented infrastructure, but it does not make computation free. It is more honest to frame the project as compute-conscious rather than inherently green. The project limits waste by keeping experiments bounded and by rejecting the assumption that cheap local execution justifies unlimited reruns or premature hardware replacement.
- **Access and misuse.** A native single-workstation engine lowers the barrier for students, smaller institutions, and independent researchers who cannot rely on large cloud budgets. That supports educational access and relates to Sustainable Development Goals (SDGs) 4 and 10 [22]. The same accessibility also makes advanced optimization techniques easier to reproduce outside stronger institutional controls. The project therefore cannot treat openness as automatically ethical. Openness is defensible here only because the system is presented with a narrow scope, explicit limitations, and evidence tied to one constrained hardware regime.
- **Reliability.** On an 8 GB GPU, efficiency work happens close to the OOM boundary. Aggressive acceleration can improve selected measurements while making the complete runtime harder to inspect or recover. That is an ethical concern, not only a technical one: an opaque fast path can waste compute, hide failure causes, and support claims that the evidence does not justify. For this reason, the project treats observability, fallback behavior, and recovery as part of the engineering obligation.
- **Evaluation truthfulness.** Verifiable mathematical tasks allow automatic checking of final numerical answers, so binary reward is a practical and defensible signal for this project. GSM8K is the clearest baseline case, while later datasets keep the same requirement that the target answer can be checked deterministically. This signal is still a proxy. Literature on reasoning-oriented reinforcement learning warns that proxy rewards can encourage shortcut

behavior, reward hacking, bias amplification, and over-optimization toward what the verifier can detect [23] [24]. The ethical position is therefore to use verifier-centered rewards as narrow experimental evidence, not as proof of general reasoning quality, fairness, or safety.

- **Privacy and data governance.** Local execution reduces routine exposure of prompts, logs, checkpoints, and intermediate traces to third-party providers. That is a real gain in data autonomy. It is not a privacy guarantee. The responsibility shifts to the local operator: dataset provenance, access control, storage hygiene, model licensing, and checkpoint handling still matter. The project does not involve human-subject experimentation, clinical intervention, or collection of personally identifying participant data, so its main obligations are truthful reporting, responsible use of public datasets and checkpoints, and careful local artifact governance.
- **Gender perspective.** This dimension is relevant, but it must be kept proportional to the project. The system does not classify people or make user-facing decisions, so gender is not the central risk axis. Even so, access to hardware, time, and advanced experimentation is not socially neutral. Reducing the material threshold for technical experimentation can support inclusion, including for groups underrepresented in computing, but it should be described as a modest contribution rather than a complete response to structural inequality.

10.2 ALTERNATIVES, DECISIONS, AND JUSTIFICATION

The project considered several practical alternatives that would have made parts of the work easier to present, but each would also have weakened its ethical basis: expanded cloud use, isolated speed-first choices, heavier reference or framework paths, aggressive default acceleration, and broad privacy or sustainability claims. These rejected options are not abstract preferences; they have direct consequences for resource use, access, reliability, evidence, privacy, and architectural responsibility.

The adopted decision was conservative: keep the project local and single-device, but pair that choice with bounded claims, benchmark discipline, and explicit responsibility for failures. The project did not aim to run the most ambitious configuration that the RTX 3060 Ti could occasionally tolerate. It aimed to build a training engine whose behavior remained inspectable enough to justify its claims.

Ethical issue	Rejected alternative	Adopted decision	Justification
Environmental cost	Expand experimentation through rented cloud capacity	Use already available single-device hardware and keep runs bounded	Reduces pressure toward exploratory overuse while preserving the resource-aware research question

Access and misuse	Treat openness as sufficient by itself	Present the engine with narrow scope, limitations, and reproducible evidence	Improves educational access without pretending that broader availability has no risk
Reliability	Promote isolated speed wins or enable aggressive generation paths by default	Accept optimizations only after correctness, end-to-end timing, and diagnostic checks	Keeps the complete runtime trustworthy near the memory limit
Experimental validity	Treat reward success as broad evidence of reasoning quality	Restrict conclusions to verifier-centered mathematical tasks	Avoids overclaiming from a narrow proxy signal
Privacy and data governance	Present local execution as a privacy guarantee	Describe it as limited data autonomy plus retained operator responsibility	Keeps residual obligations around logs, checkpoints, licenses, and storage visible
Architectural responsibility	Delegate critical runtime behavior to higher-level abstractions	Keep explicit control over LoRA, phase separation, memory release, checkpoints, and backend promotion	Makes failure diagnosis and design ownership clearer

Table 10.1: Ethical alternatives, adopted decisions, and justifications.

These decisions also connect the project to SDGs 12 and 13, but only in a restrained sense [22]. The project supports responsible consumption of compute by making resource cost visible and by rejecting unnecessary escalation. It does not claim that consumer-GPU training is always more sustainable than centralized infrastructure. Datacenters may have better utilization, while cloud convenience may encourage more exploratory waste. The ethical claim is about proportionality, not universal superiority.

10.3 APPLICATION TO THE ENGINEERING WORK

The ethical decisions shaped the implementation. The 8 GB VRAM limit forced the project to justify each extra allocation and each more expensive experiment. Quantized backbone loading, manual

LoRA integration, selective checkpointing, explicit memory release, bounded benchmark suites, and conservative operating defaults were not only feasibility mechanisms. They also reduced the chance of spending compute on fragile configurations whose behavior could not be explained.

The benchmark policy followed the same logic. Candidate optimizations were not accepted because they produced an isolated local improvement. They had to preserve correctness, survive end-to-end measurement, and remain diagnosable under the operating conditions that mattered for the project. This is why selective acceleration was preferred to blanket acceleration and why backend promotion was tied to full-loop evidence rather than headline throughput alone.

OOM behavior was treated as part of the operating envelope instead of being hidden as an embarrassing failure case. Backoff rules, checkpoint-aware recovery, and conservative defaults make the system more honest: the designer remains responsible for behavior under stress, not only for the best benchmark window. This point is central to the engineering perspective, because the contribution is not simply that GRPO-style training can run on a small GPU, but that it can be made observable and recoverable enough to study.

The same restraint applies to evaluation. The project limits its strongest claims to the verifier-centered task family actually tested. It does not present reward improvement on any single math benchmark as proof of general intelligence, safety, or fairness. Broader deployment risks are discussed separately from the narrower evidence produced by the engine. That boundary protects the technical contribution because it makes the claims easier to defend.

Data autonomy is handled in the same operational way. Local execution avoids unnecessary external transmission during routine experimentation, but the resulting artifacts still require care. Logs, prompts, checkpoints, benchmark outputs, model licenses, and dataset provenance remain part of the project's ethical surface. Since the project uses public technical datasets and model checkpoints rather than human participant data, the main duty is not human-subject protection but responsible artifact governance and accurate reporting.

10.4 USE OF AI ASSISTANCE AND ACADEMIC INTEGRITY

AI assistance was used as part of the engineering workflow. It supported code exploration, debugging, command preparation, interpretation of long logs, comparison of implementation alternatives, and drafting support. This use did not replace engineering judgment. A suggested fix, explanation, or command was useful only when it could be checked against the codebase, tests, benchmark artifacts, profiler output, logs, or preserved configuration files.

The safeguards were authorship, verification, and technical traceability. AI-generated text was not treated as final academic prose: any generated wording used during drafting was revised, rewritten, and checked by the author before inclusion. AI tools were not used as bibliographic authority.

References, implementation claims, performance conclusions, and ethical arguments were checked against papers, documentation, source code, benchmark outputs, and project artifacts.

Using an assistant to inspect logs or explore alternatives is acceptable only if the final technical responsibility remains with the author. The design decisions, accepted optimizations, rejected alternatives, and conclusions therefore remain tied to evidence that can be inspected independently of the assistant's suggestions.

10.5 ETHICAL ASSESSMENT

The ethical assessment rests on discipline under constraint. Lowering the cost of participation is useful when it expands access without hiding misuse risk. Efficiency work matters when it reduces waste without turning the system into an opaque benchmark artifact. Local execution improves autonomy when it is not misrepresented as a complete privacy solution. Verifiable rewards are useful when they are treated as narrow instruments rather than as full measures of model quality.

Under those conditions, the project supports a defensible ethical position. It advances local access to post-training methods, but only with bounded claims, explicit limitations, and responsibility for the operating risks introduced by the design. It treats sustainability as proportionality, reliability as professional duty, and evaluation as evidence that must stay inside its real scope. Local post-training is responsible here because the project keeps its costs, risks, and claims visible.

11. CONCLUSIONS, LIMITATIONS, AND FUTURE WORK

This work began from a narrow but demanding engineering question: whether GRPO-style post-training for a reasoning-oriented language model could be implemented, inspected, and operated locally on a single consumer GPU with 8 GB of VRAM, without turning the result into an opaque demonstration or a benchmark artifact that only works under one favorable condition. The final answer is qualified but positive. The project did not produce a universal recipe for local reinforcement learning, and it did not remove the structural cost of grouped generation near the memory boundary. Its contribution is more specific: it establishes a complete engineering route for making this form of training understandable enough to build, diagnose, constrain, and defend on hardware that leaves very little unused capacity.

The main engineering difficulty was never only to make the model fit. The harder requirement was to preserve responsibility for the system after feasibility had been reached: the phases had to remain separable, memory pressure had to remain observable, failures had to remain recoverable, and performance improvements had to be accepted only when they strengthened the complete runtime. The result is therefore not a smaller copy of a larger training stack. It is a constrained systems-engineering contribution defined mainly by the discipline imposed on the implementation, rather than by the bare fact that the implementation can execute.

The practical outcome is clearer when separated into three connected layers:

- Architecturally, the native execution model exposes the main costs of GRPO training instead of delegating them to a high-level trainer.
- Operationally, the system runs inside a narrow but coherent hardware envelope without hiding the cost of that envelope.
- Methodologically, design choices became final only after tests, benchmarks, diagnostics, and comparisons against plausible alternatives.

Together, these layers define the contribution more accurately than any isolated throughput number. The work demonstrates a defensible way of engineering under constraint, not a general recipe for every model, dataset, or device.

11.1 ENGINEERING CONTRIBUTIONS AND VERIFIED OUTCOMES

The objectives defined in Section 1.4 were fulfilled with different degrees of generality.

- **O1 was achieved.** The project produced a native PyTorch GRPO-style training engine that can execute local post-training on the target consumer GPU. The result is more than a one-off

script: the main execution stages are explicit, the trainable surface is intentionally limited, and the complete loop can be inspected phase by phase.

- **O2 was achieved within a bounded operating profile.** The system improved end-to-end efficiency through selective Triton acceleration, response-cap analysis, benchmark-governed promotion, and rejection of local speedups that did not survive full-loop comparison. The objective was not fulfilled as a universal performance recipe; it was fulfilled as a defended operating point for the tested hardware and workload.
- **O3 was achieved within the target hardware scope.** The runtime fits and operates under the 8 GB constraint by combining a quantized backbone, LoRA-only training, explicit tensor lifetime control, pressure-sensitive checkpointing, and phase-aware recovery. The work does not remove the memory boundary, but it makes that boundary manageable enough for repeated bounded execution.
- **O4 was achieved.** The implementation exposes direct control over LoRA behavior, checkpointing, phase transitions, memory cleanup, profiler hooks, benchmark paths, and recovery logic. That control makes the system more transparent than a high-level trainer for the specific engineering questions studied here.

These contribution areas are summarized in Table 11.1; the following prose explains why their combination matters.

Contribution	Mechanism in the project	Bound of the claim
Native local GRPO engine	Generation, scoring, update, cleanup, checkpointing, and recovery remain explicit stages.	The result validates a native single-GPU route for the tested hardware and workload, not a universal GRPO engine.
Phase-aware recovery design	Phase-aware retries protect prepared state and reduce attempted work without changing step meaning.	The design manages memory failure as a bounded operating condition; it does not remove the OOM boundary.
Benchmark-governed optimization workflow	Correctness checks, diagnostic runs, bounded comparisons, and promotion rules govern complete-system changes.	The workflow supports defensible local decisions, but remains tied to measured benchmark windows and preserved diagnostics.

Selective Triton specialization	Triton remains on regular tensor paths that survive correctness checks and full-loop comparison.	Specialization is useful only where shapes, correctness risk, and end-to-end cost justify it.
Legible design responsibility	Architecture, manual LoRA/adaptor path, phase-explicit runtime, recovery policy, and benchmark gates remain visible design choices.	The contribution is traceable engineering control under one 8 GB hardware budget and one operating objective, not novelty claims for every technique.

Table 11.1: Verified engineering contributions and claim boundaries.

The table separates the implemented mechanisms from the scope of the claims they support. Memory residency, phase boundaries, and recovery behavior are treated as design constraints that can be inspected across the loop, not as incidental details discovered only after failure. Because OOM remains expected near the boundary, the contribution is bounded recovery semantics rather than an absence of memory failure. This phase visibility matters because a change in one stage can be traced to its effect on the complete runtime. Near the memory boundary, recovery also protects the meaning of a training step: a transient OOM should not silently become corrupted state or an uncontrolled schedule change.

Benchmark governance matters for the same reason, since constrained hardware can make a local speedup look useful until correctness, stability, and full-loop cost are checked together.

Triton was used as a middle ground between plain torch execution and full custom CUDA work, not as a blanket replacement strategy. The contribution is not that each technique is new in isolation. Nor is the result merely a set of borrowed defaults assembled under a new name. Ideas from the wider language-model training ecosystem were translated into a locally executable system, while paths that did not survive the constraint regime were rejected and documented.

This boundary is part of the result: it identifies where low-level engineering remains worthwhile on this hardware class and where it adds complexity without enough end-to-end return.

Native execution, phase-aware recovery, benchmark governance, selective Triton use, and legible design responsibility form the final engineering contribution set. These mechanisms matter as much for how they are governed under constraint as for what they do individually: native control keeps the runtime inspectable, recovery gives memory failure bounded semantics, benchmark authority turns local experimentation into defensible engineering decisions, and selective specialization keeps low-level work tied to evidence.

The evidence discipline behind these contributions is itself part of the result. Kernel specialization, response budgeting, grouped rollout cost, and accumulation policy were not treated as independent knobs whose best local values could be combined mechanically. Each decision had to survive as part of the whole loop. Short measurements near the memory boundary can hide secondary costs: longer diagnostics may expose truncation drift, allocator pressure, recovery frequency, or correctness risk. Rejected optimizations also helped define the boundary of useful specialization. The project therefore contributes a selection process as much as a set of selected values, because the process prevents local wins from being mistaken for complete-system improvements.

The main conclusion from these contributions is bounded but strong. Local GRPO-style post-training on a single consumer GPU is feasible when the system is engineered around explicit control, narrow operating assumptions, and evidence-backed selection. Feasibility alone would be too weak a target. A useful local engine must remain understandable after it becomes runnable: its failures must be interpretable, its optimization choices must be auditable, and its limits must stay visible. The final evidence is not a single isolated benchmark line, but the convergence of a native phase-explicit engine, an accepted 8 GB operating point, recovery-aware execution, and a selective acceleration policy that survived correctness checks and full-loop comparison. Within the available evidence, the final system reaches that stronger standard.

11.2 BOUNDED CLAIMS, LIMITATIONS, AND THREATS TO VALIDITY

The conclusions of this work are intentionally narrower than the implementation effort might suggest. The engine does not define the best general way to perform reinforcement learning for language models. Its strongest claim is narrower: for the hardware tier and task regime studied here, a native and benchmark-governed implementation can produce a viable local training runtime with explicit operational control. This qualification matters most for O2 and O3, because efficiency and memory robustness were validated inside a specific hardware, model, and response-length envelope. Broader interpretations require separate evidence.

Limitation	Effect on generalization	How the claim is bounded
Single RTX 3060 Ti-class hardware scope with 8 GB of VRAM	VRAM, bandwidth, and device behavior shape the envelope; roomier GPUs may favor different policies.	The final preset is treated as a validated local operating point rather than as a universal default.

Specialized operating profile	Evidence is tied to one model scale, adapter strategy, verifier-centered reward, and response-length regime.	Broader model families, verifier designs, reward models, and response structures require separate validation.
Dataset-transfer boundary	Broader normalized math datasets may bring rollout lengths or reward distributions that do not fit the 768-token cap.	Structural dataset support is weaker evidence than validated transfer, so each new dataset family requires separate testing.
Bounded evaluation breadth	Reruns across seeds, response policies, and hardware states were limited; local reinforcement-learning behavior remains path-dependent.	The conclusions rely on structured comparisons, diagnostics, and preserved operational traces, not exhaustive statistical coverage.
Omitted reference-model-heavy regularization	A reference-model-heavy path would exceed the comfortable 8 GB budget.	The claim concerns the selected low-VRAM design, not the full set of reinforcement-learning regularization strategies.
Binary verifier-centered reward scope	Mathematical answer checking fits the target task, but is narrower than richer alignment or preference-learning settings.	The results support narrow verifiable reasoning experiments, not broad fairness, safety, or general-reasoning claims.
Integrated stack coupling	Memory policy, response length, backend policy, and grouped rollouts influence each other.	The accepted preset is a joint operating solution, not the isolated optimum of each parameter.

Table 11.2: Main limitations and their effect on the conclusions.

The implementation is no longer structurally limited to GSM8K, but the strongest validated evidence still comes from the original benchmark-centered operating envelope. This specialization is not a

defect in the result; it is part of what makes the claims defensible, because the project only generalizes where the evidence supports generalization.

The 8 GB constraint is therefore both a threat to generalization and the source of the project's engineering relevance, because the contribution is measured by how much control, robustness, and evidence discipline can be preserved under that budget.

A configuration that is defensible on this device may not be optimal on a larger GPU, while a policy that looks too fragile here may become acceptable when more headroom is available.

These limits also interact with each other. Hardware pressure shapes the accepted response length, response length affects rollout and reward behavior, and both influence which optimization results remain valid. For that reason, the final preset is a joint operating solution rather than the independent optimum of each individual parameter.

The omitted reference-model-heavy path and the verifier-centered reward design should be read as deliberate engineering trade-offs under the target hardware budget. They keep the local runtime feasible, but they also limit transfer to heavier regularization regimes and richer alignment settings.

These limitations lead to three main threats to validity:

- Some accepted performance conclusions depend on bounded benchmark windows interpreted through longer but still selective diagnostics. This evidence was appropriate for the project constraints, but a broader endurance budget could still change the ranking among closely matched alternatives.
- The evaluation studies a tightly integrated runtime, so changing one policy may alter the effect of another. This makes isolated parameter claims weaker than complete-system claims.
- The final system may look conservative relative to the amount of low-level optimization work carried out during the project. That conservatism follows from the acceptance criteria: local speedups were rejected when they weakened correctness, recoverability, or stability, or when they failed to improve the end-to-end operating profile.

None of these limitations overturn the main conclusions. They define their correct scope. The project does not claim that constrained local reinforcement learning becomes easy once a quantized model and adapters are in place. It claims that explicit systems design, disciplined benchmarking, and recovery-aware execution can make GRPO-style training viable enough to study and operate meaningfully on hardware that would normally be considered too limited for this class of work.

11.3 FUTURE WORK PRIORITIZED BY EXPECTED VALUE

The most useful future work starts from the operating decisions already established. The project narrowed the design space to a small set of defensible choices, so the next steps should test those choices more deeply rather than replace them with another broad search. Generation remains the main runtime hotspot, but long-run validation comes first because it strengthens the evidence for the accepted operating profile before more complex acceleration work is attempted. Dataset validation is also needed because the limitation analysis shows that structural dataset support is not the same as demonstrated transfer.

1. Strengthen long-run validation of the accepted preset. The current evidence is sufficient to defend the accepted profile as a workable local default; longer validation would deepen confidence and map its limits rather than make the conclusion possible for the first time. Longer runs and a broader rerun set would help separate persistent behavior from bounded diagnostic behavior, especially in late truncation drift, recovered OOM frequency, allocator instability, and step-time inflation. This line extends O2 and O3 without reopening settled design choices.
2. Target the generation path without accepting isolated speedups. Generation remains the dominant runtime cost and the part of the system where local optimization was hardest to convert into end-to-end benefit. Future work in this line would test whether any new generation path survives correctness checks, full-loop comparison, and longer operational diagnostics strongly enough to replace the current default.
3. Broaden normalized math-dataset validation. The data path can support broader mathematical datasets, but structural support is not the same as validated transfer. Future work should test whether the accepted response cap, verifier design, rollout behavior, and memory policy remain coherent when the dataset changes. Given this limitation, the value would be evidentiary rather than architectural: it would justify broader generalization claims without pretending to add a larger implementation contribution.
4. Refine update-side kernels where specialization has already shown value. The benchmark record supports additional selective Triton work more strongly than a broad attempt to replace the remaining torch execution paths. The best candidates are regular tensor operations with manageable correctness surfaces and persistent measured cost. Deeper revisions of existing kernels should therefore be justified by repeated cost evidence, because the implementation burden only pays off when the full loop preserves correctness and stability.
5. Test additional hardware tiers. The current conclusions are tied to one constrained GPU class, and its thresholds and trade-offs should not be assumed to transfer unchanged. Running the accepted engine on at least one roomier GPU, and if possible on one similarly constrained but architecturally different consumer device, would clarify which limits belong to the implementation and which belong mainly to the original hardware budget. These

comparisons should reuse the same acceptance logic rather than become a broad retuning exercise. This line tests the portability of O3 and the practical generality of O4.

6. Refine compilation policy for stable critical paths. This line is distinct from adding more selective Triton kernels: it targets repeated paths whose shapes and correctness surface are already well understood, where ahead-of-time compilation, compilation policy, or lower-level code may reduce cost without reopening the full runtime design. The same acceptance standard remains necessary: a faster hotspot is not a meaningful improvement if it narrows the operating envelope or weakens the reliability of the complete runtime.
7. Extend to larger execution models once the single-device path is mature. Multi-GPU execution, lane separation across devices, or other parallel strategies could later relax some of the wall-clock and memory constraints that shaped the current system. These extensions should preserve observability, recoverability, and comparability; otherwise, distributed execution would risk reintroducing the opacity that the native single-device design was built to avoid.

These future-work lines point toward consolidation. The main result is not an open search space, but a small set of engineering decisions that proved defensible under hard hardware constraints. The system can evolve by tightening the evidence around that operating profile and specializing only where the full loop keeps justifying the added complexity.

The ethical conclusion is consistent with that boundary. The work supports wider access to advanced post-training methods, but only under bounded claims and explicit operational discipline. Local execution improves autonomy without eliminating privacy obligations. Efficiency work reduces waste only when it is tied to observability and recoverability rather than to speed in isolation. Verifier-centered rewards remain useful for narrow mathematical tasks, but they do not justify broader claims about fairness, safety, or general reasoning quality. AI assistance requires the same discipline: it is acceptable when it supports inspection, debugging and documentation, not when it replaces validation, authorship or technical responsibility. The responsible outcome is a constrained and transparent operating model whose limits remain visible.

Restricted hardware does not make local reinforcement learning simple, but it can still be handled with discipline. With deliberate systems design and strict benchmark governance, GRPO-style training on an 8 GB GPU can be organized as a constrained, observable, and recoverable training engine rather than as a fragile experiment. Beyond the immediate implementation, this keeps advanced post-training study within reach of local hardware while refusing to hide the limits that make such access technically difficult. The result is a technical foundation with narrow claims, explicit limits, and future improvements that can be judged by the same evidence standard that produced the system.

12. BIBLIOGRAPHY

- [1] D. Guo, D. Yang, H. Zhang, et al., "DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning," *Nature*, vol. 645, 633-638, 2025. <https://doi.org/10.1038/s41586-025-09422-z>
- [2] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, et al., "DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models," *arXiv preprint arXiv:2402.03300*, 2024. <https://arxiv.org/abs/2402.03300>
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," *arXiv preprint arXiv:1707.06347*, 2017. <https://arxiv.org/abs/1707.06347>
- [4] MSR Avinash, "Profiling LoRA/QLoRA Fine-Tuning Efficiency on Consumer GPUs: An RTX 4060 Case Study," *arXiv preprint arXiv:2509.12229*, 2025. <https://arxiv.org/abs/2509.12229>
- [5] S. Wang, J. Asilis, O. F. Akgul, E. B. Bilgin, O. Liu, and W. Neiswanger, "Tina: Tiny Reasoning Models via LoRA," *International Conference on Learning Representations (ICLR 2026)*, 2026. <https://openreview.net/forum?id=P2OXYO3bEe>
- [6] Z. Bi, X. Chen, L. Sun, Y. Yao, Q. Shen, J. Lou, and C. Deng, "RooflineBench: A Benchmarking Framework for On-Device LLMs via Roofline Analysis," *arXiv preprint arXiv:2602.11506*, 2026. <https://arxiv.org/abs/2602.11506>
- [7] NVIDIA, "CUDA Programming Guide," <https://docs.nvidia.com/cuda/cuda-programming-guide/> (accessed May 16, 2026).
- [8] T. Zadouri, M. Hoehnerbach, J. Shah, T. Liu, V. Thakkar, and T. Dao, "FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling," *arXiv preprint arXiv:2603.05451*, 2026. <https://arxiv.org/abs/2603.05451>
- [9] B. Kong, W. Jia, E. Zhang, G. Li, Y. Dong, Y. Wang, Y. Wang, Y. Peng, and K. Yuan, "Pre-training Large Language Models with Dynamic Precision: Low-Cost Computation with High-Fidelity Performance," *ICLR 2026 DeLTa Workshop*, 2026. <https://openreview.net/forum?id=2oXPKzs2iz>
- [10] T. Dao, D. Fu, S. Ermon, A. Rudra, and C. Re, "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness," *Advances in Neural Information Processing Systems*, vol. 35, 16344-16359, 2022. https://proceedings.neurips.cc/paper_files/paper/2022/hash/67d57c32e20fd0a7a302cb81d36e40d5-Abstract.html

- [11] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," *Advances in Neural Information Processing Systems*, vol. 36, 10088-10115, 2023.
https://proceedings.neurips.cc/paper_files/paper/2023/hash/1feb87871436031bdc0f2beaa62a049b-Abstract-Conference.html
- [12] R. Prabhu, A. Nayak, J. Mohan, R. Ramjee, and A. Panwar, "vAttention: Dynamic Memory Management for Serving LLMs without PagedAttention," *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1 (ASPLOS '25)*, 1133-1150, 2025.
<https://doi.org/10.1145/3669940.3707256>
- [13] J. Zhou, H. Yang, S. J. Tang, M. Xiang, H. Guan, and T. Liu, "Understanding and Alleviating Memory Issue in RLHF for LLMs," *Machine Learning for Systems Workshop at NeurIPS 2024*, 2024. <https://mlforsystems.org/assets/papers/neurips2024/paper8.pdf>
- [14] A. S. Gautam and S. Jha, "RAMP: Reinforcement Adaptive Mixed Precision Quantization for Efficient On-Device LLM Inference," *arXiv preprint arXiv:2603.17891*, 2026.
<https://arxiv.org/abs/2603.17891>
- [15] X. Yan, C. Bao, Z. Li, T. Zhang, S. Zhang, R. Xie, S. Sun, and Y. Zhang, "D²Quant: Accurate Low-Bit Post-Training Weight Quantization for LLMs," *arXiv preprint arXiv:2602.02546*, 2026.
<https://arxiv.org/abs/2602.02546>
- [16] S. Luo, X. Zhang, Y. Hu, B. Zhang, K. Wang, J. Su, M. Sun, L. Liang, and J. Zhang, "Sparse-RL: Breaking the Memory Wall in LLM Reinforcement Learning via Stable Sparse Rollouts," *arXiv preprint arXiv:2601.10079*, 2026. <https://arxiv.org/abs/2601.10079>
- [17] "Reinforcement Learning (RL) Guide," <https://unsloth.ai/docs/get-started/reinforcement-learning-rl-guide> (accessed May 10, 2026).
- [18] "Tutorial: Train your own reasoning model with GRPO," <https://unsloth.ai/docs/get-started/reinforcement-learning-rl-guide/tutorial-train-your-own-reasoning-model-with-grpo> (accessed May 10, 2026).
- [19] S. Ono, J. Ackermann, S. Nishimori, T. Ishida, and M. Sugiyama, "Mitigating Reward Hacking in RLHF via Advantage Sign Robustness," *arXiv preprint arXiv:2604.02986*, 2026.
<https://arxiv.org/abs/2604.02986>
- [20] "autoresearch README," <https://github.com/karpathy/autoresearch/blob/master/README.md> (accessed May 10, 2026).

- [21] J. Morrison, C. Na, J. Fernandez, T. Dettmers, E. Strubell, and J. Dodge, "Holistically Evaluating the Environmental Impact of Creating Language Models," *International Conference on Learning Representations (ICLR 2025)*, 2025. <https://openreview.net/forum?id=04qx93Viwj>
- [22] United Nations Department of Economic and Social Affairs, "The Sustainable Development Goals Report 2025," *United Nations*, 2025. <https://unstats.un.org/sdgs/report/2025/The-Sustainable-Development-Goals-Report-2025.pdf>
- [23] P.-C. Pan, Y. Liang, and S. Lin, "Reward Modeling for Reinforcement Learning-Based LLM Reasoning: Design, Challenges, and Evaluation," *arXiv preprint arXiv:2602.09305*, 2026. <https://arxiv.org/abs/2602.09305>
- [24] K. Zhang, Y. Zuo, B. He, et al., "A Survey of Reinforcement Learning for Large Reasoning Models," *arXiv preprint arXiv:2509.08827*, 2025. <https://arxiv.org/abs/2509.08827>

Abbreviations

AI: Artificial Intelligence

API: Application Programming Interface

BF16: BFloat16

CLI: Command-Line Interface

CPU: Central Processing Unit

CRISP-DM: Cross-Industry Standard Process for Data Mining

CSV: Comma-Separated Values

CUDA: Compute Unified Device Architecture

DRAM: Dynamic Random-Access Memory

FDP: Final Degree Project

FLOPS: Floating-Point Operations Per Second

FP16: 16-bit Floating Point

GB: Gigabyte

GPU: Graphics Processing Unit

GNMR: Gradient Norm Ratio

GRPO: Group Relative Policy Optimization

GSM8K: Grade School Math 8K

HBM: High-Bandwidth Memory

INT4: 4-bit Integer

JSON: JavaScript Object Notation

JSONL: JSON Lines

KL: Kullback-Leibler Divergence

KV: Key-Value

L2: Level 2

LLM: Large Language Model

LoRA: Low-Rank Adaptation

ML: Machine Learning

NF4: NormalFloat 4-bit

OOM: Out of Memory

Open-RS: Open Reasoning in Small LLMs

PEFT: Parameter-Efficient Fine-Tuning

PPO: Proximal Policy Optimization

QLoRA: Quantized Low-Rank Adaptation

RAMP: Reinforcement Adaptive Mixed-Precision Quantization

RL: Reinforcement Learning

RLHF: Reinforcement Learning from Human Feedback

RNG: Random Number Generator

SDG: Sustainable Development Goal

SENT: Semantic-Entropy Curriculum Learning

TF32: TensorFloat-32

TRL: Transformer Reinforcement Learning

VRAM: Video Random-Access Memory

W&B: Weights & Biases

Appendix A: Use of AI assistance

This appendix records how I used Codex, Gemini Deep Research and NotebookLM during the Computer Engineering FDP. They were part of the working process, mainly for inspection, organization and revision. They did not replace the engineering work, the validation work or my responsibility for the final text.

The main form of day-to-day assistance was Codex in the working environment. Gemini Deep Research and NotebookLM were used more narrowly to organize background material, compare notes and identify claims that needed manual checking. I limited all of these uses to tasks that could be checked against project evidence: source code, tests, benchmark artifacts, profiler output, logs, configuration files, papers and official documentation. I did not treat suggested wording as final academic prose. When I used drafting support, I reviewed the text, rewrote it where necessary and checked it before including it in the memory.

No technical conclusion in the memory depends on the assistant as an authority. Claims about implementation behavior, performance, memory use, failure handling and ethical implications remain tied to project evidence and cited sources. Bibliographic entries were prepared and checked separately from the assistant's suggestions.

AI tool	FDP stage	Purpose of use	Human verification and boundary
Codex	Code exploration and system understanding	Locating relevant modules, following control flow, comparing implementation alternatives and summarizing long source files.	I checked useful suggestions against the source code and kept them only when they matched the implemented behavior.
Codex	Debugging and command preparation	Preparing shell commands, checking failure hypotheses and organizing debugging steps during memory, training and benchmark work.	I executed and inspected commands myself, then validated fixes through tests, logs, preserved artifacts and observed runtime behavior.

Codex	Profiler, log and benchmark interpretation	Reading long profiler outputs, benchmark summaries and training logs, then deciding which evidence mattered for the engineering argument.	I verified performance and memory claims against measured outputs, benchmark files, profiler traces and configuration details.
Codex, Gemini Deep Research and NotebookLM	Technical writing and revision	Reorganizing prose, reducing repetition and keeping explanations focused on the Computer Engineering contribution.	I reviewed and rewrote assisted wording. I did not use the assistant as a source for factual claims.
Codex, Gemini Deep Research and NotebookLM	Consistency and source-check flags	Finding repeated wording, missing cross-references, terminology drift and technical claims that needed manual checking.	I checked the actual sources myself and prepared the bibliography separately. The assistant was not used to write bibliography entries or as authority for citations.

Table A.1: AI assistance used in the Computer Engineering FDP.

The final responsibility for the memory remains mine. The assistant helped me inspect and organize parts of the engineering workflow, but it did not decide which implementation choices were correct, which optimizations were accepted, which evidence was sufficient or which conclusions were justified.

Appendix B: Repository and experiment tracking links

This appendix gives the external access points for the implementation repository and the experiment tracking workspace used during this FDP.

Resource	Link	Role in the FDP
GitHub repository	https://github.com/eblancocabana/grpo-training-engine	Public source repository for the GRPO training engine, including the implementation, benchmark utilities, configuration entry points and project.
Weights & Biases project	https://wandb.ai/e-blanco-universidad-de-deusto/grpo-training	Experiment tracking workspace for training runs, metric curves and run-level metadata associated with the project.

Table B.1: Public repository and experiment tracking links.

Weights & Biases was used for online run tracking, apart from the local artifacts preserved during the project.